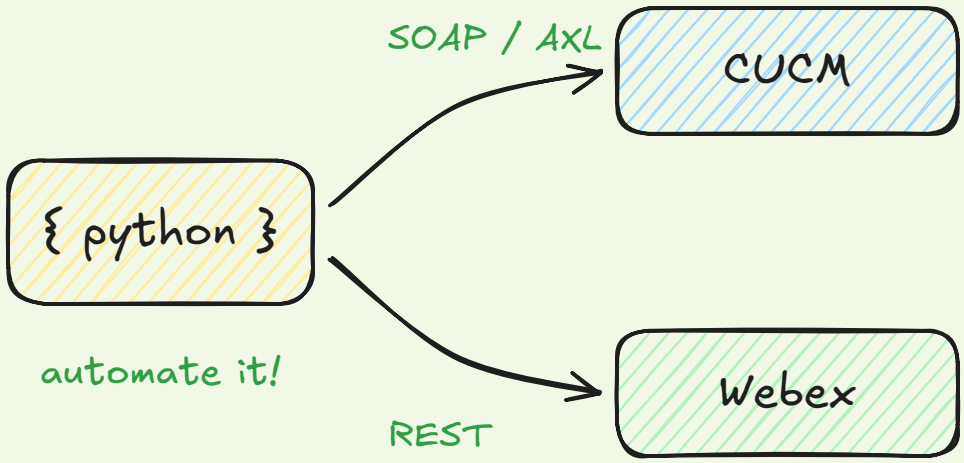
clauto



EĞITIM KITABI

Cisco Collaboration Automation

Ses ve collaboration mühendisleri için sıfırdan Python

Ilker Mansur - CCIE 63555

2026

İçindekiler

1 Python Fundamentals 1
1.1 Why Python? 2
1.2 Variables and Objects 3
1.3 Data Types 5
1.3.1 int 8
1.3.2 float 9
1.3.3 str 11
1.3.4 bool 14
1.3.5 list 15
1.3.6 tuple 18
1.3.7 range 19
1.3.8 dict 20
1.3.9 set 23
1.3.10 bytes 25
1.3.11 None 27
1.4 if Statements 28
1.5 Loops 31
1.5.1 for 32
1.5.2 while 34
1.6 Functions 36
1.6.1 args and kwargs 39
1.6.2 Modules and import 41
1.7 Exceptions 45
1.8 Files 48
1.8.1 CSV 50
1.8.2 JSON 52
1.9 Comprehensions 53
1.10 Classes 56
1.11 Credentials 59
2 APIs and Libraries 62
2.1 HTTP Basics 63
2.2 JSON and XML 65
2.3 requests 69
2.4 zeep and AXL 73
2.5 RisPort70 83
2.6 Netmiko 86
2.7 CMS REST API 90
2.8 Remote Phone Control 94
2.9 Wrap-up 97

Bu Kitap Hakkında

Bu kitap, **ses ve collaboration mühendisleri** için yazıldı: CUCM'de partition ve CSS tasarlayan, CUBE üzerinde dial-peer yazan, Webex Calling organizasyonlarını yöneten ama hayatında hiç Python yazmamış kişiler için. Programcı olmanız gerekmiyor. Python'dan önce başka bir dil bilmenize de gerek yok. Tek ön koşul, kendi alanınızı bilmeniz: DN nedir, bir phone CUCM'e nasıl register olur, AXL'in ne işe yaradığı gibi şeyler sizin için zaten tanıdık.

Bu kitabın temel fikri şu: **tekrar eden her işi bir bilgisayara yaptırabilirsiniz.** 500 phone eklemek, 3000 DN'in partition'ını kontrol etmek, her sabah hangi cihazların register olmadığını raporlamak. Bunları tek tek tıklayarak yapmak yerine kısa bir Python script'i yazarsınız ve script bunu saniyeler içinde, hatasız ve her seferinde aynı şekilde yapar.

Kitap nasıl ilerliyor?

Kitap iki bölümden oluşuyor ve ikisi de bir önceki üzerine kurulu:

1. **Python Fundamentals:** dilin kendisi. Veri tipleri, karar vermek (`if`), tekrar etmek (loop), kod parçalarını isimlendirmek (function), hata yönetimi, dosyalar ve parolaların güvenli saklanması. Her konu, collaboration dünyasından bir örnekle anlatılıyor.
2. **APIs and Libraries:** Python'u Cisco sistemlerine bağlamak. HTTP, JSON ve XML, `requests` ve `zeep` kütüphaneleri, CUCM'in AXL API'si, RisPort70 ile cihaz durumu, Netmiko ile CLI, CMS REST API'si ve uzaktan phone kontrolü.

Birinci bölümü atlamayın. İkinci bölümdeki her örnek, birinci bölümde öğrendiğiniz parçalardan oluşuyor. Bir AXL isteğinin neden bir `dict` ile hazırlandığını, cevabın neden iç içe geçmiş list ve dict'lerden oluştuğunu ancak o parçaları bildiğinizde rahatça okuyabilirsiniz.

Her konu aynı yolu izler

Kitaptaki her konu başlığı aynı sırayla ilerler, böylece ne bekleyeceğinizi bilirsiniz:

1. **Önce neden?** Konunun hangi probleme çözüm olduğu ve çoğu zaman ses dünyasından tanıdık bir benzetme.
2. **Sonra ne?** Kavramın sade bir anlatımı ve bir diyagram.
3. **Kod.** Kısa, çalıştırılabilir bir örnek. Satır numaraları solda durur.
4. **Satır satır okuma.** Kodun her satırının ne yaptığı, sırayla.
5. **Tuzaklar ve gerçek hayat.** Sık yapılan hatalar ve konunun CUCM, Webex ya da CUBE tarafındaki karşılığı.

Her section'ın sonunda kısa bir **Özetle** kutusu ve bir **Kendin dene** alıştırması bulunur. Okumak yetmez: kodu kendi bilgisayarınızda yazıp çalıştırın, değerlerini değiştirin, bozun ve düzeltin. Programlama izleyerek değil, deneyerek öğrenilir.

Dil hakkında bir not

Metin Türkçe, ama **teknik terimler İngilizce kalıyor**: variable, list, loop, function, exception gibi. Bunun iki nedeni var. Birincisi, internetteki dokümanlar, hata mesajları ve Cisco DevNet örnekleri İngilizce; terimi bildiğiniz gibi görürsünüz. İkincisi, bu terimlerin Türkçe karşılıkları kafa karıştırıcı olabiliyor. Her terim ilk geçtiği yerde **kalın** yazılır ve Türkçe karşılığı parantez içinde verilir: **variable** (değişken). Sonrasında yalnızca İngilizcesi kullanılır ve Türkçe ekler kesme işaretiyle eklenir: list'in, loop'u, function'ı.

Kutuların anlamı

> **≈ BENZETME**
> Ses ve telefoni dünyasından bir benzetme. Yeni bir kavramı, zaten bildiğiniz bir şeye bağlar.

> **▸ COLLAB DÜNYASINDA**
> Konunun CUCM, CUBE, Webex ya da başka bir Cisco collaboration ürünündeki karşılığı.

> ⋆ İPUCU
> Zaman kazandıran bir kısayol ya da iyi bir alışkanlık.

> ! DIKKAT
> Dikkat edilmezse sürpriz yaratabilecek bir ayrıntı.

> × SIK YAPILAN HATA
> Yeni başlayanların neredeyse hepsinin yaptığı bir hata ve nasıl düzeltileceği.

Kod örneklerinde program çıktısı iki şekilde gösterilir. Kısa sonuçlar, satırın sonunda `#` işaretinden sonra yazılır:

```
2 + 3        # 5
```

Birkaç satırlık çıktılar ise ayrı bir `ÇIKTI` kutusunda gösterilir:

ÇIKTI
```
alice.smith 1001
bob.jones 1002
```

Lab dünyası: örneklerdeki isimler

Kitaptaki bütün örnekler aynı hayali ortamı kullanır. **Hiçbir örnekte IP adresi yoktur**, her zaman isim kullanılır; bu hem okumayı kolaylaştırır hem de gerçek bir sisteme yanlışlıkla bağlanmanızı önler. Bu tabloyu bir kenarda tutun, örneklerde karşınıza çıkacak:

Öğe	Değer
Domain	`clauto.lab` (e-posta ve SIP URI: `alice.smith@clauto.lab`)
CUCM publisher / subscriber	`cucm-pub.clauto.lab`, `cucm-sub1.clauto.lab`
Unity Connection / IM&P	`cuc-pub.clauto.lab`, `imp-pub.clauto.lab`
CUBE	`cube-01.clauto.lab`
CMS	`cms-01.clauto.lab` (API port 445, API kullanıcısı `api_svc`)
Expressway C / E	`expc-01.clauto.lab`, `expe-01.clauto.lab`
CUCM sürümü	15
Webex organizasyonu	`Clauto Lab` (Webex Calling)
DN aralığı	`1001` – `1999` (kullanıcılar), `2000` – `2099` (hunt / CTI)
Phone'lar	`SEP0A1B2C3D4E5F` (8851), `SEP0A1B2C3D4E60` (8865), `SEP0A1B2C3D4E61` (7841)
Jabber	`CSFALICE` (alice.smith), agent'lar `CSFAGENT01`, `CSFAGENT02`
Kullanıcılar	`alice.smith` → 1001, `bob.jones` → 1002, `carol.white` → 1003
Partition / CSS	`PT_Internal`, `CSS_Internal`
Device pool	`DP_HQ`, `DP_Branch`
Hunt pilot	`2000` (Sales)
Codec'ler	`G722`, `G711u`, `G711a`, `OPUS`
AXL application user	`axl_svc`

Python'u çalıştırmak

Kitabı okurken kodları denemek için kendi bilgisayarınızda Python 3 kurulu olmalı (3.10 ya da üstü yeterli). Kurulu olup olmadığını terminalde şöyle kontrol edersiniz:

```
python3 --version
```

Python ile kod çalıştırmanın iki yolu vardır. Bu kitapta ikisini de kullanacağız.

1. Etkileşimli mod (REPL)

Terminalde sadece `python3` yazarsanız, `>>>` işaretiyle bir komut satırı açılır. Buraya yazdığınız her satır hemen çalışır ve sonucu görürsünüz. Bu, bir CUCM ya da CUBE CLI'ında komut yazmaya çok benzer. Küçük şeyleri denemek için idealdir:

ÇIKTI

```
$ python3
>>> 2 + 3
5
>>> "SEP" + "0A1B2C3D4E5F"
'SEP0A1B2C3D4E5F'
>>> exit()
```

Kitaptaki kısa kod parçalarını (özellikle `# sonuç` yorumu olanları) REPL'e satır satır yazarak deneyebilirsiniz. REPL, bir satırın sonucunu otomatik yazdırır. Bu yüzden kitaptaki bazı örneklerde `print()` yoktur: sonuç yorumda gösterilir.

2. Script dosyası

Birkaç satırdan uzun işler için kodu bir dosyaya yazarsınız. Python dosyalarının uzantısı `.py`'dir. Örneğin `hello.py` adlı bir dosyaya şunu yazın:

```
1 print("Merhaba, clauto.lab")
```

Ve terminalde çalıştırın:

```
python3 hello.py
```

Bu, CLI'a tek tek komut yazmakla, aynı komutları bir dosyaya yazıp tek seferde çalıştırmak arasındaki farka benzer. Bir script'i bir kez yazarsınız, istediğiniz kadar çalıştırırsınız. Dosya yönteminde sonucu görmek için `print()` kullanmanız gerekir.

> ⋆ İPUCU
> Kodu **kopyalamak yerine kendiniz yazın**. Parmaklarınızla yazdığınızda hataları fark etmek, sözdizimini (syntax) öğrenmek çok daha kolay olur. Hata mesajı almanız kötü bir şey değil: Python'un neyin yanlış gittiğini size söylemesidir ve 1.7 numaralı bölümde bu mesajları okumayı öğreneceksiniz.

1 Python Fundamentals

1.1 Why Python?
1.2 Variables and Objects
1.3 Data Types
1.4 if Statements
1.5 Loops
1.6 Functions
1.7 Exceptions
1.8 Files
1.9 Comprehensions
1.10 Classes
1.11 Credentials

CUCM ve Webex ile konuşmadan önce Python'un kendisini öğreniyoruz: veri tipleri, karar verme, tekrar etme, function'lar, dosyalar. Bütün örnekler collaboration dünyasından geliyor, böylece sözdizimini ve onun ne işe yaradığını aynı anda öğreniyorsunuz.

BU BÖLÜMDE ÖĞRENECEKLERİNİZ

- Bir değeri bir variable'a bağlamak ve verinin tipini tanımak
- `if` ile karar vermek, loop ile yüzlerce cihaz üzerinde aynı işi yapmak
- Kendi function'larınızı yazmak, hazır modülleri `import` etmek
- Hata mesajlarını (exception) okumak; toplu işte tek bir satırın tüm işi durdurmasını engellemek
- CSV ve JSON dosyalarını okuyup yazmak
- Parolaları koddan ayırıp güvenli saklamak

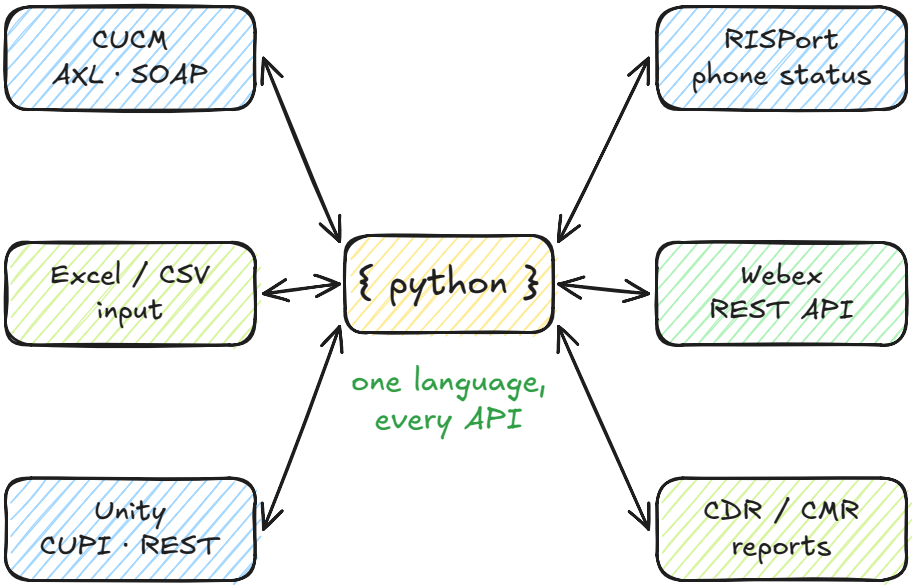
1.1 Why Python?

Python for collaboration engineers

Bir collaboration mühendisinin iş günü büyük ölçüde tekrar eden işlerden oluşur. Yeni başlayan bir çalışan için phone, line, voicemail box ve Webex hesabı açmak. Her sabah hangi cihazların register olmadığına bakmak. Bir CSS değişikliğini kırk device pool'a uygulamak. Bunların her biri tek başına küçüktür, ama 500 kez tekrarlanınca saatler alır ve her tıklamada bir yazım hatası yapma riski vardır.

Bu tür işleri bilgisayara yaptırmanın yolu **program** (ya da **script**) yazmaktır. Bir program, bilgisayara adım adım yazılmış bir talimat listesidir: tıpkı bir call flow diyagramı gibi. "Şunu al, bunu kontrol et, şuna göre şunu yap." Aradaki tek fark, bu talimatları bir insanın değil bilgisayarın uygulamasıdır: 3 phone için de, 3000 phone için de aynı titizlikle.

> **≈ BENZETME**
> CUCM'de **BAT** (Bulk Administration Tool) kullandıysanız mantığı zaten biliyorsunuz: bir şablon, bir CSV dosyası ve tek tıkla toplu iş. Python, BAT'ın sınırlarını kaldırır. Yalnızca CUCM ile değil; Unity Connection, CUBE, CMS ve Webex ile de aynı dilde konuşursunuz. Üstelik BAT'ın yapamadığı şeyleri de ekleyebilirsiniz: "önce kontrol et, sonra karar ver, hata olursa rapora yaz."



Şekil 1.1. Python, collaboration dünyasının bütün API'lerine tek bir dille bağlanır.

Python, okunması kolay, neredeyse İngilizce gibi akan bir sözdizimi (syntax) olan, genel amaçlı (general-purpose) bir programlama dilidir. İki özelliği sizi hemen ilgilendirir:

- **Interpreted** (yorumlanan): Kodu yazarsınız ve çalıştırırsınız, sonucu hemen görürsünüz. Ayrı bir derleme (compile) adımı yoktur.
- **Dynamically typed** (dinamik tipli): Bir değerin ne olduğunu (sayı mı, metin mi) önceden bildirmeniz gerekmez; Python bunu değerden anlar. Bu konuya "Data Types" bölümünde ayrıntılı döneceğiz.

Collaboration'da neden önemli?

- **Ölçek:** 500 kullanıcı ve phone eklemek, 500 ayrı GUI formu yerine tek bir loop'tur.
- **Tutarlılık:** Her seferinde aynı partition, CSS ve device pool atanır. Yazım hatası olmaz.
- **Görünürlük:** Register durumu, envanter ve CDR verisi saniyeler içinde tek bir raporda toplanır.
- **Entegrasyon:** Tek bir dil AXL, RIS, CUPI ve Webex REST API'sinin hepsini konuşabilir.

▸ **COLLAB DÜNYASINDA**
Cisco DevNet'teki AXL ve Webex örneklerinin çoğu Python ile yazılmıştır. Yani Python öğrendiğiniz günden itibaren bu örnekleri okuyabilir ve kendi ortamınıza uyarlayabilirsiniz.

Python neye benziyor?

Henüz hiçbir şey öğrenmeden, küçük bir örneğe bakalım. Bir phone'un register durumunu kontrol eden kısa bir script:

```
1 status = "UnRegistered"
2
3 if status != "Registered":
4     print("UYARI: SEP0A1B2C3D4E5F kapalı")
```

Bunu şimdi tam olarak anlamanız gerekmiyor. Yine de yüksek sesle okuyun: "status, 'UnRegistered' olsun. Eğer status 'Registered' değilse, bir uyarı yazdır." Python'un okunabilirliği bu: kod, bir mühendisin kendi kendine söylediği cümleye çok yakın.

KODU SATIR SATIR OKUYALIM

1. `status` adında bir isim yaratır ve içine `"UnRegistered"` metnini koyar.
3. Koşul: `status`, `"Registered"`'a eşit **değilse**…
4. …ekrana uyarı yazdır. Bu satır içeri girintili (indented) olduğu için yalnızca koşul doğruysa çalışır.

Bu kitabın sonunda, bu küçük kontrolü 500 phone'a uygulayıp sonucu bir CSV dosyasına yazan script'leri kendiniz yazabileceksiniz.

Özetle

- Script, bilgisayara yazılmış adım adım bir talimat listesidir.
- Python okunması kolay, genel amaçlı bir dildir; CUCM, Webex ve diğer Cisco sistemleriyle aynı dilde konuşur.
- Tekrar eden iş, bir loop'a dönüşür: ölçek, tutarlılık ve hız kazanırsınız.

1.2 Variables and Objects

İlk adım: print ve yorumlar

Variable'lara geçmeden önce kodlarda sürekli göreceğiniz iki küçük aracı tanıyalım.

`print()` ekrana bir şey yazdırır. Parantezin içine yazdırmak istediğiniz değeri koyarsınız. `#` işaretinden sonra gelen her şey **yorumdur** (comment): Python onu görmezden gelir, yalnızca sizin ve kodu okuyacak kişilerin içindir. Router konfigürasyonundaki `!` açıklama satırlarına benzer.

```
1 # bu bir yorum: Python bu satırı atlar
2 print("Merhaba, clauto.lab")     # ekrana yazdırır
3 print(1001)                      # sayıları da yazdırabilir
4 print("DN:", 1001)               # virgülle birden fazla değer
```

ÇIKTI

```
Merhaba, clauto.lab
1001
DN: 1001
```

KODU SATIR SATIR OKUYALIM

1. Tamamen yorum. Çalışma üzerinde hiçbir etkisi yok.

2 Tırnak içindeki yazı **metindir**. `print` onu olduğu gibi ekrana yazar. Satır sonundaki yorum da serbesttir.

3 Tırnaksız `1001` bir **sayıdır**. Ekranda aynı görünür, ama Python için farklı bir şeydir. Bu farkı "Data Types" bölümünde göreceğiz.

4 Virgülle ayrılan değerler, aralarına boşluk konularak yan yana yazılır.

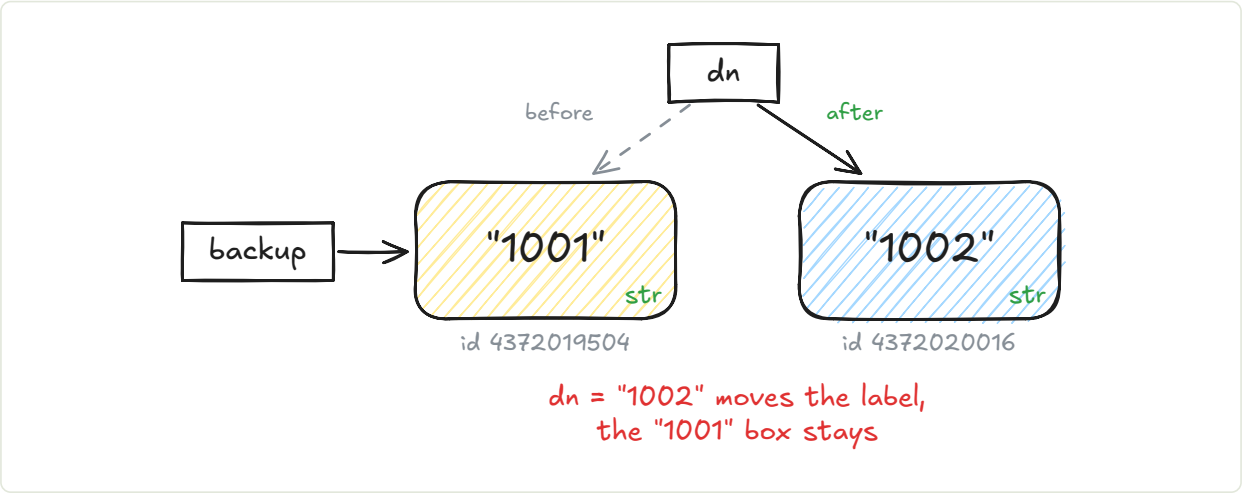
> ⋆ **İPUCU**
> Etkileşimli modda (REPL) `print()` yazmanıza gerek yoktur. Bir ifadeyi yazıp Enter'a bastığınızda sonucu zaten görürsünüz. Dosyadan çalışan script'lerde ise sonucu görmek için `print()` şarttır.

Variables are labels, not boxes

Bir script çalışırken hatırlaması gereken şeyler olur: bir DN, bir device adı, bir hostname. Bunları isimlendirip saklamamızı sağlayan yapıya **variable** (değişken) denir. Ama Python'un variable'ı, çoğu insanın sandığından farklı çalışır ve bu farkı baştan anlamak, ileride yaşayacağınız birçok şaşkınlığı önler.

Değeri bir **kutu** (box) olarak düşünün. Kutunun içinde bir değer vardır (`"1001"` metni gibi) ve kutunun bir tipi vardır (metin, sayı…). Python'da bu kutuya **object** (nesne) denir. Variable ise kutunun kendisi değildir: kutuya **yapıştırılmış bir etikettir**. Python'da işaretçi (pointer) kavramını ayrıca öğrenmeniz gerekmez. Her isim zaten bir object'e işaret eder.

> ≈ **BENZETME**
> Bir phone'a bağlı DN'i düşünün. Phone (kutu) CUCM'de durur; `1001` ise ona yapıştırılmış bir etikettir. Aynı phone'a ikinci bir DN (ikinci etiket) eklenebilir, o zaman iki etiket aynı phone'u gösterir. Bir etiketi başka bir phone'a taşırsanız, eski phone yerinde durmaya devam eder, yalnızca etiket yer değiştirmiştir.



Şekil 1.2. `backup = dn` ikinci bir etiket ekler. `dn = "1002"` ise etiketi yeni bir kutuya taşır.

```
1 dn = "1001"
2 backup = dn           # ikinci etiket, aynı kutu
3 id(dn), id(backup)    # (4372019504, 4372019504)
4 dn is backup          # True
5
6 dn = "1002"           # dn yeni bir kutuya taşınır
7 backup                # '1001'
```

KODU SATIR SATIR OKUYALIM

1 `=` işareti Python'da "eşittir" değil, **"bağla"** anlamına gelir. `"1001"` metninden bir kutu yaratılır ve `dn` etiketi ona yapıştırılır.

2 `backup = dn` değeri **kopyalamaz**. `dn`'in gösterdiği kutuya ikinci bir etiket yapıştırır.

3 `id(x)` kutunun kimlik numarasını verir: object'in ömrü boyunca değişmeyen benzersiz bir sayı. İki etiketin kimliği aynı çıktı, çünkü kutu aynı. (Sizdeki sayı farklı olacaktır.)

4 `a is b` iki etiketin **aynı kutuyu** gösterip göstermediğini sorar. Değerlerin eşit olup olmadığını ise `a == b` sorar. İkisi farklı sorulardır.

6 `dn` etiketi eski kutudan çözülür ve `"1002"` metnini içeren yeni bir kutuya yapıştırılır. Eski kutuya dokunulmaz.

7 `backup` hâlâ eski kutuyu gösterdiği için `'1001'` verir.

▸ **COLLAB DÜNYASINDA**

AXL'den gelen bir phone `dict`'i için `new = phone` yazmak ikinci bir **etiket** yaratır, kopya değil. `new["description"]` değerini değiştirirseniz `phone` da değişir, çünkü ikisi aynı kutuyu gösterir. Gerçek bir kopya için `phone.copy()` kullanın.

Özetle

- Variable bir kutu değil, bir objeye yapıştırılmış **etikettir**.
- `=` hiçbir zaman kopyalamaz; etiketi bir object'e bağlar.
- `id()` object'in kimliğini, `is` iki etiketin aynı object'i gösterip göstermediğini, `==` ise değerlerin eşit olup olmadığını söyler.

✎ **Kendin dene**

REPL'de `a = "SEP0A1B2C3D4E5F"` ve `b = a` yazın. `a is b` ne verir? Sonra `a = "SEP0A1B2C3D4E60"` yazıp `b`'nin değerine bakın. Neden değişmedi?

1.3 Data Types

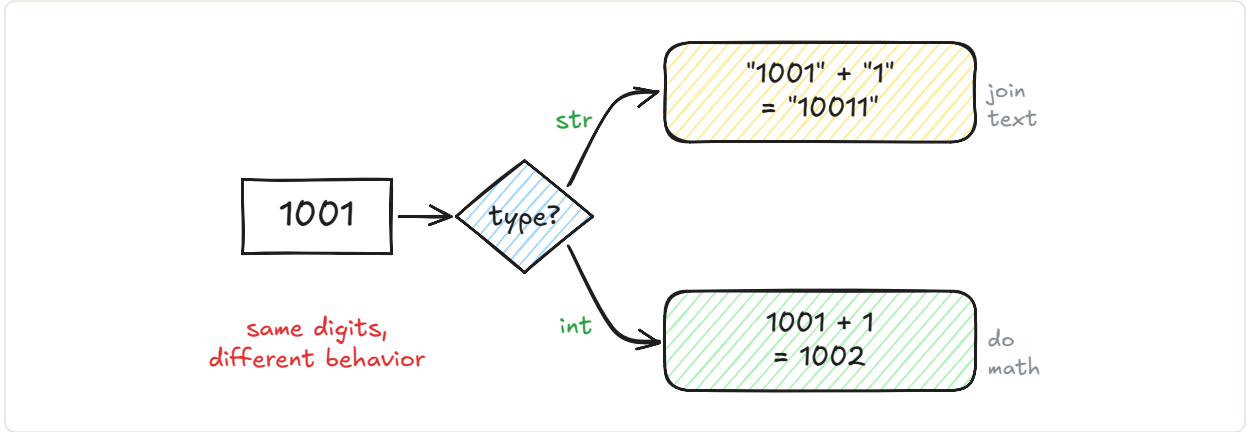
Why do we need data types?

Bilgisayarın belleği yalnızca byte tutar. Tek başına bakıldığında `1001` bir sayı mıdır, bir DN midir, yoksa dört karakterlik bir yazı mıdır? Bellek bunu bilemez. **Data type** (veri tipi), ham veriye anlam kazandıran etikettir ve üç soruyu cevaplar:

1. **Bu nedir?** Metin, tam sayı, evet/hayır bilgisi, bir koleksiyon…
2. **Bununla ne yapabilirim?** Sayıları toplarsınız, metinleri birleştirirsiniz, bir list üzerinde dolaşırsınız.
3. **Nasıl gösterilir ve saklanır?** `"0012"` baştaki sıfırları korur, `12` korumaz.

≈ **BENZETME**

CUCM'de bir **pattern** ile bir **hostname** aynı alana yazılabilir gibi görünür ama sistem ikisine farklı davranır: pattern'de `X` ve `!` joker karakterdir, hostname'de değildir. Python'da tip de böyledir: aynı görünen iki değer, tipine göre farklı şeyler yapar.



Şekil 1.3. Aynı görünen `1001`, tipine göre sayı da olabilir, metin de.

```
1 "1001" + "1"      # '10011'
2 1001 + 1          # 1002
3 "1001" + 1        # TypeError: can only
4                   # concatenate str to str
5 int("0012")       # 12  ← baştaki sıfırlar kayboldu
```

KODU SATIR SATIR OKUYALIM

1 İki **metin** `+` ile birleştirilir (yan yana yazılır). Sonuç yine bir metindir: `'10011'`.

2 İki **sayı** `+` ile toplanır: `1002`. Aynı `+` işareti, tipe göre farklı bir iş yaptı.

3–4 Metin ile sayıyı toplamaya çalışmak hata verir. Python tahmin yürütmez.

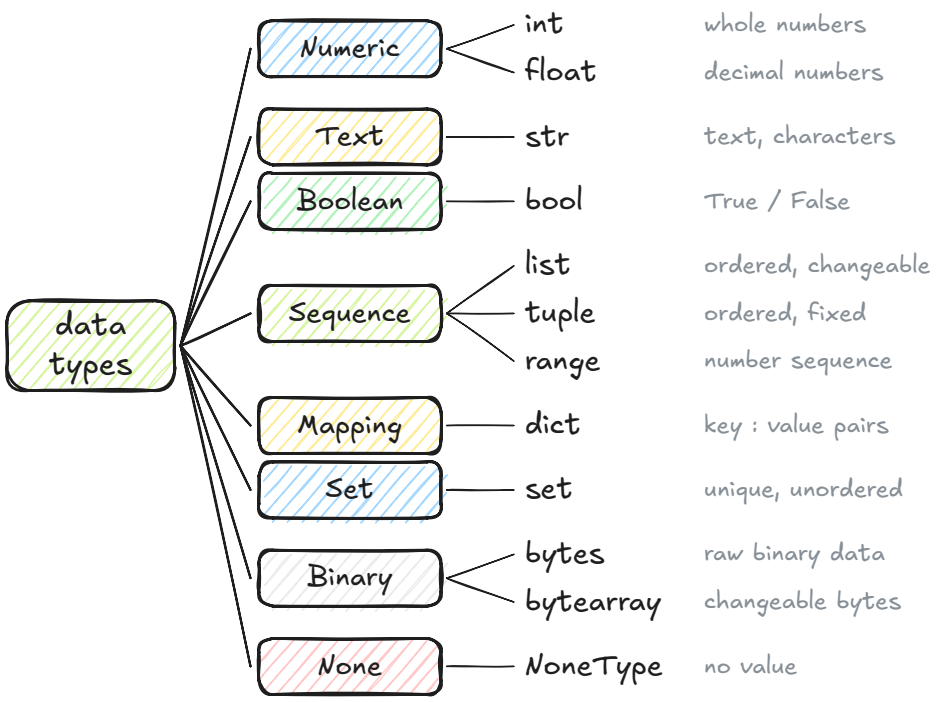
5 `int()` metni sayıya çevirir, ama sayılarda baştaki sıfır anlamsızdır: `"0012"` olarak yazılan DN `12` olur ve bilgi kaybolur.

Python bu açıdan **strongly typed** (güçlü tipli) bir dildir: tipleri kendi kendine karıştırmaz. `"1001" + 1` yazdığınızda iki yorumdan birini sessizce seçmek yerine `TypeError` verir. Bu iyi bir şeydir: hata, sorun üretime çıkmadan, daha ilk çalıştırmada ortaya çıkar.

> ▸ **COLLAB DÜNYASINDA**
> Bir DN, sayıya benzese de `str`'dir (metin). `0012`, `+905551112233` ve `\+90XXX` gibi değerler `int` olarak saklanırsa bozulur. `int`'i yalnızca saydığınız ya da karşılaştırdığınız şeyler için kullanın: ring sayısı, maksimum call sayısı, timeout gibi.

Built-in data types

Python'un hazır gelen (built-in) tipleri bellidir. Aşağıdaki diyagram hepsini tek bakışta gösteriyor, tablo ise her tipin collaboration dünyasındaki tipik kullanımını. Bu bölümün geri kalanında sırayla her birini ayrıntılı göreceğiz, şimdi sadece haritaya bakın.



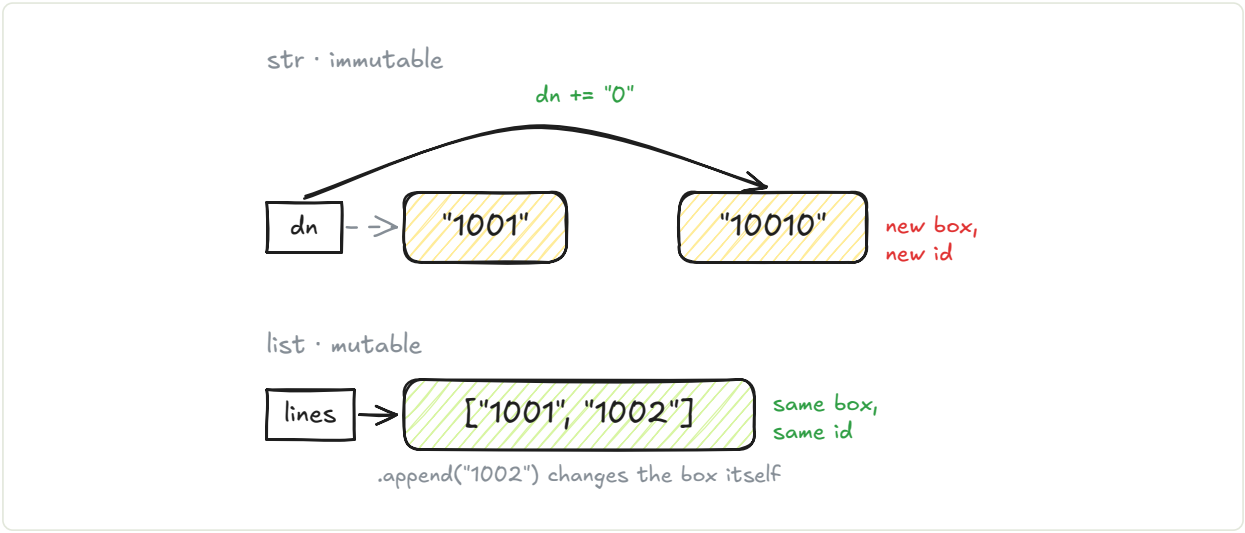
Şekil 1.4. Python'un built-in veri tipleri: sayılar, metin, koleksiyonlar ve "değer yok".

Tip	Örnek	Collab'da kullanımı
`int`	`4`	ring sayısı, maksimum call sayısı
`float`	`0.25`	paket kaybı oranı
`str`	`"1001"`	DN, device adı, MAC adresi
`bool`	`True`	phone register mı?
`list`	`["1001", "1002"]`	bir phone'un line'ları
`tuple`	`("cucm-pub", 8443)`	host + port çifti
`range`	`range(1001, 1100)`	bir DN bloğu üretmek
`dict`	`{"dn": "1001"}`	tek bir AXL / REST kaydı
`set`	`{"PT_Internal"}`	benzersiz partition'lar
`bytes`	`b"INVITE sip:..."`	ham SIP mesajı
`None`	`None`	cevaptaki boş alan

Mutable vs immutable

Tipleri iki büyük gruba ayırmak, kitabın geri kalanında karşınıza çıkacak birçok davranışı açıklar: **immutable** (değiştirilemez) ve **mutable** (değiştirilebilir) tipler.

> **≈ BENZETME**
> Bir **CDR kaydını** düşünün: yazıldıktan sonra üzerinde oynanmaz. Bir hata varsa kaydı düzeltmezsiniz, yeni bir kayıt oluşturursunuz. Bu **immutable**. Bir device'ın **description** alanı ise aynı kayıt üzerinde güncellenir: kayıt aynı kalır, içeriği değişir. Bu da **mutable**.



Şekil 1.5. Immutable tipte her "değişiklik" yeni bir kutu yaratır. Mutable tipte kutu aynı kalır, içi değişir.

```
1  dn = "1001"
2  before = id(dn)
3  dn += "0"
4  id(dn) == before       # False → yeni object
5
6  lines = ["1001"]
7  before = id(lines)
8  lines.append("1002")
9  id(lines) == before    # True → aynı object
```

KODU SATIR SATIR OKUYALIM

1–2 `dn` etiketi bir `str` kutusuna bağlanır ve kutunun kimliği saklanır.

3 `dn += "0"` kısaca `dn = dn + "0"` demektir. `str` immutable olduğu için `"10010"` değeriyle **yeni bir kutu** yaratılır ve `dn` ona taşınır.

4 Kimlik değişti: `False`. Eski kutu hiç değişmedi.

6–7 Bu kez `list` kullanıyoruz. Kimliğini saklıyoruz.

8 `append` list'in kendisini değiştirir. Yeni kutu yok.

9 Kimlik aynı: `True`.

IMMUTABLE Değer, object yaratıldıktan sonra asla değişemez. Her "değişiklik" aslında yeni bir object yaratır ve etiketi oraya taşır. Bu gruptakiler: `int`, `float`, `str`, `bool`, `tuple`, `range`, `bytes`, `None`.

MUTABLE Object **yerinde** değiştirilir. Kimliği aynı kalır. Bu gruptakiler: `list`, `dict`, `set` ve `bytearray`.

Bu ayrım iki yüzden önemlidir:

- Mutable bir kutuya iki etiket bağlıysa, birinden yapılan değişiklik diğerinde de görünür.
- Yalnızca immutable değerler `dict` anahtarı (key) ya da `set` elemanı olabilir.

> ▸ COLLAB DÜNYASINDA
> Bir phone şablonu `dict`'ini loop içinde tekrar tekrar kullanıp düzenlerseniz, o şablondan türettiğiniz **bütün phone'lar** değişir; çünkü hepsi aynı kutuyu gösterir. Her phone için taze bir `dict` kurun.

1.3.1 int

int · whole numbers

IMMUTABLE

`int` (integer), tam sayıları tutar: pozitif ya da negatif, boyut sınırı olmadan. Saydığınız ya da hesapladığınız her şey için kullanın: call sayısı, port, zamanlayıcı, deneme sayısı.

```
1  max_calls = 4
2  max_calls + 1          # 5
3  7 // 2, 7 % 2          # (3, 1)  tam bölme, kalan
4  7 / 2                  # 3.5  ← "/" her zaman float verir
5  2 ** 100               # taşma yok, her boyutta
6  1_000_000              # okunabilirlik için alt çizgi
7
8  int("8443")            # 8443
9  int("1001A")           # ValueError
```

KODU SATIR SATIR OKUYALIM

1–2 `max_calls` etiketine `4` bağlanır; `+ 1` ile bir fazlası hesaplanır: `5`.

3 `//` bölmenin **tam kısmını**, `%` **kalanını** verir. `7` içinde `2` üç kere vardır ve `1` artar. Aynı satırda virgülle iki işlem yazınca sonuç bir çift (tuple) olarak gelir.

4 Normal bölme `/` her zaman `float` döndürür, sonuç tam sayı olsa bile (`4 / 2` → `2.0`).

5 `**` üs alma işlemidir. Python'da `int`'in büyüklük sınırı yoktur.

6 Uzun sayılarda `_` yalnızca okunabilirlik içindir: Python onu yok sayar.

8–9 `int()` metni sayıya çevirir. Metin geçerli bir sayı değilse `ValueError` hatası verir.

İşlem	Anlamı
`+` `-` `*`	toplama, çıkarma, çarpma
`/`	bölme, sonuç her zaman `float`
`//` `%`	tam bölme ve kalan
`**`	üs alma
`int(x)`	metni ya da `float`'ı tam sayıya çevirir

▸ **COLLAB DÜNYASINDA**
AXL her değeri **metin** olarak döndürür: `"maxNumCalls": "4"`. Karşılaştırmadan önce çevirin: metin olarak `"10" > "9"` değeri `False`'tur (karakter karakter karşılaştırılır), ama `int("10") > int("9")` değeri `True`'dur.

1.3.2 float

float · decimal numbers

IMMUTABLE

`float`, ondalık noktası olan sayıları tutar. `/` ile yaptığınız her bölme bir `float` üretir; `4 / 2` bile `2.0` verir.

```
1  lost, received = 12, 4800
2  loss = lost / received   # 0.0025
3  f"{loss:.2%}"            # '0.25%'
4
5  mos = float("4.1")       # metin → float
6  mos >= 4.0               # True
7
8  0.1 + 0.2                # 0.30000000000000004
9  round(0.1 + 0.2, 2)      # 0.3
10 int(4.9)                 # 4  (keser, yuvarlamaz)
```

KODU SATIR SATIR OKUYALIM

1 Virgülle iki variable birden tanımlanır: `lost = 12` ve `received = 4800`. Buna **unpacking** diyeceğiz.

2 Kayıp oranı: `12 / 4800`. Bölme sonucu bir `float`'tır.

3 `f"..."` bir **f-string**'dir: süslü parantez içine değer koyup biçimlendirirsiniz. `:.2%` "yüzde olarak göster, iki basamak" demektir.

5 `float()` metni ondalık sayıya çevirir.

6 Karşılaştırma `bool` verir.

8 Sürpriz: `0.1 + 0.2` tam `0.3` değildir.

9 Gösterirken yuvarlayın: `round(x, 2)` iki basamak bırakır.

10 `int(4.9)` ondalık kısmı **atar**, yuvarlamaz: `4`.

Float'lar bilgisayarda ikili (binary) sistemde saklanır; bu yüzden bazı ondalık sayılar tam olarak ifade edilemez, tıpkı üçte birin ondalık yazılışının `0.333...` diye bitmemesi gibi. Kural basittir: **gösterirken yuvarlayın, iki float'ı asla `==` ile karşılaştırmayın.**

> ▸ **COLLAB DÜNYASINDA**
> CMR (call management record) verisi, kaybolan ve alınan paket sayılarını verir. Bunlardan hesapladığınız kayıp oranı, jitter ve MOS değerleri `float`'tır.

Working with numbers

Sayıların kendi method'ları azdır. İşin çoğunu **built-in function**'lar ve **f-string biçimleri** yapar. Aşağıdaki örnek, bir CDR süre listesinden rapor çıkarmaya yetecek araçları gösteriyor.

```
durations = [125, 340, 62, 410]  # saniye
sum(durations) / len(durations)  # 234.25
min(durations), max(durations)   # (62, 410)
divmod(3725, 60)       # (62, 5) → 62 dk 5 sn
round(234.567, 1)      # 234.6
round(2.5), round(3.5) # (2, 4) çifte yuvarlar!
abs(-3)                # 3
int("1F", 16), hex(31) # (31, '0x1f')
(4.0).is_integer()     # True
f"{1234567:,}"         # '1,234,567'
f"{0.0025:.2%}"        # '0.25%'
```

Metot	Ne yapar
`sum` `min` `max`	bir list'in toplamı, en küçüğü, en büyüğü
`divmod(a, b)`	bölümü ve kalanı tek çağrıda verir
`round(x, n)`	`n` basamağa yuvarlar (eşitlikte çifte gider)
`abs(x)`	sıfıra uzaklık (mutlak değer)
`int(s, 16)` `hex(n)`	16'lık tabandaki metin ↔ sayı
`float.is_integer()`	ondalık kısmı yok mu?
`f"{x:.2f}"` `f"{x:,}"`	yalnızca gösterim için biçimlendirme

KODU SATIR SATIR OKUYALIM

1 Dört call'ın süresini tutan bir list. Köşeli parantez içinde virgülle ayrılmış değerler.

2 Toplam bölü eleman sayısı, yani ortalama süre.

3 En kısa ve en uzun call.

4 `3725` saniyeyi `60`'a böler: `62` dakika ve `5` saniye. Rapor için okunabilir bir süre.

5-6 Yuvarlama. Dikkat: `2.5` ve `3.5` yuvarlanınca `2` ve `4` çıkar; Python eşitlikte **en yakın çift sayıya** gider.

8 `int("1F", 16)` 16'lık tabandaki metni sayıya çevirir. Cihaz kimliklerini ve MAC'leri çözerken işe yarar.

10-11 Gösterim biçimleri: binlik ayırıcı ve yüzde.

▸ **COLLAB DÜNYASINDA**
CDR'larda süreler saniyedir. `divmod(duration, 60)` ile `3725` değerini "62 dk 5 sn" olarak göstermek, call raporlarını insanlar için okunur kılar.

✎ **Kendin dene**
Bir call'ın süresi `3725` saniye. `divmod` kullanarak saat, dakika ve saniyeyi hesaplayın (ipucu: `divmod`'u iki kez kullanın). Sonra `1001` ve `"1001"` için `type(...)` yazın: `type(x)` bir değerin tipini söyler. Farkı görün.

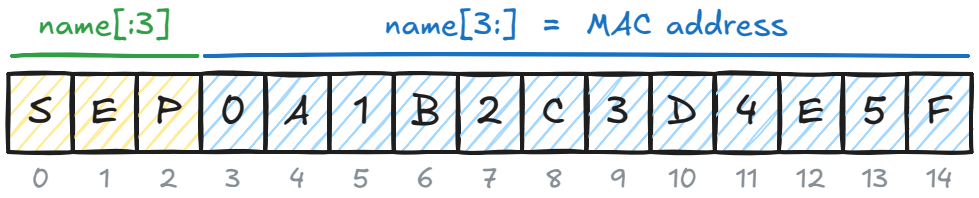
1.3.3 str

str · text

IMMUTABLE

`str` (string), bir karakter dizisidir: yani metin. Tek tırnak (`'...'`), çift tırnak (`"..."`) ya da üç tırnak (`"""..."""`) içine yazılır. Çift ve tek tırnak aynı işi yapar; yeter ki açtığınız tırnakla kapatın. Üç tırnak ise birden çok satıra yayılan metinler içindir.

Collaboration işlerinde string'leri her yerde görürsünüz: DN, device adı, MAC adresi, kullanıcı kimliği, SIP URI. Bunların hepsi metindir.



Şekil 1.6. Her karakterin bir **index**'i (sıra numarası) vardır. Sayma `0`'dan başlar; sondan sayarken `-1` kullanılır.

```
name = "SEP0A1B2C3D4E5F"
len(name)        # 15
name[:3]         # 'SEP'
name[3:]         # '0A1B2C3D4E5F'
name.lower()     # yeni str, name değişmedi
name[0] = "X"    # TypeError: immutable
```

KODU SATIR SATIR OKUYALIM

1. Bir phone'un device adını `name` etiketine bağlıyoruz.
2. `len()` uzunluğu verir: `SEP` (3) + MAC (12) = `15` karakter.
3. **Slicing** (dilimleme): `[:3]` "baştan 3'e kadar, 3 **dahil değil**" demektir. Sonuç `'SEP'`: device tipini veren önek.
4. `[3:]` "3. index'ten sona kadar" demektir: phone'un MAC adresi.
5. `.lower()` küçük harfli **yeni bir** string döndürür. `name` olduğu gibi kalır.
6. String immutable olduğu için tek bir karakteri yerinde değiştiremezsiniz.

String'ler yerinde değiştirilemediği için `.lower()` ya da `.replace()` gibi method'lar her zaman **yeni bir** string döndürür. Sonucu saklamak istiyorsanız yeniden atayın: `name = name.lower()`.

▸ **COLLAB DÜNYASINDA**
Phone'un MAC adresi, `SEP` öneki çıkarılmış device adıdır: `name[3:]`. Aynı şekilde `name[:3]` device'ın cinsini söyler (`SEP` masa telefonu, `CSF` Jabber).

str methods · clean up

Gerçek dünyadaki veri temiz gelmez. Excel'den kopyalanan bir MAC adresinde baştaki boşluklar, küçük harfler, iki nokta üst üste işaretleri vardır. String method'ları bu veriyi temizlemek içindir. Hepsi **yeni bir string** döndürür; bu yüzden soldan sağa zincirlenebilir.



Şekil 1.7. Method zinciri: her adım bir önceki adımın sonucu üzerinde çalışır.

```
1 raw = "  sep0a1b2c3d4e5f \n"   # CSV hücresi
2 raw.strip()              # 'sep0a1b2c3d4e5f'
3 raw.strip().upper()      # 'SEP0A1B2C3D4E5F'
4 "alice smith".title()    # 'Alice Smith'
5 "0A:1B:2C:3D:4E:5F".replace(":", "")
6                          # '0A1B2C3D4E5F'
7 "12".zfill(4)            # '0012'
8 "sip:1001".removeprefix("sip:")  # '1001'
```

KODU SATIR SATIR OKUYALIM

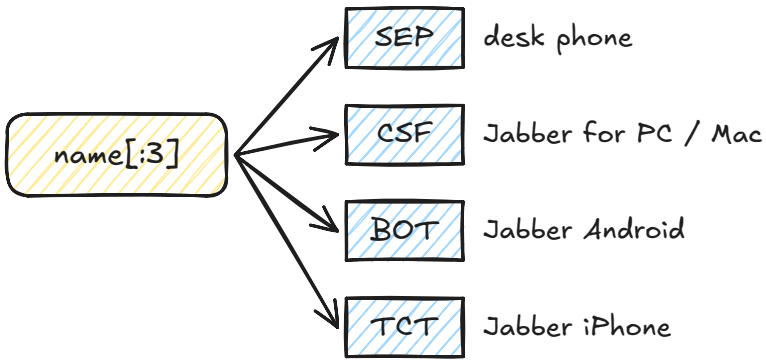
1. `\n` bir **satır sonu** karakteridir. CSV ya da dosyadan okunan metinlerin sonunda genellikle görünmez olarak bulunur.
2. `strip()` iki uçtaki boşluk, tab ve satır sonlarını siler.
3. Zincir: önce temizle, sonra büyük harfe çevir.
4. `title()` her kelimenin ilk harfini büyütür.
5–6. `replace(eski, yeni)` her `eski`'yi `yeni` ile değiştirir. Burada iki nokta üst üste silinir.
7. `zfill(4)` sola sıfır ekleyerek uzunluğu `4`'e tamamlar. DN'lerde baştaki sıfırı geri kazandırır.
8. `removeprefix()` baştaki sabit bir parçayı atar.

Metot	Ne yapar
`strip()`	iki uçtaki boşluk, tab, satır sonunu siler
`lstrip()` `rstrip()`	yalnızca sol / yalnızca sağ taraf
`lower()` `upper()` `title()`	harf büyüklüğünü değiştirir
`replace(eski, yeni)`	her `eski`'yi `yeni` ile değiştirir
`zfill(n)`	sola sıfır ekleyerek `n` uzunluğuna tamamlar
`removeprefix(p)` `removesuffix(s)`	sabit başlangıcı / sonu atar

▸ **COLLAB DÜNYASINDA**
Excel'den yapıştırılan MAC'ler boşluklu, küçük harfli ve iki nokta üst üstelidir.
`"SEP" + mac.strip().upper().replace(":", "")` ifadesi, AXL'in kabul ettiği geçerli bir device adı üretir.

str methods · search and test

Bir sonraki grup method, metnin **içinde bir şey olup olmadığını** ya da **nerede olduğunu** söyler. Hiçbiri string'i değiştirmez, sadece soru sorar.



Şekil 1.8. Device adının ilk üç harfi cihazın cinsini belirler.

```
1 name = "SEP0A1B2C3D4E5F"
2 name.startswith("SEP")      # True
3 "cucm-pub.clauto.lab".endswith(".lab")
4                             # True
5 "A1" in name                # True
6 name.find("A1")             # 4  (yoksa -1)
7 name.index("ZZ")            # ValueError
8 "G711u,G711a,G722".count("G711")   # 2
9 "1001".isdigit()            # True
```

KODU SATIR SATIR OKUYALIM

2 `startswith()` metin bu parçayla **başlıyor mu** diye sorar: `True` ya da `False`.

3–4 `endswith()` metin bu parçayla **bitiyor mu** diye sorar.

5 `in` bir operatördür: "bu parça metnin herhangi bir yerinde var mı?"

6 `find()` parçanın **ilk bulunduğu index'i** verir. Bulamazsa `-1` döner, hata vermez.

7 `index()` `find()` gibidir, ama bulamazsa `ValueError` verir.

8 `count()` parçanın kaç kere geçtiğini sayar.

9 `isdigit()` metnin yalnızca rakamlardan oluşup oluşmadığını söyler.

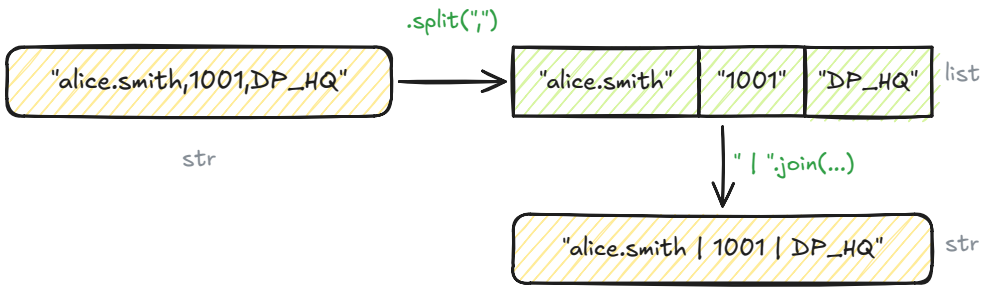
Metot	Ne yapar
`startswith(p)` `endswith(s)`	bu parçayla başlıyor / bitiyor mu?
`x in s`	`x` metnin herhangi bir yerinde var mı? (operatör)
`find(x)`	ilk konum; yoksa `-1`
`index(x)`	`find` gibi, ama yoksa `ValueError`
`count(x)`	`x` kaç kere geçiyor
`isdigit()` `isalpha()` `isalnum()`	yalnızca rakam / harf / ikisi mi?

▸ **COLLAB DÜNYASINDA**

Device adının ilk üç harfi cihaz tipini söyler. `name.startswith("CSF")` ile bir phone listesindeki bütün Jabber client'ları bulabilirsiniz.

str methods · split, join, format

`split` bir string'i parçalara ayırıp bir **list** yapar. `join` ise bir list'i tek bir string'e yapıştırır. İkisi birbirinin tersidir ve veri ayrıştırma (parsing) işlerinin çoğu bu ikilinin karışımıdır.



Şekil 1.9. `split` metni keser ve list yapar, `join` list'i tekrar metne yapıştırır.

```
1 row = "alice.smith,1001,DP_HQ"
2 user, dn, pool = row.split(",")
3 uri = "sip:1001@cucm-pub.clauto.lab"
4 uri.split("@")     # ['sip:1001', 'cucm-pub…']
5 uri.partition("@") # ('sip:1001', '@', 'cucm…')
6 ", ".join(["1001", "1002"])  # '1001, 1002'
7 f"{user:<12}{dn:>6}"  # 'alice.smith   1001'
```

KODU SATIR SATIR OKUYALIM

1. Bir CSV satırı: virgüllerle ayrılmış üç alan.
2. `split(",")` üç parçalı bir list döndürür. Soldaki üç isim, parçaları sırayla alır (**unpacking**).
4. `@` işaretinden kesince iki parça çıkar: kullanıcı kısmı ve host kısmı.
5. `partition()` metni **bir kere** keser ve `(önce, ayraç, sonra)` verir. Ayracın kendisi de sonuçtadır.
6. `join()` ayracı `", "` olarak kullanarak list'in elemanlarını yapıştırır.
7. `:<12` "sola yasla, 12 genişlikte", `:>6` "sağa yasla, 6 genişlikte" demektir. Rapor sütunlarını hizalamak için kullanılır.

Metot	Ne yapar
`split(sep)`	her `sep` yerinden keser → list (`sep` verilmezse boşluktan)
`split(sep, 1)` `rsplit(sep, 1)`	yalnızca bir kere keser: soldan / sağdan
`partition(sep)`	bir kere keser → `(önce, sep, sonra)`
`sep.join(items)`	bir list'in string'lerini `sep` ile yapıştırır
`f"{x:<10}"` `f"{x:>6}"`	sabit genişlikte sola / sağa yaslar

> ▸ COLLAB DÜNYASINDA
> Bir SIP URI'yi kullanıcı ve host olarak ayırmak için `uri.partition("@")` yeterlidir. `join` ise geri gönderdiğiniz CSV satırlarını ve rapor satırlarını kurar.

1.3.4 bool

bool · True or False

IMMUTABLE

`bool` yalnızca iki değer alabilir: `True` (doğru) ve `False` (yanlış). Karşılaştırmalar (`==`, `>`, `in`) bir `bool` üretir ve `if` deyimleri bu değerleri tüketir. Bir sonraki bölümlerin (`if`, loop'lar) temelinde bu iki değer vardır.

```python
status = "Registered"
is_reg = status == "Registered"   # True
type(is_reg)              # <class 'bool'>

bool(""), bool("1001")    # (False, True)
bool([]), bool(["1001"])  # (False, True)
True + True               # 2  (bool bir int'tir)

bool("false")             # True  ← dikkat!
```

KODU SATIR SATIR OKUYALIM

1. `status` etiketine bir metin bağlanır.
2. `==` karşılaştırmasının sonucu `True`'dur ve `is_reg` etiketine bağlanır. Dikkat: tek `=` bağlar, çift `==` karşılaştırır.
3. `type()` bir değerin tipini söyler.
5–6. Her object'in bir **truth value**'su (doğruluk değeri) vardır. **Boş** ya da **sıfır** olan her şey `False` sayılır (**falsy**): `""`, `0`, `[]`, `{}`, `None`. Geri kalan her şey `True` sayılır (**truthy**).
7. Perde arkasında `bool`, `int`'in bir alt tipidir: `True` aslında `1`, `False` ise `0`'dır.
9. `"false"` yazısı boş olmayan bir metindir. Bu yüzden truthy sayılır.

× SIK YAPILAN HATA

AXL boolean değerleri **metin** olarak gönderir: `"true"` ya da `"false"`. Boş olmayan her metin truthy olduğu için `bool("false")` değeri `True`'dur ve bu hatayla bol bol karşılaşırsınız. Bunun yerine karşılaştırın: `enabled = value == "true"`.

✎ Kendin dene

`"  Sep0a1b2c3d4e5f \n"` string'ini tek bir ifadeyle `SEP0A1B2C3D4E5F` haline getirin. Sonra bu sonucun `"SEP"` ile başladığını ve uzunluğunun `15` olduğunu `startswith` ve `len` ile doğrulayın.

1.3.5 list

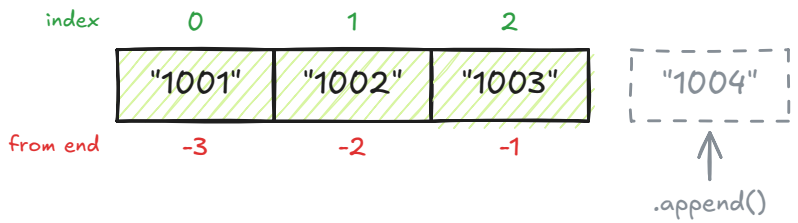
list · ordered collection

MUTABLE

Şimdiye kadar tek değerlerle çalıştık: bir DN, bir device adı. Gerçek işlerde ise hep **çoğul** şeylerle uğraşırsınız: bir phone'un line'ları, bir CSV'deki satırlar, bir cluster'daki bütün phone'lar. Bunları tutan en temel yapı `list`'tir (liste).

≈ BENZETME

Bir **hunt list**'i düşünün: sıralı üyeleri vardır, aynı üye iki kere yer alabilir ve üye ekleyip çıkarabilirsiniz. Python'un `list`'i de budur: sıralı, değiştirilebilir bir elemanlar topluluğu.



Şekil 1.10. list elemanlarına index ile ulaşılır: baştan `0, 1, 2...`, sondan `-1, -2...`

```
lines = ["1001", "1002", "1003"]
lines[0]              # '1001'
lines[-1]             # '1003'
lines.append("1004")  # sona ekle
lines[1] = "1099"     # yerinde değiştir
len(lines)            # 4
```

KODU SATIR SATIR OKUYALIM

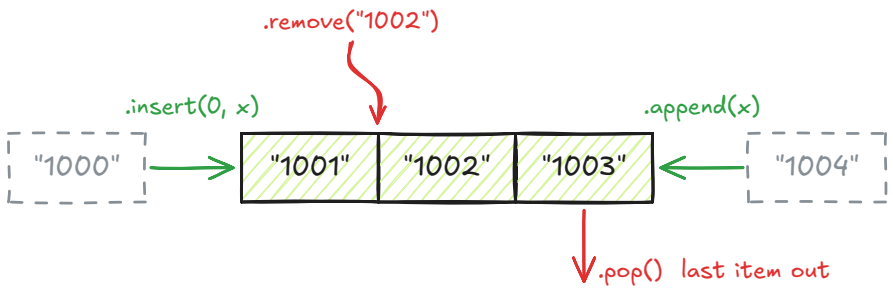
1. Köşeli parantez ve virgüllerle üç elemanlı bir list kurulur.
2. İlk elemana `0` index'iyle ulaşılır. Bilgisayarlarda sayma çoğu zaman `0`'dan başlar.
3. `-1` sondan birinci demektir: list ne kadar uzun olursa olsun son elemanı verir.
4. `append()` list'in sonuna bir eleman ekler. list şimdi dört elemanlıdır.
5. Index'e atama yaparak bir elemanı **yerinde** değiştirirsiniz: `lines[1]` artık `"1099"`'dur.
6. `len()` eleman sayısını söyler.

`list` her tipten eleman tutabilir, farklı tipleri bir arada barındırabilir ve tekrar eden değerlere izin verir. Sıra korunur. Bu yüzden "bir şeyin çoğulu" olan her yerde ilk akla gelen kaptır.

> ▸ **COLLAB DÜNYASINDA**
> `listPhone` bir phone listesi döndürür, bir CSV import'u satır listesidir, bir phone'un line'ları da DN listesidir. Otomasyon script'lerinin çoğu bir list üzerinde dolaşır.

list methods · add and remove

Aşağıdaki method'ların hepsi list'i **yerinde** değiştirir; list'in `id`'si aynı kalır. Bu method'ların çoğu bir değer döndürmez, doğrudan list'i değiştirir.



Şekil 1.11. Ekleme ve çıkarma method'ları list'in kendisini değiştirir.

```
1 lines = ["1001", "1002"]
2 lines.append("1003")          # bir tane, sona
3 lines.extend(["1004", "1005"])  # birçok tane
4 lines.insert(0, "1000")       # 0. index'e
5 lines.remove("1002")          # değere göre
6 lines.pop()                   # '1005' sonuncu
7 lines.pop(0)                  # '1000' index'e göre
8 lines         # ['1001', '1003', '1004']
```

KODU SATIR SATIR OKUYALIM

1 Başlangıç: iki elemanlı bir list.

2 `append()` tek bir eleman ekler.

3 `extend()` başka bir list'in **bütün elemanlarını** tek tek ekler.

4 `insert(0, ...)` elemanı baştan `0.` konuma koyar, diğerleri kayar.

5 `remove()` **değere göre** siler: ilk `"1002"` gider. Böyle bir değer yoksa `ValueError` verir.

6–7 `pop()` son elemanı (ya da verdiğiniz index'tekini) listeden çıkarır **ve size geri verir**. Bu çıkarılan değeri kullanabileceğiniz anlamına gelir.

8 Sonuçta list'te `'1001'`, `'1003'`, `'1004'` kalır.

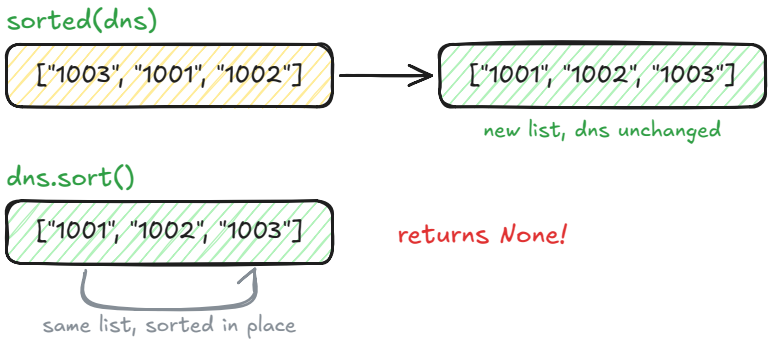
Metot	Ne yapar
`append(x)`	sona bir eleman ekler
`extend(items)`	başka bir list'in bütün elemanlarını ekler
`insert(i, x)`	`i` konumundan önce `x` ekler
`remove(x)`	ilk `x`'i siler (yoksa `ValueError`)
`pop()` `pop(i)`	sonuncuyu / `i` numaralıyı siler ve **geri verir**
`clear()`	her şeyi siler

× SIK YAPILAN HATA

`lines.append(["1002", "1003"])` **tek bir** eleman ekler: iç içe bir list. Sonuç `["1001", ["1002", "1003"]]` olur. Birçok eleman eklemek için `extend` kullanın.

list methods · order and search

Sıralama ve arama her list üzerinde çalışır. Burada kritik olan, sıralamanın iki farklı yolu olduğunu bilmektir: **yeni bir list** döndüren `sorted()` ve list'i **yerinde** sıralayan `.sort()`.



Şekil 1.12. `sorted()` yeni bir list verir, `.sort()` var olan list'i değiştirir.

```python
dns = ["1003", "1001", "1002"]
sorted(dns)           # yeni sıralı list
dns.sort(reverse=True)  # yerinde
dns.index("1002")     # 1
dns.count("1001")     # 1
"1004" in dns         # False
backup = dns.copy()   # gerçek bir kopya
sorted(names, key=str.lower)  # harf büyüklüğünü yok say
```

KODU SATIR SATIR OKUYALIM

2 `sorted()` orijinal list'e dokunmaz; sıralı **yeni** bir list döner.

3 `.sort()` list'in kendisini sıralar ve hiçbir şey döndürmez. `reverse=True` ile sıra tersine çevrilir.

4 `index()` bir elemanın konumunu verir. Sıralamadan sonra `"1002"` artık `1` numaralı konumdadır.

5 `count()` elemanın kaç kere geçtiğini sayar.

6 `in` ile üyelik sorulur.

7 `copy()` list'in yeni bir kopyasını yapar. Hatırlayın: `backup = dns` kopya değil, ikinci bir etikettir.

8 `key=` bir sıralama kuralı verir. Burada `names` başka bir list'tir ve karşılaştırma küçük harfe çevirilerek yapılır.

Metot	Ne yapar
`sorted(lst)`	**yeni** sıralı list; orijinal değişmez
`lst.sort()`	yerinde sıralar; `None` döndürür
`reverse=True` `key=...`	azalan sıra / bir kurala göre sıra
`reverse()`	sırayı yerinde ters çevirir
`index(x)` `count(x)`	`x`'in konumu / kaç tane `x` var
`copy()`	yüzeysel (shallow) kopya: yeni list, aynı elemanlar

× SIK YAPILAN HATA

`dns = dns.sort()` yazarsanız `dns` `None` olur, çünkü `sort()` hiçbir şey döndürmez. Ya yalnızca `dns.sort()` yazın ya da `dns = sorted(dns)`.

1.3.6 tuple

tuple · fixed sequence

IMMUTABLE

`tuple`, list'e çok benzer ama yuvarlak parantezle yazılır ve **oluşturulduktan sonra değiştirilemez**. Birbirine ait sabit bir değer grubu için kullanılır: bir host ve onun portu gibi.

```python
server = ("cucm-pub.clauto.lab", 8443)
server[0]             # 'cucm-pub.clauto.lab'
host, port = server   # unpacking
server[1] = 443       # TypeError: immutable

codecs = ("G722", "G711u", "G711a")
"G722" in codecs      # True

nodes = {("cucm-pub.clauto.lab", 8443): "pub"}
```

KODU SATIR SATIR OKUYALIM

1 Host ve port tek bir tuple içinde.

2 Elemanlara list gibi index ile ulaşılır.

3 **Unpacking** (açma): tuple'ın her elemanı kendi ismine tek satırda atanır. `host` artık hostname'i, `port` ise `8443`'ü tutar.

4 Tuple immutable olduğu için eleman değiştirilemez.

6–7 Codec tercih sırası değişmeyecek bir bilgidir; tuple bunu garanti eder. `in` ile üyelik sorulabilir.

9 Tuple'lar immutable olduğu için `dict` anahtarı (key) olabilir. list olamaz.

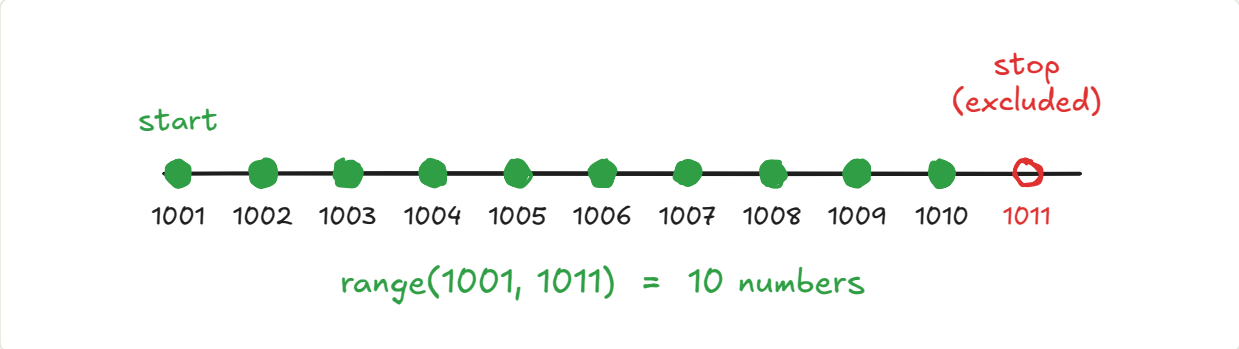
> ▸ **COLLAB DÜNYASINDA**
> Bir sunucu adresi `(host, port)`, sabit bir codec tercih sırası ya da bir AXL çağrısından sonra dönen `(ok, mesaj)` sonucu, tuple için güzel örneklerdir.

1.3.7 range

range · number sequence

IMMUTABLE

`range(start, stop, step)` bir tam sayı dizisini **tarif eder**. `start` dahildir, `stop` **dahil değildir**. Varsayılan adım (step) `1`'dir.



Şekil 1.13. `range(1001, 1011)` 1001'den 1010'a kadar sayıları tarif eder. `stop` dahil değildir.

```
1 block = range(1001, 1011)
2 len(block)                  # 10
3 1005 in block               # True
4 list(range(1001, 1011, 3)) # [1001, 1004, 1007, 1010]
5
6 for n in range(1001, 1004):
7     print("DN", str(n))     # DN 1001 … DN 1003
```

KODU SATIR SATIR OKUYALIM

1 1001'den 1010'a kadar 10 sayı. `1011` dahil değil.

2 `len()` kaç sayı olduğunu söyler.

3 `in` ile üyelik çok hızlı sorulur.

4 `range` sayıları saklamaz, ihtiyaç olunca hesaplar. Değerleri görmek için `list()` ile sarmalayın. Üçüncü argüman adımdır: 3'er 3'er atlar.

6–7 Bir `for` loop'u `range` üzerinde dolaşabilir. Loop'ları ilerleyen bölümde ayrıntılı göreceğiz. Her turda `n`, sıradaki sayıyı alır.

Bir range sayıları belleğe doldurmadığı için `range(1, 1_000_000)` neredeyse hiç bellek kullanmaz.

> ▸ **COLLAB DÜNYASINDA**
> Yeni bir site için DN bloğu üretmek `range(1001, 1100)` kadar kolaydır. Ama unutmayın: DN'ler **string**'tir. AXL'e göndermeden önce her sayıyı `str(n)` ile çevirin.

tuple and range · useful operations

Tuple'ın yalnızca iki method'u vardır. Geri kalan her şey, bütün **sequence**'lerin (sıralı koleksiyonların) ortak operasyonlarından gelir: `len`, `in`, index ile erişim, slicing ve unpacking. `range` de bunları destekler, ayrıca `start`, `stop` ve `step` özelliklerini sunar.

```
1  codecs = ("G722", "G711u", "G711a", "G711u")
2  codecs.count("G711u")     # 2
3  codecs.index("G711a")     # 2
4  len(codecs), codecs[:2]   # (4, ('G722', 'G711u'))
5
6  block = range(1001, 1100)
7  block.start, block.stop   # (1001, 1100)
8  block[-1], len(block)     # (1099, 99)
9  list(range(1003, 1000, -1))   # [1003, 1002, 1001]
10
11 dns = ["1001", "1002"]
12 users = ["alice.smith", "bob.jones"]
13 dict(zip(dns, users))     # {'1001': 'alice.smith', …}
14 list(enumerate(dns, start=1))  # [(1, '1001'), …]
```

KODU SATIR SATIR OKUYALIM

2–3 Tuple'ın iki method'u: `count()` bir değerin kaç kere geçtiğini, `index()` ilk konumunu verir.

4 `len` ve slicing, tuple'da da list ile aynı çalışır.

6–8 Bir `range`'in başlangıcı ve sonu özellik (attribute) olarak okunur. `block[-1]` son sayıyı verir.

9 Negatif adım **geriye doğru** sayar: `1003`'ten `1001`'e.

11–13 `zip(a, b)` iki sequence'i eleman eleman eşleştirir. `dict()` ile birleşince DN ve kullanıcı listesinden bir arama tablosu çıkar.

14 `enumerate()` her elemana sıra numarası ekler. `start=1` numaralamayı `1`'den başlatır.

Metot	Ne yapar
`count(x)` `index(x)`	kaç tane `x` / `x`'in konumu
`len` `in` `[i]` `[a:b]`	boyut, üyelik, index, dilim
`range(a, b, -1)`	negatif adımla geriye say
`zip(a, b)`	iki sequence'in elemanlarını eşleştirir
`enumerate(seq, start=1)`	her elemana sıra numarası verir

▸ COLLAB DÜNYASINDA

Excel'deki iki sütun, DN'ler ve kullanıcı kimlikleri, `dict(zip(dns, users))` ile tek bir arama tablosuna dönüşür.

1.3.8 dict

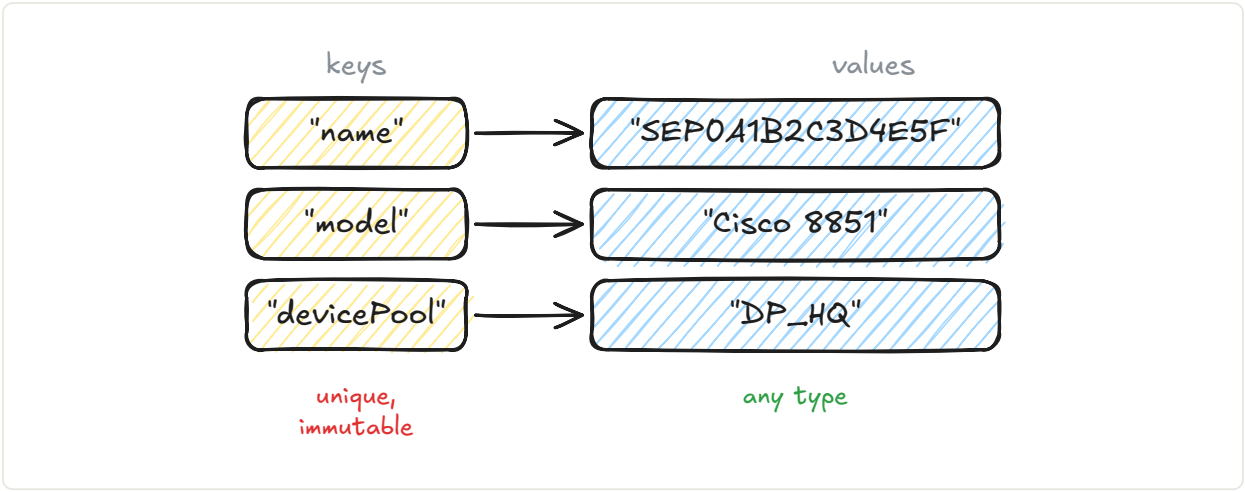
dict · key → value

MUTABLE

List'te elemanları sıra numarasıyla bulursunuz: "3. eleman". Ama bir phone'un bilgilerini düşünün: adı, modeli, device pool'u, description'ı. "3. bilgi" demek anlamsızdır, "model" demek anlamlıdır. İşte `dict` (dictionary, sözlük) bilgiyi **isimle** tutar: her **key** (anahtar) bir **value**'ya (değere) bağlanır.

≈ BENZETME

Bir telefon rehberi gibi düşünün: isme bakarsınız, numarayı bulursunuz. İsim **key**, numara **value**'dur. Rehberde aynı isim iki kere yer almaz: dict'te de key'ler benzersizdir.



Şekil 1.14. `dict` her key'i bir value'ya bağlar: soldaki isimler key, sağdakiler value.

```
1 phone = {"name": "SEP0A1B2C3D4E5F",
2          "model": "Cisco 8851"}
3 phone["model"]                  # 'Cisco 8851'
4 phone["devicePool"] = "DP_HQ"   # key ekle
5 phone.get("css")                # None, hata yok
6 "name" in phone                 # True
```

KODU SATIR SATIR OKUYALIM

1–2 Süslü parantez içinde `key: value` çiftleri virgülle ayrılır. İki satıra bölmek sadece okunabilirlik içindir.

3 Köşeli parantez içinde **key** verirsiniz, value'yu alırsınız. Key yoksa `KeyError` hatası alırsınız.

4 Var olmayan bir key'e atama yapmak, onu **ekler**. Var olan bir key'e atama yapmak değerini **değiştirir**.

5 `get()` ise daha hoşgörülüdür: key yoksa hata vermez, `None` döndürür.

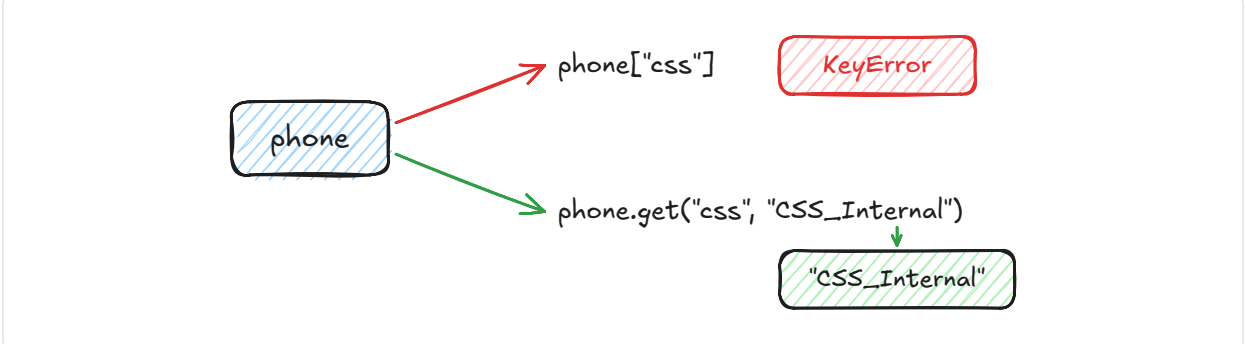
6 `in` bir **key** olup olmadığını sorar (value'ya bakmaz).

Key'ler benzersizdir ve immutable olmalıdır (çoğunlukla `str`). Value ise her şey olabilir: başka dict'ler, list'ler. Python eklenme sırasını korur.

> ▸ **COLLAB DÜNYASINDA**
> Her AXL ya da REST kaydı bir dict'tir: bir phone, bir line, bir Webex kullanıcısı. Webex API'sinden gelen JSON, doğrudan iç içe dict ve list'lere dönüşür.

dict methods · read and update

`phone["css"]` **katı** davranır: key yoksa program hata verip durur. `get` ise **hoşgörülüdür**: `None` ya da sizin verdiğiniz varsayılan değeri döndürür. Hangisini seçeceğiniz, verinin eksik gelip gelmeyeceğine bağlıdır.



Şekil 1.15. Köşeli parantez key yoksa hata verir, `get` ise `None` ya da varsayılan değer döndürür.

```python
phone = {"name": "SEP0A1B2C3D4E5F",
         "model": "Cisco 8851"}
phone.get("css")                  # None
phone.get("css", "CSS_Internal")  # varsayılan
phone.update({"devicePool": "DP_HQ",
              "description": "Alice desk"})
phone.pop("description")    # 'Alice desk'
phone.setdefault("css", "CSS_Internal")
del phone["css"]
```

KODU SATIR SATIR OKUYALIM

3 `get("css")` key yok, bu yüzden `None`.

4 İkinci argüman **varsayılan değerdir**: key yoksa onu döndürür.

5–6 `update()` başka bir dict'in bütün çiftlerini ekler ya da üzerine yazar. Birçok key'i tek çağrıda değiştirmenin yoludur.

7 `pop(key)` çifti siler ve value'yu size **geri verir**.

8 `setdefault()` key **yoksa** ekler, varsa dokunmaz.

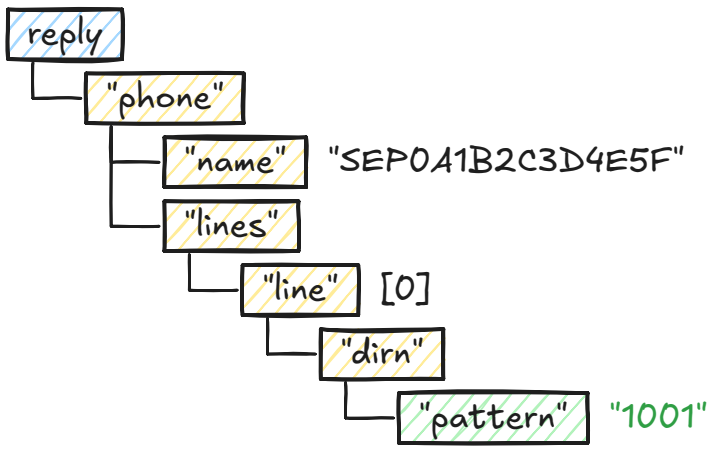
9 `del` bir method değil, bir deyimdir: çifti siler, geri bir şey vermez.

Metot	Ne yapar
`d[k]`	`k`'nin değeri; yoksa `KeyError`
`get(k, default)`	`k`'nin değeri; yoksa `default` ya da `None`
`update(other)`	birçok key'i birden ekler ya da üzerine yazar
`pop(k)`	`k`'yi siler ve değerini geri verir
`setdefault(k, v)`	yalnızca `k` yoksa `v`'yi atar
`del d[k]`	`k`'yi siler (deyim, method değil)

▸ **COLLAB DÜNYASINDA**
AXL cevabında isteğe bağlı alanlar eksik gelebilir. Onları `phone.get("css")` ile okuyun: böylece tek bir eksik kayıt, 500 phone'u gezen script'i durdurmaz.

dict methods · loop and nest

Bir dict'in içeriği üç **görünüm** (view) ile dışarı verilir. Bunlar loop'larda ve testlerde kullanılır. İç içe dict ve list'lerde ise bir seferde bir seviye aşağı inersiniz.



Şekil 1.16. AXL cevabı iç içe dict ve list'lerdir. Her adımda bir seviye aşağı inilir.

```python
for key, value in phone.items():
    print(f"{key:<12} {value}")

list(phone.keys())     # ['name', 'model']
"SEP0A1B2C3D4E5F" in phone.values()  # True

line = reply["phone"]["lines"]["line"][0]
line["dirn"]["pattern"]   # '1001'
copy = phone.copy()       # yeni bir dict
```

KODU SATIR SATIR OKUYALIM

1–2 `items()` her turda bir `(key, value)` çifti verir. Bu, dict üzerinde dolaşmanın en yaygın yoludur.

4 `keys()` bütün key'leri verir. `list()` ile sarmalayınca görünür olur.

5 `values()` bütün value'ları verir. `in` ile value'da arama yapılabilir.

7 İç içe veri: `reply` içindeki `"phone"`'a, onun içindeki `"lines"`'a, sonra `"line"` list'inin ilk elemanına inilir.

8 Bir seviye daha: line'ın `"dirn"` alanındaki `"pattern"`.

9 `copy()` yüzeysel (shallow) bir kopya yapar: dict yeni, içindeki objeler hâlâ aynı.

Metot	Ne yapar
`keys()`	bütün key'ler
`values()`	bütün value'lar
`items()`	`(key, value)` çiftleri; en yaygın loop kalıbı
`k in d`	`k` bir key mi? (key'lere bakar, value'lara değil)
`d[a][b][0]`	iç içe verinin içine adım adım inmek
`copy()`	yüzeysel kopya: yeni dict, aynı iç objeler

▸ COLLAB DÜNYASINDA

`getPhone` derin iç içe bir veri döndürür. Her adımı kısa ve okunur tutmak için bir iç seviyeyi bir variable'a atayın (`line = …`).

1.3.9 set

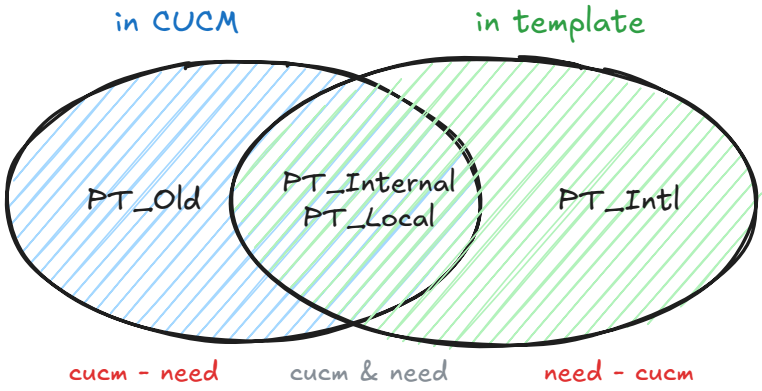
set · unique items

MUTABLE

`set` (küme), **benzersiz** ve immutable elemanlardan oluşan, **sırasız** bir koleksiyondur. Zaten var olan bir değeri eklemek hiçbir şey yapmaz. Index'i yoktur ve sıra garantisi yoktur. Set'in asıl gücü iki iştir: "bu eleman içinde mi?" sorusunu çok hızlı cevaplar ve iki grubun farkını tek satırda çıkarır.

≈ BENZETME

İki ayrı partition listesi düşünün: biri CUCM'de gerçekte olanlar, diğeri tasarım dokümanında olması gerekenler. "Eksik olanlar hangileri, fazla olanlar hangileri?" sorusu, kümeler arası bir farktır.



Şekil 1.17. İki set'in farkı ve kesişimi: eksik, fazla ve ortak olanlar tek satırda çıkar.

```
cucm = {"PT_Internal", "PT_Local", "PT_Old"}
need = {"PT_Internal", "PT_Local", "PT_Intl"}
need - cucm    # {'PT_Intl'}  eksik
cucm - need    # {'PT_Old'}   fazla
cucm & need    # ikisinde de var
set(["1001", "1001", "1002"])  # tekrarlar gider
```

KODU SATIR SATIR OKUYALIM

1–2 Süslü parantez içinde tek tek değerler (key yok): bu bir set.

3 `-` fark demektir: tasarımda olup CUCM'de **olmayan** partition: `PT_Intl`. Bu, **eksik** olandır.

4 Ters yönde fark: CUCM'de olup tasarımda olmayan, yani **fazla** olan: `PT_Old`.

5 `&` kesişim: ikisinde de bulunanlar.

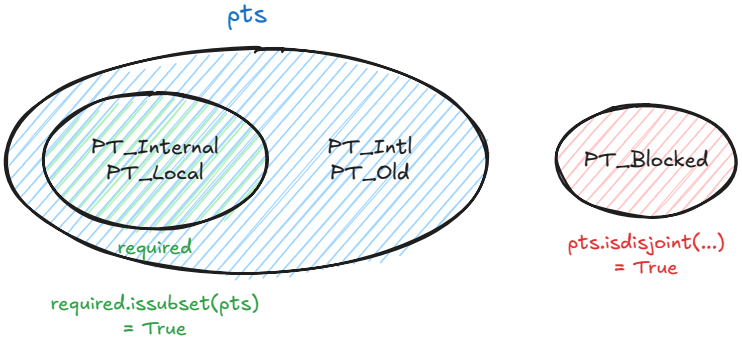
6 `set()` bir list'i set'e çevirir ve tekrar edenleri otomatik temizler.

Üç operatörü hatırlayın: `-` fark, `&` kesişim, `|` birleşim.

> ▸ **COLLAB DÜNYASINDA**
> Config denetimi (audit): CUCM'deki partition'ları tasarım dokümanındaki partition'larla karşılaştırın. Eksik ve fazla olanlar tek satırda ortaya çıkar.

set methods

Set method'ları eleman ekler ve çıkarır, bütün grupları karşılaştırır. Önceki konudaki `| & -` operatörlerinin method karşılıkları da vardır ve onlar list gibi başka koleksiyonları da kabul eder.



Şekil 1.18. `issubset()`: bir set'in bütün elemanları öbürünün içinde mi?

```python
pts = {"PT_Internal"}
pts.add("PT_Local")       # bir tane ekle
pts.add("PT_Local")       # etkisiz, zaten var
pts.update(["PT_Intl", "PT_Old"])  # birçok ekle
pts.discard("PT_None")    # yoksa: hata yok
pts.remove("PT_None")     # yoksa: KeyError

required = {"PT_Internal", "PT_Local"}
required.issubset(pts)    # True
pts.isdisjoint({"PT_Blocked"})  # True
```

KODU SATIR SATIR OKUYALIM

2-3 `add()` ekler. İkinci çağrı hiçbir şeyi değiştirmez çünkü eleman zaten vardır.

4 `update()` bir list'teki bütün elemanları ekler.

5-6 `discard()` sessizce siler, eleman yoksa bir şey olmaz. `remove()` ise eleman yoksa `KeyError` verir.

8-9 `issubset()` "`required`'ın bütün elemanları `pts` içinde mi?" diye sorar.

10 `isdisjoint()` iki set'in hiç ortak elemanı olmadığını sorar.

Metot	Ne yapar
`add(x)` `update(items)`	bir / birçok eleman ekler (tekrarlar yok sayılır)
`discard(x)` `remove(x)`	siler: sessizce / yoksa `KeyError` ile
`union` `intersection` `difference`	`|`, `&`, `-` ile aynı
`issubset(b)` `issuperset(b)`	hepsi `b` içinde mi / `b`'nin hepsini içeriyor mu
`isdisjoint(b)`	hiç ortak eleman yok mu?

▸ COLLAB DÜNYASINDA

Bir CSS'in gerekli bütün partition'ları içerdiğini `required.issubset(css_partitions)` ile kontrol edin: tek satır, tek bir `True` ya da `False`.

1.3.10 bytes

bytes · raw binary data

IMMUTABLE `bytes` MUTABLE `bytearray`

Ağlar ve dosyalar **metin** değil, **byte** taşır. Bir `bytes`, 0 ile 255 arasındaki değerlerden oluşan ham bir byte dizisidir ve `b"..."` biçiminde yazılır. Metin `.encode()` ile bytes'a, `.decode()` ile tekrar metne çevrilir. `bytearray` ise bytes'ın değiştirilebilir sürümüdür.

≈ BENZETME

Bir RTP paketinin içindeki ses, bir **metin** değil, ham byte'lardır. Onu bir insanın okuyabileceği hale getirmek için bir codec gerekir. `encode` / `decode` de aynen bu iş için vardır: metin ↔ byte dönüşümü.

```python
msg = "INVITE sip:1001@cucm-pub.clauto.lab SIP/2.0"
raw = msg.encode()      # str → bytes (UTF-8)
raw[:6]                 # b'INVITE'
raw[0]                  # 73  ← bir byte aslında bir sayı
raw.decode()            # bytes → str

buf = bytearray(raw)    # değiştirilebilir kopya
buf[0:6] = b"CANCEL"    # yerinde değiştir
```

KODU SATIR SATIR OKUYALIM

1 Bir SIP INVITE satırı: sıradan bir metin.

2 `.encode()` metni byte dizisine çevirir. Varsayılan kodlama UTF-8'dir.

3 Dilimleme `bytes`'ta da çalışır; sonuç yine bytes'tır (`b'INVITE'`).

4 Tek bir byte'a indexle erişirseniz bir **sayı** alırsınız: `73`, `"I"` harfinin sayısal karşılığıdır.

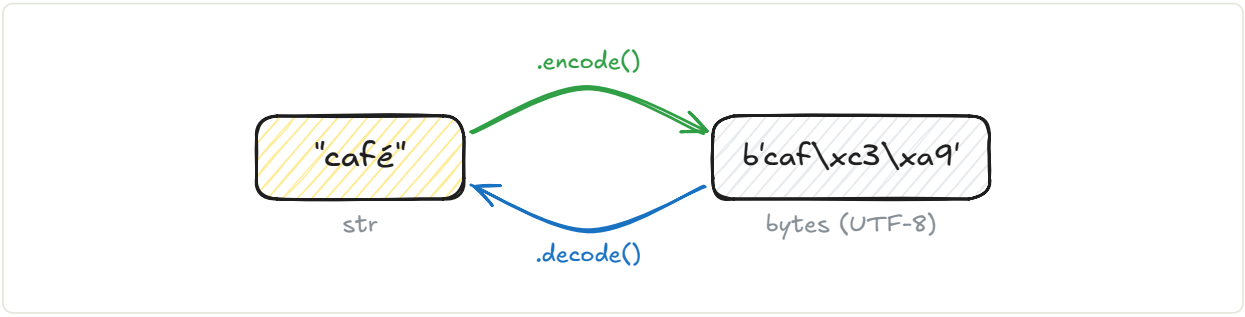
5 `.decode()` bytes'ı tekrar metne çevirir.

7–8 `bytearray` mutable bir kopyadır: `INVITE`'ı `CANCEL` ile değiştirmek mümkündür.

> ▸ **COLLAB DÜNYASINDA**
> `requests` ile bir cevap aldığınızda `r.text` bir `str`, `r.content` ise `bytes`'tır. CDR dosyası ya da sertifika gibi indirmeler ve ham SIP mesajları için `bytes` kullanın.

bytes methods

`bytes`, `str`'nin method'larının çoğunu paylaşır (`split`, `find`, `startswith`, `strip`), ama bütün argümanların da `bytes` olması gerekir: `b"..."`. Asıl önemli adım, ikisi arasında dönüşüm yapmaktır.



Şekil 1.19. `encode()` metni byte'a, `decode()` byte'ı metne çevirir.

```
raw = b"SIP/2.0 200 OK\r\nCSeq: 1 INVITE\r\n"
raw.startswith(b"SIP/2.0 200")  # True
raw.split(b"\r\n")[0].decode()  # 'SIP/2.0 200 OK'
raw.find(b"CSeq")               # 16
raw.hex()[:8]                   # '5349502f'
bytes.fromhex("53495020")       # b'SIP '
"café".encode()                 # b'caf\xc3\xa9'
b"caf\xe9".decode(errors="replace")  # 'caf\ufffd'
```

KODU SATIR SATIR OKUYALIM

1 Bir SIP cevabının ham hali. `\r\n` satır sonudur.

2 Karşılaştırmada argüman da `bytes` olmalı: `b"..."`.

3 Satırlara bölüp ilkini al, sonra metne çevir: cevabın durum satırı.

4 `find()` bir parçanın konumunu verir.

5–6 `hex()` byte'ları onaltılık metne, `fromhex()` tersine çevirir.

7 Türkçe karakterler, `é` gibi, UTF-8'de **birden fazla byte** tutar.

8 Geçersiz byte'ları çözmeye çalışırken `errors="replace"` programın çökmesini önler: bozuk karakter yerine `\ufffd` yazar.

Metot	Ne yapar
`s.encode()`	`str` → `bytes` (varsayılan UTF-8)
`b.decode()`	`bytes` → `str`
`decode(errors="replace")`	bozuk byte'larda çökme
`hex()` `bytes.fromhex(s)`	bytes ↔ onaltılık metin
`split` `find` `startswith`	`str` ile aynı, `b"..."` argümanlarıyla

▸ COLLAB DÜNYASINDA
AXL, HTTP Basic kimlik doğrulaması kullanır. Başlık `base64.b64encode(b"axl_svc:password")` ile üretilir. `requests` bunu sizin yerinize yapar ama altında yatan şey bytes'tır.

1.3.11 None

None · no value

IMMUTABLE

`None`, "burada değer yok" demektir. `NoneType` tipinin tek objesidir. `return` içermeyen bir function `None` döndürür; `dict.get()` olmayan bir key için `None` döndürür. `None` falsy'dir: yani `if` içinde `False` gibi davranır.

```
1 phone = {"name": "SEP0A1B2C3D4E5F",
2          "description": None}
3 desc = phone["description"]
4 desc is None              # True
5 phone.get("css")          # None, key yok
6
7 result = print("hi")      # print None döndürür
8 desc or "no description"  # yedek değer
9 desc.upper()              # AttributeError
```

KODU SATIR SATIR OKUYALIM

1–2 Description alanı boş: AXL'de bu alan genellikle `None` olarak gelir.

3–4 `None`'ı `is None` ile kontrol edin, `== None` ile değil. `None` tek bir objedir, kimlik karşılaştırması yeterlidir.

5 `get()` var olmayan key için `None` verir.

7 `print()` ekrana yazar ama değer döndürmez; bu yüzden `result` `None` olur.

8 `x or default` kalıbı: `x` falsy ise (`None`, `""`, `0`…) yedek değeri kullanır.

9 `None` üzerinde string method'u çağırmak `AttributeError` verir.

× SIK YAPILAN HATA
AXL'deki boş alanlar `None` olarak gelir. Üzerlerinde `desc.upper()` gibi bir string method'u çağırırsanız `AttributeError` alırsınız. Önce `None` olup olmadığını kontrol edin ya da yedek değer kullanın: `(desc or "").upper()`.

Özetle

- Tip seçimi, verinin anlamını belirler: DN `str`, ring sayısı `int`, register durumu `bool`.
- Koleksiyonlar: sıralı ve değiştirilebilir `list`, sabit `tuple`, bir sayı aralığı `range`, isimle erişilen `dict`, benzersiz elemanlı `set`.
- Immutable tipleri (`int`, `str`, `tuple`…) değiştiremezsiniz; mutable tipler (`list`, `dict`, `set`) yerinde değişir ve iki etiket aynı object'i gösteriyorsa ikisinde de görünür.
- AXL'den gelen her şey genelde `str`'dir ve boş alanlar `None` olabilir: dönüştürün, kontrol edin.

✎ Kendin dene

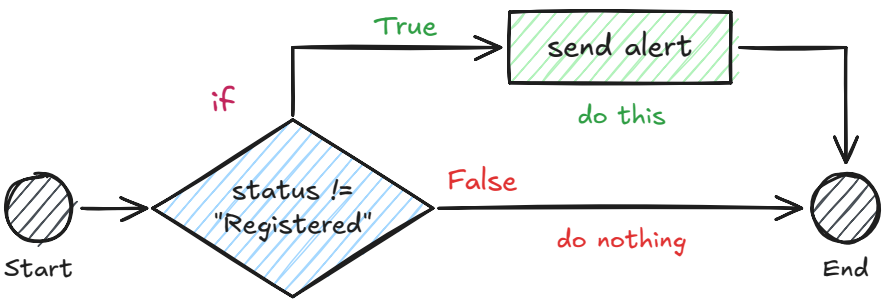
Bir `dict` kurun: `{"name": "SEP0A1B2C3D4E5F", "model": "Cisco 8851"}`. Önce `phone["css"]` yazın ve `KeyError`'ı görün. Sonra `phone.get("css", "CSS_Internal")` ile aynı soruyu hatasız sorun. Son olarak `setdefault` ile `css` key'ini ekleyin.

1.4 if Statements

if · run code only when

Şimdiye kadar yazdığımız script'ler yukarıdan aşağıya, her satırı sırayla çalıştırıyordu. Gerçek işlerde ise **karar vermek** gerekir: "phone register değilse uyarı ver", "DN `2000`'le başlıyorsa hunt pilot'tur". Python'da bunun yolu `if` deyimidir.

> **≈ BENZETME**
> CUBE üzerinde bir **dial-peer** eşleştirmesini düşünün: "çağrının hedefi `9000` ise bu peer'e gönder, değilse bir sonrakine bak." `if` da aynı mantıkla çalışır: **koşul doğruysa** bir blok çalışır, yanlışsa atlanır.



Şekil 1.20. Koşul `True` ise blok çalışır; `False` ise Python bloğu atlayıp devam eder.

```
1 status = "UnRegistered"
2
3 if status != "Registered":
4     print("ALERT: SEP0A1B2C3D4E5F is down")
5
6 print("check finished")   # her zaman çalışır
```

KODU SATIR SATIR OKUYALIM

1. `status` etiketine bir durum metni bağlanır.
3. **Koşul**: `status`, `"Registered"`'a eşit **değil** mi? `!=` "eşit değil" demektir. Satır `:` ile biter.
4. İçeri girintili (indented) satır **bloktur**. Yalnızca koşul `True` ise çalışır. Burada koşul doğru, o yüzden uyarı yazdırılır.
6. Girintisiz satır bloğun **dışındadır**: koşul ne olursa olsun çalışır.

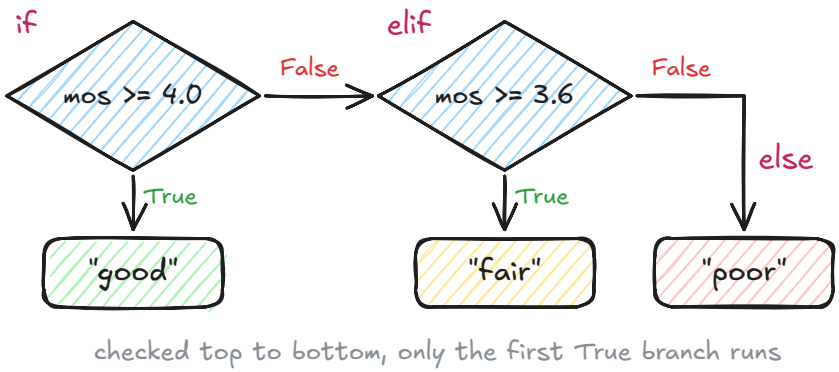
İki şeye dikkat edin. Birincisi, `if` satırı bir **iki nokta** (`:`) ile biter. İkincisi, bloğu parantezler değil **girinti** belirler: bloktaki her satır aynı miktarda (geleneksel olarak 4 boşluk) içeri girer ve girintiye geri dönüldüğünde blok biter. Python'da girinti bir süs değil, sözdiziminin kendisidir.

Operatör	Ne zaman `True`
`==` `!=`	eşit / eşit değil
`<` `>` `<=` `>=`	küçük / büyük (ya da eşit)
`in` `not in`	eleman bir koleksiyonun içinde / dışında
`is` `is not`	aynı object, ör. `x is None`

▸ **COLLAB DÜNYASINDA**
RISPort her device için bir durum döndürür. `if status != "Registered"`, her "phone kapandı" alarmının kalbidir.

if / elif / else · choose one path

Bazen iki değil, çok yol vardır. `elif` ("else if", yani "değilse ve şu doğruysa") istediğiniz kadar ek koşul ekler. `else` ise hiçbir koşula uymayan her şeyi yakalar. İkisi de isteğe bağlıdır.



Şekil 1.21. Python koşulları yukarıdan aşağı sırayla sınar ve ilk `True` olan dalı çalıştırır.

```
1  mos = 3.8
2
3  if mos >= 4.0:
4      quality = "good"
5  elif mos >= 3.6:
6      quality = "fair"
7  else:
8      quality = "poor"
9
10 print(f"MOS {mos}: {quality}")  # MOS 3.8: fair
```

KODU SATIR SATIR OKUYALIM

1 Bir call'ın MOS skoru.

3–4 İlk koşul: `3.8 >= 4.0` yanlış, dal atlanır.

5–6 İkinci koşul: `3.8 >= 3.6` doğru. `quality` `"fair"` olur. Kalan dallar **sınanmaz bile**.

7–8 `else` yalnızca yukarıdaki hiçbiri doğru çıkmadıysa çalışırdı.

10 Her durumda çalışan, bloğun dışındaki satır: sonucu yazdırır.

Python koşulları **yukarıdan aşağıya** kontrol eder ve yalnızca **ilk** `True` olan dalı çalıştırır. Diğer dallar atlanır, hatta onlar da `True` olacak olsa bile.

× SIK YAPILAN HATA

Sıra önemlidir. `mos >= 3.6` koşulunu en üste koyarsanız, `4.5` MOS'u da `"fair"` olarak etiketlenir, çünkü o dal ilk eşleşendir. **En katı koşulu en üste koyun.**

▸ COLLAB DÜNYASINDA

CMR verisinden call kalitesi not edebilirsiniz: MOS 4.0 ve üstü iyi, 3.6 ile 4.0 arası orta, 3.6'nın altı inceleme gerektirir.

Combining conditions

Koşullar birleştirilebilir. Üstelik her değer bir koşul olabilir: boş değerler (`""` , `[]` , `{}` , `0` , `None`) `False` sayılır. Bu, "liste boş mu?" sorusunu çok kısa yazmanızı sağlar.

```
1  status = "UnRegistered"
2  model = "Cisco 8851"
3  lines = []                          # henüz DN yok
4
5  if status != "Registered" and "88" in model:
6      print("8800 series phone is down")
7
8  if not lines:                       # boş → falsy
9      print("phone has no line")
10
11 if model in ("Cisco 8845", "Cisco 8865"):
12     print("video capable")
13
14 label = "OK" if status == "Registered" else "DOWN"
```

KODU SATIR SATIR OKUYALIM

1–3 Üç variable: durum, model ve boş bir line listesi.

5–6 `and` **iki koşulun da** doğru olmasını ister: phone register değil **ve** modeli `"88"` içeriyor.

8–9 `not lines` : `lines` boş bir list olduğu için falsy'dir, `not` onu `True` 'ya çevirir. "Line yoksa" demenin kısa yoludur.

11–12 `in` ile bir değeri birden fazla seçenekle karşılaştırın. Model bu iki video destekli modelden biri değil, o yüzden bu blok çalışmaz.

14 **Tek satırlık seçim**: `x if koşul else y` . Koşul doğruysa `x` , değilse `y` değerini alır.

Operatör	Ne zaman `True`
`a and b`	ikisi de doğru
`a or b`	en az biri doğru
`not a`	`a` yanlış
`x if c else y`	tek satırlık seçim: `c` doğruysa `x` , değilse `y`

× SIK YAPILAN HATA

`if model == "Cisco 8845" or "Cisco 8865":` ifadesi **her zaman** `True` 'dur: boş olmayan `"Cisco 8865"` metni zaten kendi başına truthy'dir. `or` 'un iki tarafı ayrı ayrı koşuldur. Doğrusu: `if model in ("Cisco 8845", "Cisco 8865"):` .

Özetle

- `if` koşul doğruysa bir blok çalıştırır. Blok `:` ile başlar ve girintiyle belirlenir.
- `elif` ve `else` ile birden fazla yol kurulur. Python ilk doğru olan dalı seçer, bu yüzden en katı koşul üste yazılır.
- `and`, `or`, `not` koşulları birleştirir. Boş değerler falsy'dir.

✎ Kendin dene

Bir `jitter` değişkeni (milisaniye) tanımlayın. `20`'nin altı `"good"`, `20` ile `50` arası `"fair"`, üstü `"poor"` yazdırsın. Sonra aynı kodu `elif` yerine hep `if` kullanarak yazmayı deneyin ve fark ne olduğunu gözlemleyin.

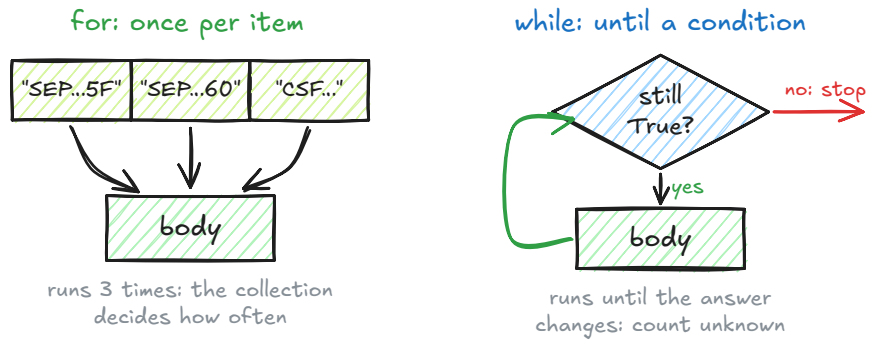
1.5 Loops

What is a loop?

Otomasyonun gerçek gücü burada başlar. Bir **loop** (döngü), bir kod bloğunu tekrar tekrar çalıştırır. İşi bir kere yazarsınız, Python onu ihtiyaç kadar çalıştırır: 3 phone için de, 3000 için de. Otomasyonun zaman kazandırdığı yer burasıdır.

≈ BENZETME

BAT'ta bir CSV yüklediğinizde, sistem her satır için aynı "phone ekle" işlemini tekrarlar. Loop tam olarak budur: **tek bir iş tarifi, birçok kayıt.**



Şekil 1.22. Python'da iki loop vardır: `for` her eleman için bir tur, `while` koşul doğru olduğu sürece tur atar.

```
1 # loop olmadan: phone başına bir satır
2 print("reset", "SEP0A1B2C3D4E5F")
3 print("reset", "SEP0A1B2C3D4E60")
4 # ... 498 satır daha
5
6 # loop ile: kaç tane olursa olsun iki satır
7 for name in phones:
8     print("reset", name)
```

KODU SATIR SATIR OKUYALIM

1–4 Loop olmadan her phone için ayrı bir satır yazmanız gerekir. 500 phone için 500 satır, ve her satırda bir yazım hatası riski.

6–8 Loop ile **aynı iş iki satıra** sığar. `phones` bir list'tir ve `name` her turda sıradaki phone'un adını alır.

Şu terimlere alışın, loop'ları anlatırken sürekli geçecekler:

Terim	Anlamı
body	tekrar edilen, içeri girintili blok
iteration	body'nin bir kez çalışması (bir "tur")
loop variable	o anki elemanı tutan isim
condition	devam edilip edilmeyeceğine karar veren `True` / `False` testi

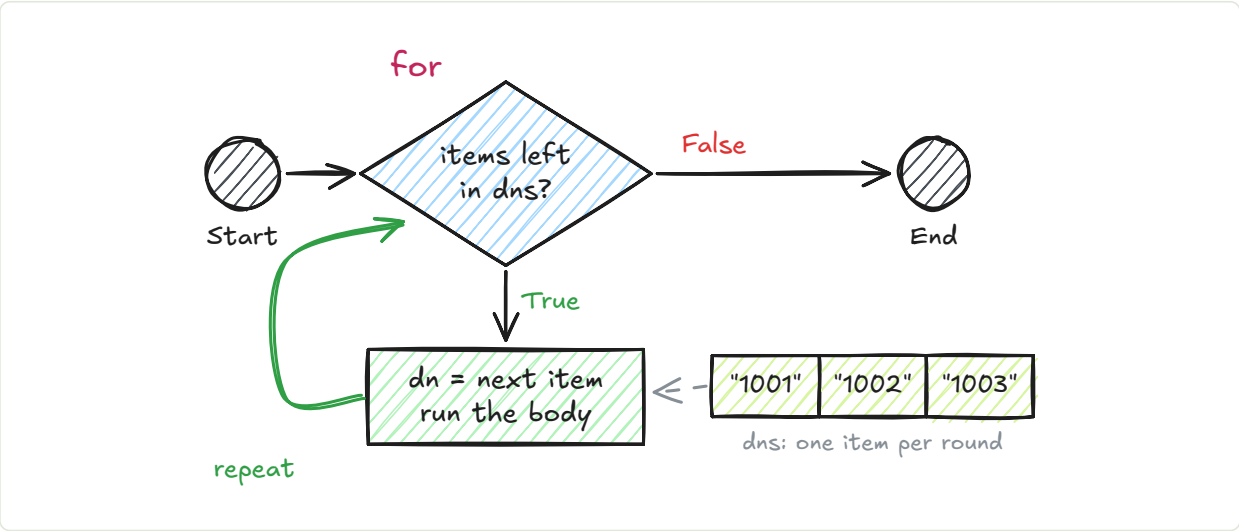
Python'un iki loop'u vardır. `for` bir koleksiyonun **her elemanı için bir kez** tekrarlar. `while` ise bir **koşul** doğru olduğu sürece tekrarlar.

> ▸ **COLLAB DÜNYASINDA**
> 500 phone'u reset etmek, denetlemek ya da güncellemek: tek bir phone için gereken kod body'ye yazılır, geri kalan 499'u loop halleder.

1.5.1 for

for · repeat for every item

`for` loop'u bir koleksiyonun **her elemanı için body'yi bir kez** çalıştırır. Her turda loop variable'ı (`dn`) bir sonraki elemanı alır. Eleman kalmayınca loop biter ve loop'un altındaki kod çalışır.



Şekil 1.23. Her turda sıradaki eleman loop variable'ına bağlanır. Eleman bitince loop sona erer.

```
1 dns = ["1001", "1002", "1003"]
2
3 for dn in dns:
4     print(f"Adding line {dn}")
5
6 print("done")          # loop'tan sonra
```

KODU SATIR SATIR OKUYALIM

1. Üç DN'li bir list.
3. `for dn in dns:` şöyle okunur: "`dns` içindeki her `dn` için". Satır `if` gibi `:` ile biter.
4. Girintili body, her tur çalışır. `f"..."` içindeki `{dn}` yerine o turdaki DN yazılır.
6. Girintisiz satır loop'un dışındadır; yalnızca bir kere, en sonda çalışır.

ÇIKTI

```
Adding line 1001
Adding line 1002
Adding line 1003
done
```

`for`, üzerinde adım adım dolaşabilen her şeyle çalışır: `list`, `tuple`, `str`, `dict`, `set`, `range` ve ilerleyen bölümlerde dosyalar ve API cevapları.

▸ COLLAB DÜNYASINDA

Toplu provisioning'in kalbi budur: bir CSV'den 500 satır okuyun, sonra `for row in rows:` ile her satır için bir AXL `addLine` ya da `addPhone` isteği kurup gönderin.

for · common patterns

Otomasyon script'lerindeki loop'ların çoğu beş kalıptan biridir. Bunları ezberlemeniz gerekmez ama tanımanız, kod okumayı çok kolaylaştırır.

```
1  for n in range(1001, 1004):            # sayılar
2      print(str(n))
3
4  for i, dn in enumerate(dns, start=1):  # sıra + eleman
5      print(i, dn)                       # 1 1001 …
6
7  for key, value in phone.items():       # dict çiftleri
8      print(key, value)
9
10 for dn, user in zip(dns, users):       # iki list yan yana
11     print(dn, "->", user)              # 1001 -> alice.smith
12
13 total = 0
14 for d in durations:                    # biriktirme
15     total += d                         # 937
```

KODU SATIR SATIR OKUYALIM

1–2 `range` ile bir sayı bloğu üzerinde tekrarlanır: DN'leri üretmek için ideal.

4–5 `enumerate()` iki şey birden verir: **sıra numarasını** ve **elemanı**. Soldaki iki isim bunları alır.

7–8 `dict.items()` her turda bir `(key, value)` çifti verir.

10–11 `zip()` iki list'i yan yana yürütür: birinci turda iki list'in ilk elemanları, ikinci turda ikinci elemanlar…

13–15 **Biriktirme** kalıbı: boş bir başlangıç değeri (`0`), her turda eklenen bir şey (`+=`). Loop bittiğinde toplam hazırdır.

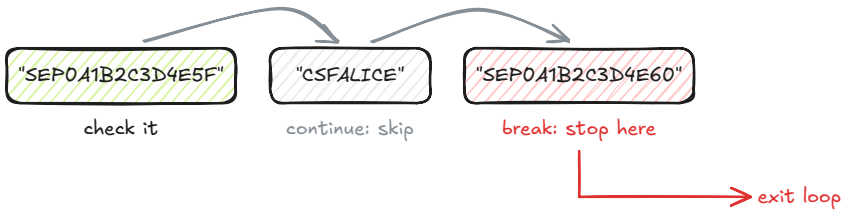
Kalıp	Ne için
`range(a, b)`	bir sayı bloğu için tekrarla
`enumerate(seq)`	sıra numarasını ve elemanı al
`d.items()`	bir dict'i key key gez
`zip(a, b)`	iki list'i yan yana gez
`total += x`	toplam, sayaç ya da yeni list biriktir

▸ COLLAB DÜNYASINDA

`zip(dns, users)` bir Excel sayfasındaki DN sütununu ve kullanıcı sütununu eşleştirir, böylece her line'ı sahibiyle ilişkilendirebilirsiniz.

break and continue

Loop'un normal akışını iki anahtar kelime değiştirir. `continue` o turun geri kalanını atlayıp **sonraki tura** geçer. `break` ise loop'tan **hemen çıkar**, kalan elemanlara bakılmaz.



Şekil 1.24. `continue` yalnızca bu turu atlar. `break` loop'un tamamını bitirir.

```
1  phones = ["SEP0A1B2C3D4E5F", "CSFALICE",
2            "SEP0A1B2C3D4E60"]
3
4  for name in phones:
5      if name.startswith("CSF"):
6          continue          # atla, sonraki elemana geç
7      if name == "SEP0A1B2C3D4E60":
8          print("found", name)
9          break             # loop'tan hemen çık
10     print("checking", name)
```

KODU SATIR SATIR OKUYALIM

1–2 Üç device: iki masa telefonu ve bir Jabber client'ı.

4 Her `name` için aşağıdaki body çalışır.

5–6 Device adı `CSF` ile başlıyorsa (Jabber), `continue` o turun kalanını atlar. `print("checking", ...)` çalışmaz.

7–9 Aranan device bulunduysa, mesaj yazdırılır ve `break` ile loop'tan çıkılır.

10 Yukarıdaki iki koşulun hiçbiri geçerli değilse buraya gelinir.

ÇIKTI

```
checking SEP0A1B2C3D4E5F
found SEP0A1B2C3D4E60
```

İkisi de genellikle bir `if`'in içinde kullanılır ve `for` ile `while`'da aynı şekilde çalışır.

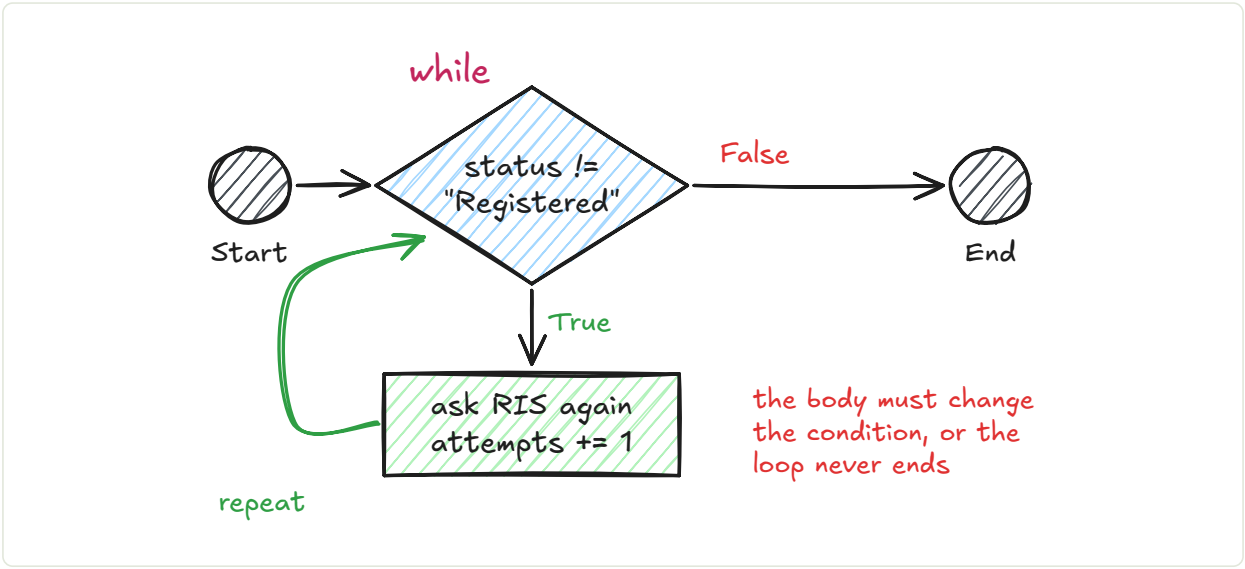
▸ **COLLAB DÜNYASINDA**

Masa telefonu denetiminde Jabber client'larını (`CSF…`) `continue` ile atlayın. Aradığınız device'ı bulur bulmaz `break` yapın, böylece script geriye kalan binlerce phone'u gezmez.

1.5.2 while

while · repeat while true

`while` loop'u koşulu `True` olduğu sürece body'yi tekrarlar. Koşul **her turdan önce** kontrol edilir. `False` olduğu anda loop biter. `for`'dan farkı: kaç tur atacağınızı önceden bilmediğiniz durumlarda kullanılır, ör. "phone register olana kadar", "iş bitene kadar".



Şekil 1.25. Koşul her turdan önce sınanır. Koşul `False` olunca loop biter.

```
1 replies = ["Unknown", "Unknown", "Registered"]
2 attempts = 0
3 status = ""
4
5 while status != "Registered" and attempts < 5:
6     status = replies[attempts]   # RIS cevabı
7     attempts += 1
8     print(f"try {attempts}: {status}")
```

KODU SATIR SATIR OKUYALIM

1. Bir phone reset sonrası RIS'ten gelecek üç cevabı taklit eden bir list.
2–3. Sayaç ve başlangıç durumu.
5. İki şart birden: phone henüz register **değil** ve deneme sayısı 5'in altında. İkisi birden doğru olduğu sürece devam.
6. Sıradaki cevap alınır.
7. Sayaç artar. Bunu unutursanız `attempts` hiç değişmez ve loop sonsuza kadar sürebilir.
8. Her turda durum yazdırılır.

ÇIKTI

```
try 1: Unknown
try 2: Unknown
try 3: Registered
```

× SIK YAPILAN HATA

Body'nin içinde koşulu değiştiren hiçbir şey yoksa loop **hiç bitmez** (sonsuz loop). Her zaman `attempts < 5` gibi bir sınır koyun. Takılan bir script'i `Ctrl+C` ile durdurursunuz.

while True · poll until done

`while True` bilerek sonsuza kadar döner. Çıkış kurallarını body'nin içine `break` ile yazarsınız. Birden fazla çıkış şartı olduğunda bu yazım çok okunurdur. **Polling** (bir şeyin bitmesini tekrar tekrar sorarak beklemek) için en yaygın kalıptır.

```python
import time

attempts = 0
while True:
    attempts += 1
    status = get_status("SEP0A1B2C3D4E5F")  # sizin RIS çağrınız
    if status == "Registered":
        print("phone is back")
        break
    if attempts == 10:
        print("giving up after 10 tries")
        break
    time.sleep(30)          # tekrar sormadan önce bekle
```

KODU SATIR SATIR OKUYALIM

1 `time` standart kütüphane modülüdür; `import` konusunu ilerleyen bölümde göreceğiz.

4 `while True` koşulu her zaman doğrudur: loop kendi kendine bitmez, çıkışı biz yazarız.

6 Cevabı kendi yazacağınız `get_status` function'ından alıyoruz. (Function'ları birazdan öğreneceğiz.)

7–9 **Birinci çıkış**: phone register oldu, mesaj yaz ve `break`.

10–12 **İkinci çıkış**: 10 deneme yapıldı, vazgeç.

13 `sleep(30)` 30 saniye bekler. İstekler arasında bekleme koymazsanız sunucuyu boğarsınız.

Loop	Ne zaman
`for`	elinizde bir koleksiyon ya da bilinen bir sayı varsa
`while koşul`	bir koşul değişene kadar tekrarlamak istiyorsanız
`while True` + `break`	birden fazla çıkış kuralı, polling

▸ **COLLAB DÜNYASINDA**

Bir phone'u reset ettikten sonra tekrar register olana kadar RIS'i sorgulayın. RIS'in istek sınırı vardır; bu yüzden istekler arasında **her zaman** `sleep` kullanın ve deneme sayısını sınırlayın.

Özetle

- Loop, bir kod bloğunu tekrar eder: `for` bir koleksiyonun her elemanı için, `while` bir koşul doğru olduğu sürece.
- `enumerate`, `zip`, `items()` ve `range` en yaygın `for` kalıplarıdır.
- `continue` turu atlar, `break` loop'tan çıkar.
- `while` loop'larında **her zaman** bir çıkış yolu ve bir deneme sınırı olmalı.

✎ Kendin dene

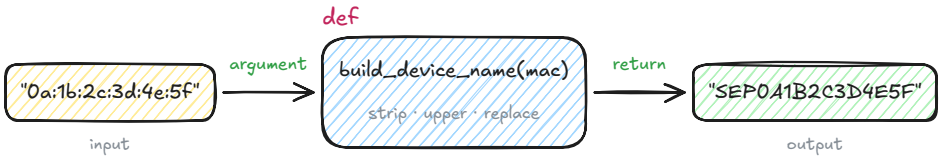
`dns = ["1001", "1002", "1003"]` ve `users = ["alice.smith", "bob.jones", "carol.white"]` list'lerini `zip` ile gezip `"alice.smith -> 1001"` biçiminde yazdırın. Sonra `continue` ile `"1002"` DN'ini atlayın.

1.6 Functions

What is a function?

Aynı işi ikinci kez yazdığınızı fark ettiğinizde, bir **function** yazma zamanı gelmiştir. Function (fonksiyon), belirli bir işi yapan, **isim verilmiş** bir kod bloğudur. Bir kere `def` ile **tanımlarsınız** (define), sonra istediğiniz kadar, her seferinde farklı girdiyle ismiyle **çağırırsınız** (call).

≈ BENZETME
CUCM'deki bir **translation pattern** ya da bir CUBE **voice translation-profile** düşünün: kuralı bir yere yazarsınız, sonra onlarca dial-peer o kurala başvurur. Kuralı değiştirirseniz hepsi etkilenir. Function da böyledir: kuralı tek yerde yazarsınız, her script onu kullanır.



Şekil 1.26. Function bir makine gibidir: girdiyi alır, işler ve sonucu geri verir.

```
1 def build_device_name(mac):
2     """Turn a raw MAC into a CUCM device name."""
3     clean = mac.strip().upper().replace(":", "")
4     return "SEP" + clean
5
6 name = build_device_name("0a:1b:2c:3d:4e:5f")
7 print(name)        # SEP0A1B2C3D4E5F
```

KODU SATIR SATIR OKUYALIM

1. `def` function tanımını başlatır. `build_device_name` function'ın adı, parantez içindeki `mac` ise **parameter**'dir (girdinin adı). Satır `:` ile biter.
2. Üç tırnaklı satır bir **docstring**'dir: function'ın ne yaptığını anlatan tek satırlık bir açıklama.
3. Temizlik: boşlukları sil, büyük harfe çevir, iki nokta üst üsteleri kaldır. `clean` yalnızca function'ın içinde yaşar.
4. `return` sonucu **geri verir** ve function'dan çıkar.
6. Function'ı çağırıyoruz. Parantez içine verdiğimiz `"0a:1b:..."` değerine **argument** denir. Dönen sonuç `name` etiketine bağlanır.
7. Function bir değer döndürdü; biz yazdırdık.

Parameter ile argument karıştırılır: **parameter** tanımdaki isimdir (`mac`), **argument** çağrıda verdiğiniz gerçek değerdir (`"0a:1b:..."`).

Parça	Anlamı
`def name(...):`	function'ı tanımlar; body altında girintili
parameter	tanımdaki girdi adı (`mac`)
argument	çağrıda verilen gerçek değer
`return`	sonucu geri verir ve function'dan çıkar
`"""docstring"""`	ne yaptığını anlatan tek satır

▸ COLLAB DÜNYASINDA
Birden fazla yerde uyguladığınız her kural (isimlendirme, temizleme, doğrulama) bir function'a aittir. Tek yerde düzeltirsiniz, bütün script'ler yararlanır.

Parameters, defaults and return

Function'lar birden fazla parameter alabilir ve bazı parameter'lara varsayılan (default) değer verilebilir. Bir function ayrıca birden fazla değer döndürebilir.

```python
def line_label(dn, user, site="HQ"):
    return f"{user} ({dn}) - {site}"

line_label("1001", "alice.smith")    # alice.smith (1001) - HQ
line_label("1002", "bob.jones", "Branch")      # sıraya göre
line_label(user="carol.white", dn="1003")      # isme göre

def check(status):
    if status == "Registered":
        return True, "ok"
    return False, f"status is {status}"

ok, msg = check("UnRegistered")    # iki değer döner (tuple)
```

KODU SATIR SATIR OKUYALIM

1 `site="HQ"` bir **varsayılan değerdir**: bu parameter verilmezse `"HQ"` kullanılır.

2 Function'ın sonucu bir f-string'dir.

4 `site` verilmediği için varsayılan `"HQ"` devreye girer.

5 Üç argument, tanımdaki **sıraya göre** eşleşir. Buna **positional** argument denir.

6 Argument'ları **isimle** verebilirsiniz (`user="..."`). Bu durumda sıra önemli değildir. Buna **keyword** argument denir.

8-11 Bu function iki farklı noktada `return` kullanıyor: birincisi koşul doğruysa, ikincisi aksi halde.

10 `return a, b` aslında bir **tuple** döndürür: `(True, "ok")`.

13 Dönen iki değeri tek satırda iki isme açmak (**unpacking**): `ok` bir `bool`, `msg` bir mesaj.

- **Positional** argument'lar sıraya göre, **keyword** argument'lar isme göre eşleşir.
- **Varsayılan değer** (`site="HQ"`) bir parameter'ı isteğe bağlı yapar.
- `return a, b` bir tuple döndürür. `return` yazılmayan bir function `None` döndürür.
- Function içinde yaratılan isimler **local**'dir: function bitince yok olurlar.

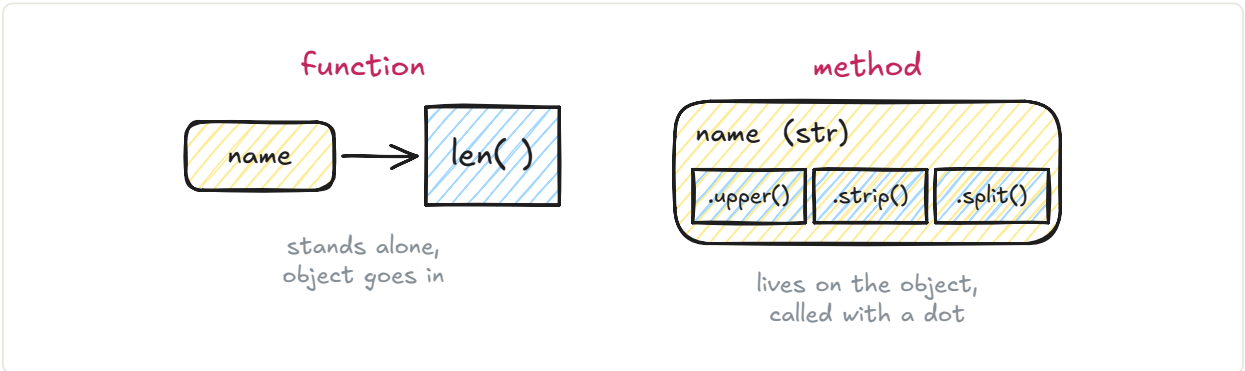
× SIK YAPILAN HATA

Asla mutable bir varsayılan değer kullanmayın: `def add(dn, lines=[])` bütün çağrılar arasında **tek bir** list paylaşır. Önceki çağrıdan kalan eleman bir sonrakinde görünür. Bunun yerine `lines=None` yazın, içeride `if lines is None: lines = []` ile taze bir list yaratın.

Function vs method

Şimdiye kadar iki tür çağrı gördünüz: `len(name)` ve `name.upper()`. İkisi de bir iş yapar ama aralarında bir fark var.

Method, bir object'in **tipine ait** function'dır. Objenin üzerinde nokta ile çağırırsınız ve obje method'a otomatik olarak geçirilir. Düz bir **function** ise tek başına durur ve objeyi argument olarak alır.



Şekil 1.27. Function'a object'i siz verirsiniz; method'u ise object'in üzerinde nokta ile çağırırsınız.

```python
name = "sep0a1b2c3d4e5f"

len(name)              # function: object içeri girer
name.upper()           # method: object üzerinde çağrılır
str.upper(name)        # aynı method, function biçiminde

lines = ["1001"]
lines.append("1002")   # list method'u
len(lines)             # her tip için tek bir len()

name.append("x")       # AttributeError: str'de
                       # böyle bir method yok
```

KODU SATIR SATIR OKUYALIM

3 `len()` bir function'dır; üzerinde çalışacağı object'i **argument** olarak alır. Birçok tipte çalışır.

4 `upper()` bir method'dur; nokta, "bu object'in `upper`'ını çağır" demektir.

5 Aynı method'u tipin adı üzerinden de çağırabilirsiniz; bu, perde arkasında olanı gösterir.

7–8 `append` yalnızca `list`'te vardır.

9 `len()` ise her koleksiyon tipinde aynı çalışır.

11–12 `str`'de `append` diye bir method yoktur: `AttributeError`.

Function	Method
`len(name)`	`name.upper()`
bir modülde `def` ile tanımlanır	bir class'ın içinde `def` ile tanımlanır
birçok tipte çalışır	yalnızca kendi tipinde bulunur
object argument olarak verilir	object otomatik geçer (`self`)

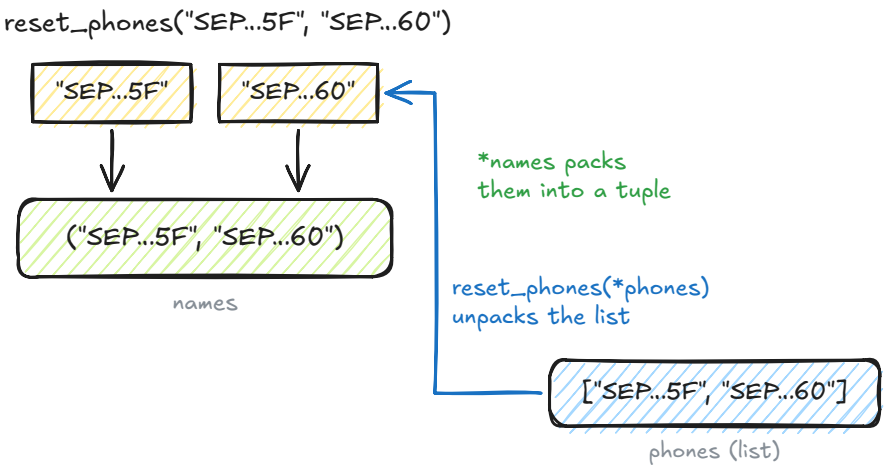
! DIKKAT

Her nokta bir method çağrısı değildir. `requests.get(url)` ya da `json.loads(text)` çağrılarında nokta yalnızca bir **module**'ü açar: `get` ve `loads` düz function'lardır.

1.6.1 args and kwargs

*args · any number of positional arguments

Bazen bir function'a kaç argument verileceğini önceden bilemezsiniz: bir phone mu, on phone mu? Tek yıldızlı bir parameter, **fazladan gelen bütün positional argument'ları** bir `tuple` içinde toplar.



Şekil 1.28. `*names` bütün fazla argument'ları bir tuple içinde toplar.

```
1  def reset_phones(*names):
2      print(type(names))      # <class 'tuple'>
3      for name in names:
4          print("reset", name)
5
6  reset_phones("SEP0A1B2C3D4E5F")
7  reset_phones("SEP0A1B2C3D4E5F", "SEP0A1B2C3D4E60")
8  reset_phones(*phones)       # bir list'i aç
```

KODU SATIR SATIR OKUYALIM

1 `*names`: yıldız, "gelen bütün positional argument'ları `names` isimli bir tuple'a topla" demektir.

2 İçeride `names` gerçekten bir `tuple`'dır.

3–4 Tuple üzerinde bir `for` loop'u ile her birini işleriz.

6 Tek argument: `names` bir elemanlı tuple olur.

7 İki argument: iki elemanlı tuple.

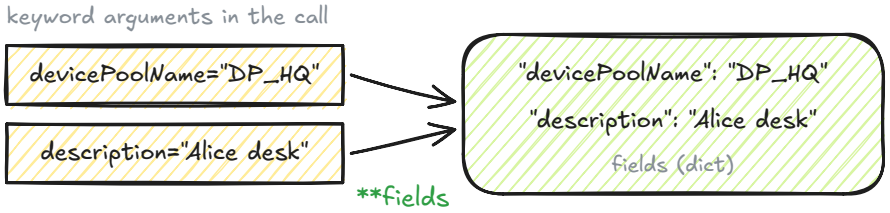
8 **Çağrıda** yıldız ters yönde çalışır: `*phones` bir list'i ayrı ayrı argument'lara **açar**.

Yıldızdan sonraki isim serbesttir; daha iyi bir isim yoksa gelenek `*args`'tır. Normal parameter'lar önce gelebilir: `def reset(site, *names)`.

▸ COLLAB DÜNYASINDA
Tek bir `reset_phones()` yardımcısı hem komut satırından tek bir phone'a hem de bir CSV'den gelen koca bir listeye hizmet eder; function'ın iki sürümünü yazmanız gerekmez.

**kwargs · any number of keyword arguments

İki yıldızlı parameter, **fazladan gelen bütün keyword argument'ları** bir `dict` içinde toplar: isimler key, değerler value olur.



Şekil 1.29. `**fields` isimli bütün fazla argument'ları bir dict içinde toplar.

```
1  def add_phone(name, model, **fields):
2      phone = {"name": name, "model": model}
3      phone.update(fields)       # fields bir dict
4      return phone
5
6  add_phone("SEP0A1B2C3D4E5F", "Cisco 8851",
7            devicePoolName="DP_HQ",
8            description="Alice desk")
9
10 defaults = {"devicePoolName": "DP_HQ"}
11 add_phone("SEP0A1B2C3D4E60", "Cisco 8865",
12           **defaults)          # dict'i aç
```

KODU SATIR SATIR OKUYALIM

1 `name` ve `model` zorunlu. `**fields` bunların dışında gelen **bütün isimli** argument'ları toplar.

2 Zorunlu alanlarla başlayan bir `dict`.

3 `update()` fazladan gelen alanları dict'e ekler.

6–8 Çağrıda `devicePoolName` ve `description` isimleriyle iki fazladan argument verilir. Bunlar `fields` dict'ine girer.

10–12 Çağrıda `**` bir dict'i isimli argument'lara **açar**. Ortak varsayılanları bir kere tanımlayıp her çağrıya geçirmenin yolu.

Sözdizimi	Anlamı
`def f(*args)`	fazladan positional → `tuple`
`def f(**kwargs)`	fazladan keyword → `dict`
`f(*lst)`	list'i positional argument'lara aç
`f(**d)`	dict'i keyword argument'lara aç

Tanımda sıra: normal parameter'lar, `*args`, keyword-only parameter'lar, `**kwargs`.

▸ COLLAB DÜNYASINDA

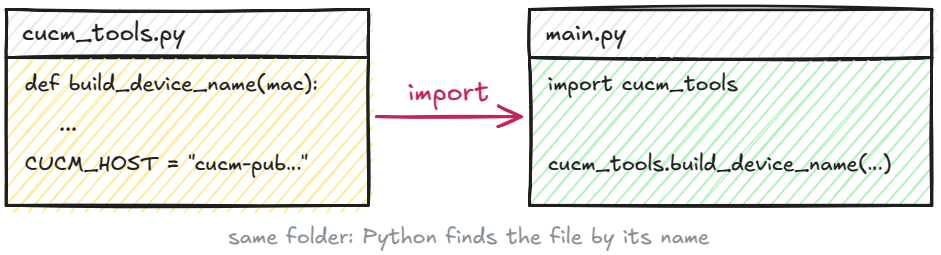
Bir AXL phone'unun onlarca isteğe bağlı alanı vardır. `**fields` ile tek bir yardımcı function, hepsini tek tek listelemeden bunların herhangi birini kabul eder.

1.6.2 Modules and import

Modules · use code from another file

Script'leriniz büyüdükçe bütün kodu tek dosyaya yazmak sürdürülemez hale gelir. Python'un çözümü **module**'dür: her `.py` dosyası bir module'dür. `import` başka bir dosyayı bir kere yükler ve onun function'larını ve variable'larını kullanmanıza izin verir. Böylece kodu bir yerde yazarsınız, her yerde kullanırsınız.

≈ BENZETME
Bir CUCM cluster'ında her şeyi tek bir partition'a koymazsınız; sorumluluğa göre ayırırsınız ve bir CSS ile erişim verirsiniz. Module'ler de böyledir: kodu konularına göre ayırır, ihtiyacı olan script'in o module'e erişmesine `import` ile izin verirsiniz.



Şekil 1.30. `main.py`, `cucm_tools.py` dosyasındaki kodu `import` ile kullanır.

```
1 # main.py
2 import cucm_tools
3 cucm_tools.build_device_name("0a:1b:2c:3d:4e:5f")
4
5 from cucm_tools import build_device_name, CUCM_HOST
6 build_device_name("0a:1b:2c:3d:4e:60")
7
8 import cucm_tools as ct     # kısa takma ad
9 ct.CUCM_HOST                # 'cucm-pub.clauto.lab'
```

KODU SATIR SATIR OKUYALIM

2–3 `import cucm_tools` bütün module'ü getirir. Function'ı `cucm_tools.` önekiyle çağırırsınız: nereden geldiği açıktır.

5–6 `from ... import ...` yalnızca istediğiniz isimleri alır; önek olmadan kullanırsınız. Daha kısadır, ama isim nereden geldi unutulabilir.

8–9 `import ... as ct` uzun bir isme kısa bir takma ad (alias) verir.

Biçim	Sonra kullanım
`import m`	`m.func()` : nereden geldiği net
`from m import func`	`func()` : daha kısa, isim seçilir
`import m as x`	`x.func()` : uzun isimler için takma ad

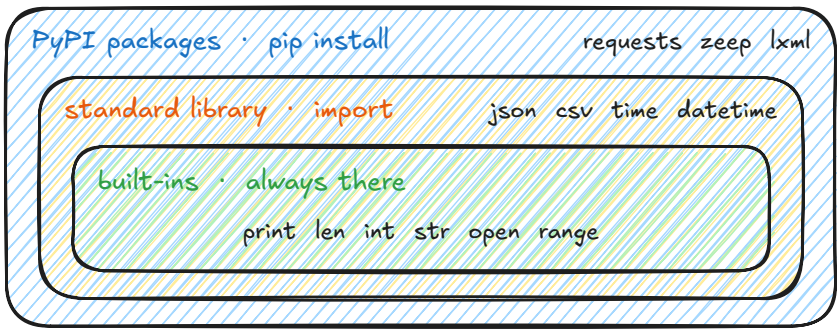
Python, module'leri şu sırayla arar: önce script'in kendi klasörü, sonra standart kütüphane, sonra kurulu paketler.

× SIK YAPILAN HATA
Kendi dosyanıza asla gerçek bir module'ün adını vermeyin: klasörünüzdeki bir `json.py` ya da `requests.py` gerçeğini gizler ve `import json` kendi dosyanızı yükler. Beklenmedik hatalar buradan çıkar.

Standard library and packages

Hazır kod üç katmandan gelir:

- **Built-in**'ler (`print` , `len`) için hiçbir `import` gerekmez.
- **Standart kütüphane**, Python ile birlikte gelir: `import json` .
- **Üçüncü parti paketler** (third-party packages), PyPI'den gelir: `pip install requests` , sonra `import requests` .



Şekil 1.31. Üç katman: built-in'ler, standart kütüphane, PyPI'den kurulan paketler.

```
1 import json, time                # standart kütüphane
2 from datetime import datetime
3
4 json.loads('{"dn": "1001"}')     # str → dict
5 time.sleep(2)                    # 2 saniye bekle
6 datetime.now().strftime("%Y-%m-%d")
7
8 import requests                  # pip install requests
```

KODU SATIR SATIR OKUYALIM

1. Virgülle iki module birden getirilebilir. İkisi de Python ile gelir; kurmanız gerekmez.
2. `datetime` module'ünden yalnızca `datetime` adını alıyoruz.
4. `json.loads()` bir JSON metnini Python `dict`'ine çevirir.
5. `time.sleep()` verilen saniye kadar bekler.
6. Bugünün tarihini `2026-10-20` biçiminde verir.
8. `requests` standart kütüphanede **yoktur**: önce `pip install requests` ile kurulması gerekir.

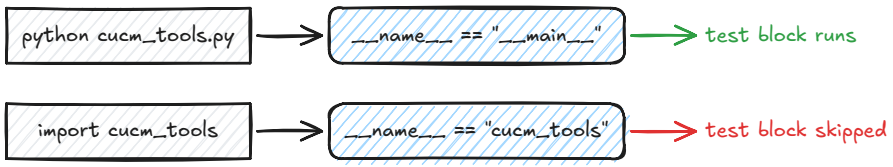
Paketleri proje başına bir **virtual environment** (sanal ortam) içine kurun. Böylece her projenin kendi sürümleri olur ve projeler birbirini bozmaz:

```
python3 -m venv .venv && source .venv/bin/activate
pip install requests zeep
```

> ▸ **COLLAB DÜNYASINDA**
> REST için (Webex, CUPI) `requests`, SOAP için (AXL, RIS) `zeep`, bunların etrafındaki veri için ise `json` ve `csv` kullanacaksınız.

Scripts vs modules · __name__

Aynı dosya hem doğrudan çalıştırılabilir hem de başka bir dosyadan `import` edilebilir. Python bunları ayırt etmek için `__name__` isimli özel bir variable kullanır: dosyayı doğrudan çalıştırırsanız `__name__` değeri `"__main__"` olur; başka bir dosyadan import ederseniz module'ün adı olur.



Şekil 1.32. Dosya doğrudan çalıştırılırsa `__name__` `"__main__"` olur; import edilirse module adı.

```
1 # cucm_tools.py
2 def build_device_name(mac):
3     return "SEP" + mac.replace(":", "").upper()
4
5 if __name__ == "__main__":
6     # yalnızca doğrudan çalıştırınca
7     print(build_device_name("0a:1b:2c:3d:4e:5f"))
```

KODU SATIR SATIR OKUYALIM

2–3 Normal bir function tanımı. Dosya `import` edildiğinde de, doğrudan çalıştırıldığında da tanımlanır.

5 `__name__ == "__main__"` yalnızca dosya **doğrudan çalıştırıldığında** doğrudur.

6–7 Bu blok bir kendi kendine test gibidir: `import` edildiğinde çalışmaz.

Module'lerden oluşan bir klasör bir **package**'dır. Noktalarla import edersiniz:

```
clauto/
├── main.py          from cucm.axl import add_phone
└── cucm/
    ├── __init__.py  (klasörü package yapar)
    ├── axl.py
    └── ris.py
```

> ▸ **COLLAB DÜNYASINDA**
> AXL, RIS ve Webex yardımcılarını kendi module'lerinde tutun. Her yeni script copy-paste yerine iki `import` satırıyla başlar.

Özetle

- Function, tekrar eden işe isim verir: `def`, parameter'lar ve `return`.
- Argument'lar positional ya da keyword olabilir. `*args` ve `**kwargs` istenen sayıda argument'ı toplar.
- Module her `.py` dosyasıdır; `import` ile kullanılır. Standart kütüphane hazırdır, üçüncü parti paketler `pip` ile kurulur.
- Proje başına bir virtual environment kullanın.

✎ Kendin dene

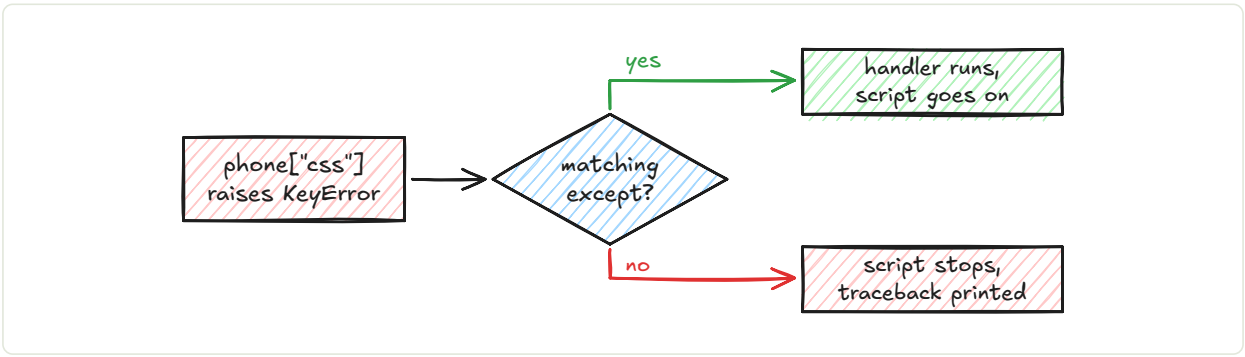
`to_extension(dn, prefix="+90")` adında bir function yazın: DN'in başına prefix ekleyip döndürsün. Hem varsayılanla (`to_extension("1001")`) hem de `prefix="+1"` ile çağırın. Sonra function'ı `cucm_tools.py` dosyasına taşıyıp `main.py`'den `import` edin.

1.7 Exceptions

What is an exception?

Programlar hata verir. Bu kaçınılmazdır ve kötü bir şey değildir: hata mesajı, Python'un size neyin ters gittiğini söylemesidir. Bir **exception** (istisna), Python'un "burada devam edemiyorum" deme biçimidir. Hiçbir şey onu yakalamazsa script durur ve bir **traceback** yazdırır.

> **≈ BENZETME**
> Bir SIP çağrısında `404 Not Found` ya da `503 Service Unavailable` cevabı gördüğünüzde çağrı kurulamaz ama sistemin neden vazgeçtiğini cevap kodundan anlarsınız. Exception da aynı: her tipin bir adı vardır ve ne olduğunu söyler.



Şekil 1.33. Bir hata oluşursa ve hiçbir şey onu yakalamazsa, script durur ve traceback yazdırılır.

```
1 phone = {"name": "SEP0A1B2C3D4E5F"}
2 phone["css"]
```

ÇIKTI

```
Traceback (most recent call last):
  File "audit.py", line 2, in <module>
    phone["css"]
KeyError: 'css'
```

Traceback'i **aşağıdan yukarıya** okuyun. **Son satır** hatanın tipini ve mesajını söyler: burada `KeyError: 'css'`, yani "`css` diye bir key yok". Üstündeki satırlar hatanın **nerede** olduğunu gösterir: hangi dosya, hangi satır, hangi kod.

KODU SATIR SATIR OKUYALIM

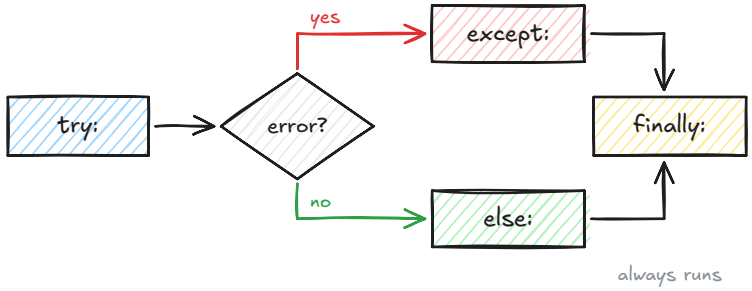
1. Yalnızca bir `name` key'i olan bir dict.
2. Olmayan bir key'i okumaya çalışıyoruz. Python bunu yapamaz ve exception yükseltir (**raise**).

Exception	Tipik neden
`KeyError`	dict'te olmayan bir key
`IndexError`	list'te olmayan bir konum
`ValueError`	doğru tip, yanlış değer: `int("84x3")`
`TypeError`	yanlış tip: `"1001" + 1`
`AttributeError`	olmayan bir method: `None.upper()`
`FileNotFoundError`	CSV yolu yanlış

▸ **COLLAB DÜNYASINDA**
Kütüphaneler kendi exception'larını ekler: `requests` bir `Timeout` yükseltir, `zeep` ise AXL bir isteği reddettiğinde `Fault` yükseltir (aynı isimde bir kayıt var, olmayan bir CSS gibi).

try / except / else / finally

Riskli kodu `try` içine yazarsınız. Belirttiğiniz tipte bir exception olursa, Python durmak yerine ilgili `except` bloğuna atlar.



Şekil 1.34. `try` hatasız biterse `else`, hata olursa `except` çalışır. `finally` her durumda çalışır.

```
1  def to_int(text):
2      try:
3          value = int(text)
4      except ValueError:
5          print(f"not a number: {text!r}")
6          value = None
7      else:
8          print("converted", value)
9      finally:
10         print("checked", text)
11     return value
12
13 to_int("8443")    # converted 8443, checked 8443
14 to_int("84x3")    # not a number: '84x3', checked 84x3
```

KODU SATIR SATIR OKUYALIM

1 Bir metni sayıya çeviren bir function. Metin geçersizse `int()` `ValueError` yükseltir.

2-3 `try` bloğu: riskli iş burada. Hata olmazsa blok sonuna kadar çalışır.

4-6 `except ValueError` bloğu: yalnızca `ValueError` olduğunda çalışır. Programı durdurmak yerine bir mesaj yazdırıp `value`'yu `None` yaparız. `!r` değeri tırnaklarıyla gösterir; boşluk gibi gizli karakterleri yakalamak için işe yarar.

7-8 `else` bloğu **yalnızca hiç hata olmadığında** çalışır.

9-10 `finally` hata olsa da olmasa da **her zaman** çalışır: temizlik işleri için.

11 Sonuç döndürülür.

13 `"8443"` geçerli: `try` biter, sonra `else`, sonra `finally`.

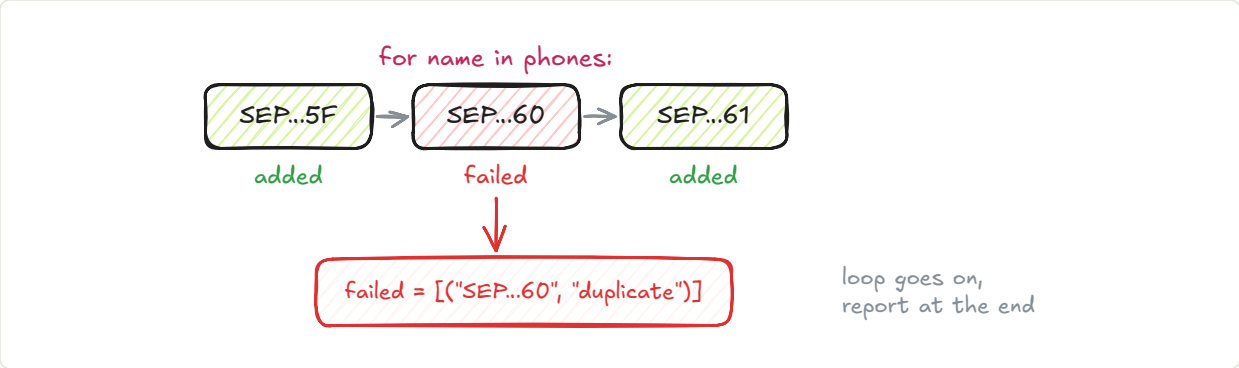
14 `"84x3"` geçersiz: `except` çalışır, `else` atlanır, `finally` yine çalışır.

Blok	Ne zaman çalışır
`try:`	her zaman ilk: riskli kod
`except Error:`	`try` içinde o tipte hata olursa
`except (A, B) as e:`	birkaç tipten biri; `e` hatanın kendisi
`else:`	`try` hatasız bittiyse
`finally:`	her zaman: temizlik

× SIK YAPILAN HATA
Çıplak bir `except:` ya da `except Exception: pass` **bütün hataları gizler**, yazım hataları dahil. Beklediğiniz **belirli** hatayı yakalayın ve neyin yanlış gittiğini her zaman yazdırın ya da loglayın.

Errors in bulk jobs · raise

Toplu işte tek bir bozuk satır, diğerlerini durdurmamalıdır. `try`'ı loop'un **içine** koyarsınız: hatayı kaydedin, `continue` ile bir sonraki elemana geçin, bütün hataları en sonda raporlayın.



Şekil 1.35. Her eleman kendi `try`'ında çalışır: hata olursa kaydedilir, loop devam eder.

```
1  failed = []
2  for name in phones:
3      try:
4          add_phone_axl(name)      # sizin AXL çağrınız
5      except Exception as e:
6          failed.append((name, str(e)))
7          continue
8      print("added", name)
9  print(f"{len(failed)} failed")
10
11 def build_device_name(mac):
12     mac = mac.replace(":", "").upper()
13     if len(mac) != 12:
14         raise ValueError(f"bad MAC: {mac!r}")
15     return "SEP" + mac
```

KODU SATIR SATIR OKUYALIM

1 Başarısız olanların listesi. Loop boyunca doldurulacak.

2–4 Her phone için `try` içinde AXL çağrısı yapılır.

5 Her türlü hata yakalanır. Burada geniş bir `except Exception` **kabul edilebilir**, çünkü hata saklanıyor ve en sonda gösteriliyor.

6 `(name, hata mesajı)` çifti `failed`'a eklenir.

7 `continue` ile bir sonraki phone'a geçilir.

8 Hata olmazsa buraya gelinir.

9 Loop bittikten sonra kaç tanesinin başarısız olduğu yazdırılır.

11–15 Bu function kendi hatasını **yükseltir** (`raise`): MAC 12 karakter değilse `ValueError` fırlatır.

`raise` bir exception'ı sizin yaratmanızı sağlar. Girdi yanlışsa, CUCM'e ulaşmadan **önce** durdurmak için kullanın: net bir `ValueError("bad MAC")`, daha sonra gelecek kafa karıştırıcı bir AXL fault'undan çok daha kolay çözülür.

> ▸ **COLLAB DÜNYASINDA**
> 500 phone import edersiniz: 499'u eklenir, 1 tekrar eden kayıt AXL hata mesajıyla birlikte `failed.csv` dosyasına yazılır ve ikinci bir çalıştırmayla düzeltilir.

Özetle

- Exception, Python'un "devam edemiyorum" demesidir. Traceback'i **aşağıdan** okuyun: son satır hatanın tipi ve mesajıdır.
- `try` / `except` hatayı yakalar; `else` hatasız bitince, `finally` her durumda çalışır.
- Toplu işte `try`'ı loop'un içine koyun: hatayı kaydedin, devam edin, sonda raporlayın.
- Bozuk girdiyi `raise` ile erkenden durdurun.

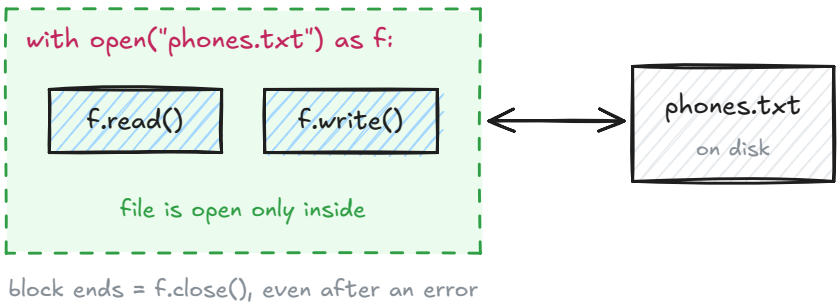
✎ Kendin dene

`ports = ["8443", "443", "abc", "5061"]` list'indeki değerleri `int()` ile çevirin. `try` / `except ValueError` kullanarak geçersizleri atlayın ve çevrilemeyenleri ayrı bir list'te toplayın.

1.8 Files

Working with files

Otomasyon script'lerinin çoğu bir dosyayla başlar ve bir dosyayla biter: bir CSV'den okunan kullanıcılar, bir rapor dosyasına yazılan sonuçlar. `open()`, script'inizi diskteki bir dosyaya bağlar ve bir **file object** döndürür.



Şekil 1.36. `with` bloğu bittiğinde dosya, bir exception olsa bile otomatik kapanır.

```
f = open("phones.txt", encoding="utf-8")
text = f.read()
f.close()                  # unutması kolay

with open("phones.txt", encoding="utf-8") as f:
    text = f.read()        # otomatik kapanır

from pathlib import Path
path = Path("data") / "phones.txt"
path.exists()              # True / False
```

KODU SATIR SATIR OKUYALIM

1 `open()` dosyayı açar. `encoding="utf-8"` metnin hangi kodlamayla okunacağını söyler; Türkçe karakterler için şarttır.

2 `read()` bütün içeriği tek bir string olarak okur.

3 `close()` dosyayı kapatır. Unutursanız dosya açık kalır.

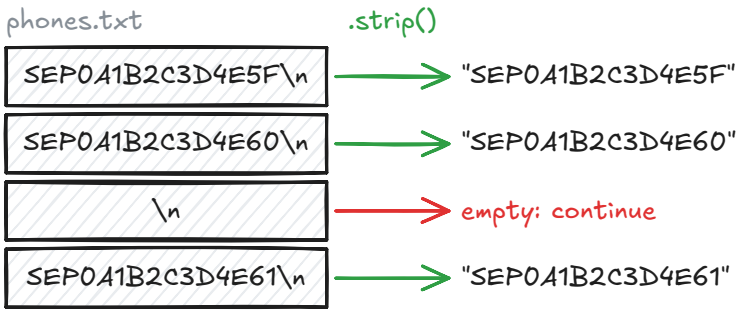
5-6 `with` kalıbı aynı işi **güvenle** yapar: blok bittiğinde, içeride bir exception olsa bile dosya kapanır. Her zaman bunu kullanın.

8-10 `pathlib.Path` yolları kurmak içindir. `/` operatörü yol parçalarını birleştirir ve macOS, Linux ve Windows'ta doğru çalışır.

Mod	Anlamı
`"r"`	okuma (varsayılan); dosya yoksa hata
`"w"`	yazma; mevcut içeriği **siler**
`"a"`	sona ekleme
`"x"`	yeni dosya yarat; zaten varsa hata
`"rb"` `"wb"`	ikili (binary): `str` yerine bytes

Reading and writing text files

Bir file object, **satır satır** dolaşılabilir. Her satırın sonunda hâlâ `"\n"` vardır; o yüzden önce `strip()` edin. Satır satır okumak, çok büyük dosyalarda bile belleği düşük tutar.



Şekil 1.37. Dosya satır satır okunur; her satırın sonunda bir satır sonu karakteri bulunur.

```
with open("phones.txt", encoding="utf-8") as f:
    for line in f:              # her turda bir satır
        name = line.strip()     # "\n"'i at
        if not name:
            continue            # boş satırları atla
        print(name)

with open("report.txt", "w", encoding="utf-8") as f:
    f.write("Phone report\n")
    f.write(f"{name}\n")        # "\n"'i kendiniz ekleyin
```

KODU SATIR SATIR OKUYALIM

1 Dosyayı okumak için açıyoruz.

2 Bir `for` loop'u dosyada satır satır dolaşır.

3 `strip()` baştaki ve sondaki boşlukları, satır sonunu atar.

4-5 Boş satırlar falsy'dir; `continue` onları atlar.

6 Temiz bir isim yazdırılır.

8 `"w"` modu yazmak için açar. **Dosya açıldığı anda içeriği silinir.**

9–10 `write()` bir string yazar. Satır sonunu **otomatik eklemez**: `\n`'i kendiniz koyun.

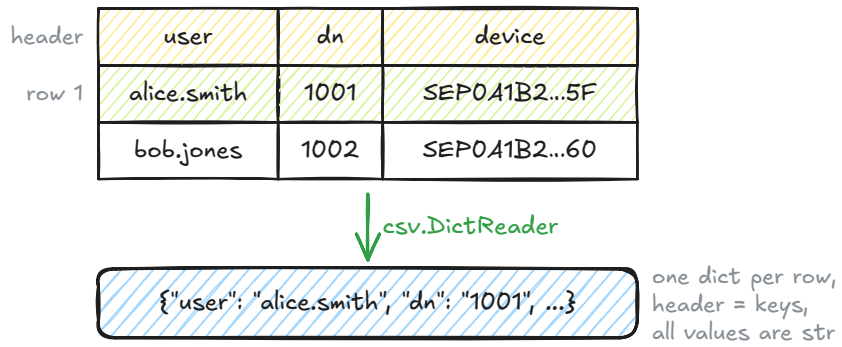
Metot	Ne yapar
`f.read()`	bütün dosya tek bir `str` olarak
`for line in f`	her seferinde bir satır
`f.readlines()`	bütün satırların `list`'i
`f.write(text)`	bir `str` yazar; satır sonu eklemez
`Path(p).read_text()`	açar, okur ve kapatır; tek çağrıda

× SIK YAPILAN HATA
`"w"` dosyayı açtığınız anda boşaltır. Raporu girdi dosyanızın üzerine yazarsanız onu yok edersiniz. Yeni bir dosya adı kullanın ya da `"a"` ile sona ekleyin.

1.8.1 CSV

Reading CSV files

CSV, toplu işlerin en yaygın girdisidir: her Excel sayfası CSV olarak kaydedilebilir. `csv.DictReader` her satırı bir `dict`'e çevirir ve başlık satırını key olarak kullanır; böylece sütunları **isimle** okursunuz.



Şekil 1.38. `DictReader` başlık satırını key, her satırı bir dict yapar.

```
1 import csv
2
3 with open("users.csv", newline="",
4           encoding="utf-8") as f:
5     for row in csv.DictReader(f):
6         print(row["user"], row["dn"])
7 # alice.smith 1001
8 # bob.jones 1002
```

KODU SATIR SATIR OKUYALIM

1 `csv` standart kütüphane modülüdür.

3–4 Dosyayı açıyoruz. `newline=""`, `csv` modülünün satır sonlarını kendisinin yönetmesi için gereklidir.

5 `DictReader(f)` dosyanın başlık satırını okur ve her satır için bir `dict` döndürür.

6 Sütunlara başlıklarıyla ulaşırız: `row["user"]`, `row["dn"]`.

Her değer bir `str`'dir. Sayıları kendiniz çevirin. `users.csv` dosyası şöyle görünür:

ÇIKTI

```
user,dn
alice.smith,1001
bob.jones,1002
```

! DIKKAT

Türkçe ya da Almanca ayarlı Excel, dosyayı ayraç olarak `;` ve gizli bir BOM (byte order mark) ile kaydeder. Bu dosyaları `delimiter=";"` ve `encoding="utf-8-sig"` ile okuyun, yoksa ilk key `"user"` olur ve `row["user"]` `KeyError` verir.

Writing CSV files

`csv.DictWriter` bir dict list'ini satırlar halinde yazar. `fieldnames` sütun sırasını belirler, `writeheader()` ilk satırı yazar.

```
1  import csv
2
3  results = [
4      {"device": "SEP0A1B2C3D4E5F", "status": "added"},
5      {"device": "SEP0A1B2C3D4E60", "status": "failed"},
6  ]
7
8  with open("result.csv", "w", newline="", encoding="utf-8") as f:
9      writer = csv.DictWriter(f, fieldnames=["device", "status"])
10     writer.writeheader()
11     writer.writerows(results)
```

ÇIKTI

```
device,status
SEP0A1B2C3D4E5F,added
SEP0A1B2C3D4E60,failed
```

KODU SATIR SATIR OKUYALIM

3–6 Sonuçlar dict'lerden oluşan bir list: her dict bir satır.

8 Dosya yazma modunda (`"w"`) açılır.

9 `DictWriter`, hangi key'lerin hangi sütun olacağını `fieldnames` ile öğrenir.

10 `writeheader()` başlık satırını yazar.

11 `writerows()` list'in bütün dict'lerini satır satır yazar.

Araç	Neyle çalışır
`csv.DictReader`	satırlar `dict`; key'ler başlıktan
`csv.DictWriter`	`dict` satırlar yazar
`csv.reader`	satırlar `list`; konuma göre
`csv.writer`	`list` satırlar yazar

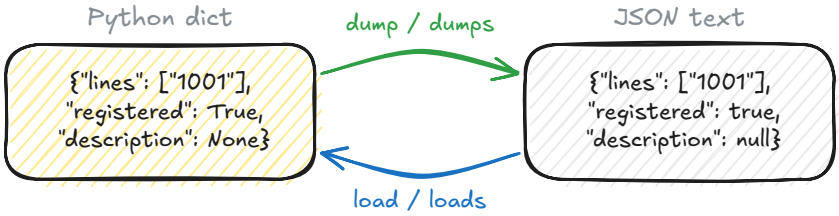
▸ COLLAB DÜNYASINDA

`users.csv` dosyasını okuyun, her satırı provision edin, device ve durumla birlikte `result.csv` yazın. Aynı dosya müşteri için Excel'de açılır.

1.8.2 JSON

JSON · load and dump

JSON, REST API'lerinin metin biçimidir. Yapısı Python'la neredeyse birebir örtüşür; bu yüzden `json` modülü iki yönde de çevirir. `dumps` / `loads` içindeki `s` **string** demektir; `s` olmayanlar dosyayla çalışır.



Şekil 1.39. JSON ↔ Python: nesne `dict`, dizi `list`; `dumps` / `loads` string, `dump` / `load` dosya ile çalışır.

```
1  import json
2
3  with open("phone.json", "w", encoding="utf-8") as f:
4      json.dump(phone, f, indent=2)   # dict → dosya
5
6  with open("phone.json", encoding="utf-8") as f:
7      data = json.load(f)             # dosya → dict
8
9  text = json.dumps(phone)            # dict → str
10 json.loads(text)["lines"]           # str → dict
```

KODU SATIR SATIR OKUYALIM

3–4 `json.dump()` bir Python objesini dosyaya yazar. `indent=2` çıktıyı okunur kılar.

6–7 `json.load()` bir dosyayı okuyup Python objesine çevirir.

9 `dumps` objeyi dosya yerine **string** olarak verir.

10 `loads` bir string'i objeye çevirir; sonra normal bir dict gibi kullanabilirsiniz.

Python	JSON
`dict`	nesne `{}`
`list` (ve `tuple`)	dizi `[]`
`str`	string `""`
`int` `float`	sayı
`True` `False` `None`	`true` `false` `null`

▸ COLLAB DÜNYASINDA

Webex API'leri JSON gönderir ve alır. Bir değişiklikten önce mevcut konfigürasyonu `json.dump(..., indent=2)` ile kaydetmek, okunabilir bir yedek verir.

Özetle

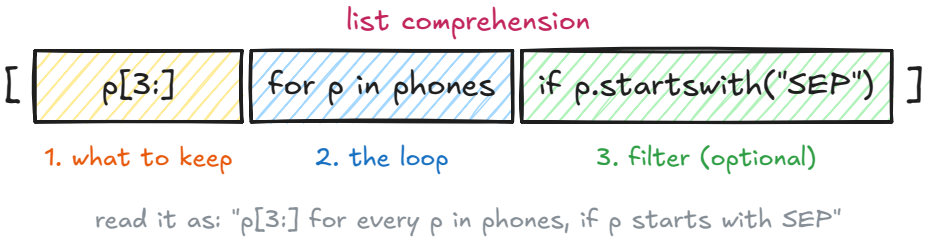
- Dosyaları her zaman `with open(...)` ile açın ve `encoding="utf-8"` yazın.
- CSV için `DictReader` ve `DictWriter` : sütunlara isimle ulaşırsınız. Türkçe Excel'in `;` ve BOM tuzağına dikkat.
- JSON, `dict` ve `list` 'lere doğrudan çevrilir: `load` / `dump` dosya, `loads` / `dumps` string için.

> ✎ **Kendin dene**
> Küçük bir `users.csv` dosyası oluşturun (`user,dn` sütunları). `DictReader` ile okuyup her kullanıcının DN'ini `int`'e çevirerek bir list'e koyun, sonra bu list'i `json.dump` ile `users.json` dosyasına yazın.

1.9 Comprehensions

List comprehension

Bir list'ten başka bir list üretmek, programlamanın en sık yapılan işlerinden biridir: "phone listesinden yalnızca masa telefonlarını al", "her cihaz adından `SEP` önekini at". Bunu şimdiye kadar öğrendiklerinizle yapabilirsiniz: boş bir list, bir loop, bir `if` ve bir `append`. **List comprehension** aynı işi tek bir ifadeyle yapar ve neredeyse bir cümle gibi okunur.



Şekil 1.40. Comprehension üç parçadan oluşur: ne tutulacak, loop ve isteğe bağlı bir filtre.

```
1  phones = ["SEP0A1B2C3D4E5F", "CSFALICE",
2            "SEP0A1B2C3D4E60"]
3
4  desk = []                          # loop sürümü
5  for p in phones:
6      if p.startswith("SEP"):
7          desk.append(p[3:])
8
9  desk = [p[3:] for p in phones if p.startswith("SEP")]
10 # ['0A1B2C3D4E5F', '0A1B2C3D4E60']
11
12 dns = [str(n) for n in range(1001, 1004)]
```

KODU SATIR SATIR OKUYALIM

1–2 İki masa telefonu ve bir Jabber client'ı içeren bir list.

4–7 **Loop sürümü**: boş bir list, bir `for`, bir `if`, bir `append`. Dört satır, tek bir amaç: `SEP` ile başlayanların MAC'lerini toplamak.

9 **Comprehension sürümü**: aynı iş tek satırda. Köşeli parantez "yeni bir list kur" der.

10 Orijinal `phones` değişmez; sonuç yeni bir list'tir.

12 Filtre olmadan da çalışır: `range`'den gelen her sayıyı `str`'e çevirip `dns` list'ini kurar.

Bir comprehension en fazla üç parçadan oluşur ve **her zaman** şu sırayla yazılır:

1. **Ne tutulacak:** elemanın kendisi ya da dönüştürülmüş hali (`p[3:]`).
2. **Loop:** `for eleman in koleksiyon`.
3. **Filtre** (isteğe bağlı): `if koşul`.

▸ **COLLAB DÜNYASINDA**
Tek satır, bir phone listesini bir Webex device onboarding sayfasının ihtiyaç duyduğu MAC listesine çevirir, ya da bir site'ın yalnızca masa telefonlarını tutar.

Dict and set comprehensions

Aynı fikir her koleksiyon için çalışır. Hangisini elde edeceğinizi **parantez türü** belirler.

```
1  rows = [{"user": "alice.smith", "dn": "1001"},
2          {"user": "bob.jones",   "dn": "1002"},
3          {"user": "carol.white", "dn": "1003"}]
4
5  dn_of = {r["user"]: r["dn"] for r in rows}    # dict
6  dn_of["bob.jones"]                            # '1002'
7
8  dns = ["1001", "1002", "2000"]
9  {dn[:2] for dn in dns}                        # set
10 # {'10', '20'}  kullanımdaki DN aralıkları
11
12 statuses = ["Registered", "UnRegistered", "Registered"]
13 ["up" if s == "Registered" else "down" for s in statuses]
14 # ['up', 'down', 'up']
15
16 sum(1 for s in statuses if s != "Registered")    # 1 kapalı
17 all(p.startswith("SEP") for p in desk)           # True
```

KODU SATIR SATIR OKUYALIM

1–3 CSV'den okunmuş üç satır, her biri bir dict.

5 **Dict comprehension**: süslü parantez ve `anahtar: değer`. Her satır için kullanıcı → DN ilişkisi kurulur.

6 Sonuç, bir arama tablosudur: kullanıcıyı verin, DN'i alın. Loop yok.

9 **Set comprehension**: süslü parantez ama `anahtar: değer` yok. Her DN'in ilk iki hanesini toplar, tekrarlar otomatik gider.

13 Bir comprehension'ın "tutulacak" kısmında `if ... else ...` kullanabilirsiniz: durumları `"up"` ya da `"down"` olarak çevirir.

16 Köşeli parantez **olmadan** bir **generator** kullanılır: `sum()` içinde her kapalı phone için `1` üretir ve toplar.

17 `all()` bütün elemanlar için koşul doğruysa `True` verir.

Biçim	Sonuç
`[x for …]`	`list`
`{k: v for …}`	`dict`
`{x for …}`	`set`; tekrarlar silinir
`(x for …)`	generator: elemanları tek tek üretir

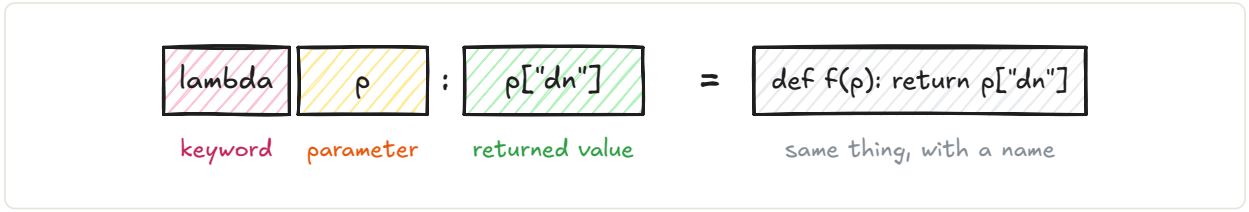
`sum()`, `any()` ya da `all()` içindeki bir generator için ek parantez gerekmez.

▸ **COLLAB DÜNYASINDA**
`{r["user"]: r["dn"] for r in rows}` bir CSV'yi arama tablosuna çevirir: bir loop yazmadan herhangi bir kullanıcının DN'ini anında bulursunuz.

lambda and key functions

`lambda`, ismi olmayan küçük bir function'dır: bir parameter listesi, tek bir ifade ve sonuç otomatik döner. Bir function'a yalnızca **bir an** için ihtiyaç duyduğunuz yerlerde kullanılır.

En yaygın işi `key=` argument'ıdır. `sorted`, `min` ve `max`, elemanların kendisini değil, key function'ın **her eleman için ürettiği değeri** karşılaştırır.



Şekil 1.41. `lambda` parametre alır ve tek bir ifade döndürür.

```
phones = [{"name": "SEP0A1B2C3D4E60", "dn": "1002"},
          {"name": "SEP0A1B2C3D4E5F", "dn": "1001"}]
sorted(phones, key=lambda p: p["dn"])  # DN'e göre

cdrs = [{"caller": "1001", "duration": 125},
        {"caller": "1002", "duration": 410}]
max(cdrs, key=lambda c: c["duration"])
# {'caller': '1002', 'duration': 410}
```

KODU SATIR SATIR OKUYALIM

1–2 İki phone dict'i, DN sırası ters.

3 `lambda p: p["dn"]` şöyle okunur: "`p` verilince `p["dn"]`'i döndür". `sorted` her phone için bu değeri hesaplar ve ona göre sıralar: önce `"1001"`, sonra `"1002"`.

5–6 İki CDR kaydı.

7–8 `max` en uzun süreli kaydı bulur: `key` süreyi seçer, sonuç yalnızca süre değil, **kaydın tamamıdır**.

> **× SIK YAPILAN HATA**
> Bir comprehension ya da lambda tek bir okunur satıra sığmıyorsa, normal bir loop ya da `def` yazın. Kısa ve anlaşılır, akıllıca ve kafa karıştırıcı olandan iyidir.

> **▸ COLLAB DÜNYASINDA**
> Rapor için phone'ları DN'e göre sıralayın ya da bir CDR export'unda en uzun call'ı `max(cdrs, key=…)` ile bulun.

Özetle

- Comprehension, "boş list + loop + if + append" kalıbını tek ifadeye sıkıştırır: `[ne for eleman in koleksiyon if filtre]`.
- Parantez türü sonucu belirler: `[]` list, `{k: v}` dict, `{x}` set, `()` generator.
- `lambda`, `key=` ile `sorted`, `min`, `max` için kısa bir kural verir.
- Okunmaz hale gelirse, normal loop'a dönün.

✎ Kendin dene

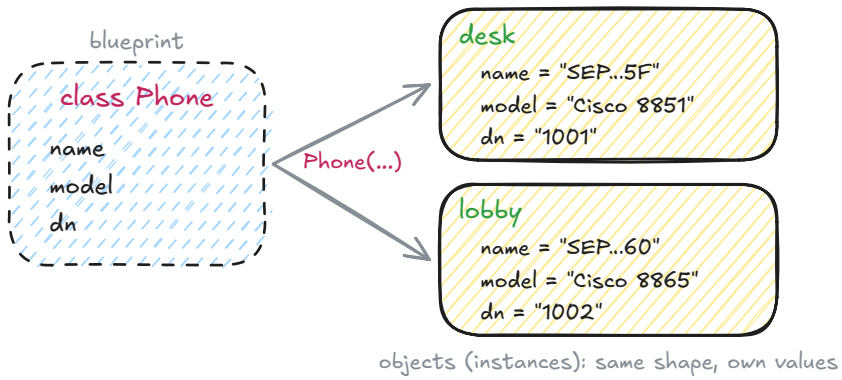
`phones = ["SEP0A1B2C3D4E5F", "CSFALICE", "SEP0A1B2C3D4E60", "CSFAGENT01"]` list'inden iki comprehension yazın: biri yalnızca `CSF` ile başlayanları (Jabber), diğeri her device adının uzunluğunu içeren bir dict (`{ad: uzunluk}`).

1.10 Classes

Classes and objects

Şimdiye kadar bir phone'u bir `dict` ile temsil ettik. Bu iş görür, ama dict'in kuralları yoktur: `"nmae"` diye yanlış yazılmış bir key'i kimse engellemez, "bu phone video destekliyor mu?" sorusunun cevabı ise başka bir yerde bir function'da yaşar. **Class** (sınıf), bir şeyin **verisini** ve ona ait **davranışı** tek bir yerde toplamanın yoludur.

> **≈ BENZETME**
> CUCM'deki bir **phone template**, ya da bir **device pool**'u düşünün: tipi tanımlar (hangi alanlar var, hangi varsayılanlar) ve ondan istediğiniz kadar phone türetirsiniz. Class bir **şablondur**; ondan türetilen her bir phone bir **object** (nesne, ya da **instance**) olur. Her objenin kendi değerleri vardır.



Şekil 1.42. Class bir şablondur; ondan türetilen her object kendi değerlerini taşır.

```
1  class Phone:
2      def __init__(self, name, model, dn):
3          self.name = name        # attributes
4          self.model = model
5          self.dn = dn
6
7  desk = Phone("SEP0A1B2C3D4E5F", "Cisco 8851", "1001")
8  lobby = Phone("SEP0A1B2C3D4E60", "Cisco 8865", "1002")
9
10 desk.dn          # '1001'
11 lobby.model      # 'Cisco 8865'
```

KODU SATIR SATIR OKUYALIM

1 `class Phone:` yeni bir tip tanımlar. Class isimleri `CamelCase` ile yazılır.

2 `__init__` özel bir method'dur: bir object **yaratıldığında** çalışır. Parameter'lar, yaratırken verilen değerlerdir.

3-5 `self.name = name` değeri objenin üzerinde saklar. Bu satırlar objenin **attribute**'larını (özelliklerini) kurar.

7 `Phone(...)` yeni bir object yaratır ve `__init__`'i çağırır. `desk` bu object'e bağlanır.

8 İkinci bir `Phone`, tamamen bağımsız değerlerle.

10-11 Attribute'lara nokta ile ulaşılır.

Parça	Anlamı
`class Phone:`	yeni bir tip tanımlar (CamelCase isim)
`__init__`	object yaratılınca çalışır
`self.name = …`	object'in üzerinde bir attribute saklar
`Phone(...)`	yeni bir object yaratır
`desk.dn`	nokta ile bir attribute okur

▸ **COLLAB DÜNYASINDA**
Bir `dict` bir phone'un verisini tutar. Bir class ise veriyi **ve ona ait kuralları** tutar: böylece her script bir phone'a aynı şekilde davranır.

Methods and self

Class'ın içinde tanımlanan bir function'a **method** denir. İlk parameter'ı her zaman `self`'tir: method'un çağrıldığı object'in kendisi. Python onu sizin yerinize doldurur: `desk.mac()` perde arkasında `Phone.mac(desk)` olarak çalışır.



Şekil 1.43. `desk.mac()` çağrısında `desk` otomatik olarak `self` olur.

```
1  class Phone:
2      VIDEO_MODELS = ("Cisco 8845", "Cisco 8865")  # ortak
3
4      def __init__(self, name, model, dn):
5          self.name, self.model, self.dn = name, model, dn
6          self.status = "Unknown"
7
8      def mac(self):
9          return self.name[3:]
10
11     def is_video(self):
12         return self.model in self.VIDEO_MODELS
13
14 desk = Phone("SEP0A1B2C3D4E5F", "Cisco 8851", "1001")
15 desk.mac()                    # '0A1B2C3D4E5F'
16 desk.is_video()               # False
17 desk.status = "Registered"    # attribute'lar değişebilir
```

KODU SATIR SATIR OKUYALIM

2 `VIDEO_MODELS` bir **class attribute**'tur: class'ın içinde, `self`'siz tanımlanır. Bütün object'ler tarafından ortak kullanılır.

4–6 `__init__`: üç değer atanır ve bir başlangıç durumu (`status`) eklenir.

8–9 İlk method: `self.name` objenin kendi device adıdır; `[3:]` ile `SEP` öneki atılır.

11–12 İkinci method: objenin modeli, ortak `VIDEO_MODELS` tuple'ı içinde mi?

14 Object yaratılır.

15 `desk.mac()`: method'a `desk` otomatik olarak `self` olarak geçer. Sonuç `'0A1B2C3D4E5F'`.

16 8851 video destekli listede değil: `False`.

17 Attribute'ları dışarıdan da değiştirebilirsiniz.

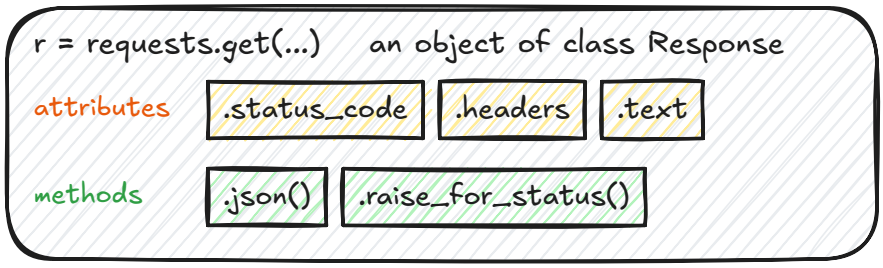
`self` üzerinden bir method, objenin kendi attribute'larını okur ve değiştirir. `VIDEO_MODELS` gibi bir **class attribute** ise bütün object'ler tarafından ortak kullanılır.

Çift alt çizgili isimler özeldir: `__init__` objeyi kurar, `__repr__` ise `print(desk)` çağrısının nasıl görüneceğine karar verir.

> **× SIK YAPILAN HATA**
> `def mac():` yazıp `self`'i unutursanız `TypeError` alırsınız: "takes 0 positional arguments but 1 was given". Python objeyi otomatik olarak geçirir; method'un onu alacak bir parameter'ı olmalı.

Classes in the libraries you will use

Çoğu zaman class yazmazsınız, **kullanırsınız**. Her kütüphane size object'ler verir; siz onların attribute'larını okur ve method'larını nokta ile çağırırsınız.



Şekil 1.44. `requests.get()` bir `Response` object'i döndürür: `.status_code`, `.json()` gibi attribute ve method'lar taşır.

```
1  class AxlClient:
2      def __init__(self, host, user, password):
3          self.url = f"https://{host}:8443/axl/"
4          self.auth = (user, password)
5
6      def get_phone(self, name):
7          ...        # AXL isteğini burada gönder
8
9  axl = AxlClient("cucm-pub.clauto.lab",
10                 "axl_svc", password)
11 axl.get_phone("SEP0A1B2C3D4E5F")
```

KODU SATIR SATIR OKUYALIM

1–4 Bir AXL istemcisi: host, kullanıcı ve parolayı **bir kere** saklar.

6–7 `get_phone` method'u `self.url` ve `self.auth`'u kullanarak isteği gönderirdi. `...` gövdenin burada bilerek boş bırakıldığını söyler.

9–10 İstemci yaratılırken bağlantı bilgileri verilir.

11 Sonra her çağrı yalnızca phone adını ister: host ve parolayı tekrar yazmak gerekmez.

Object	Nereden
`Response`	`requests.get()`: `.status_code`, `.json()`
`Session`	`requests.Session()`: oturumu ve bağlantıyı yeniden kullanır
`Client`	`zeep.Client(wsdl)`: `.service.getPhone()`

Aynı ayarları paylaşan birden çok function olduğunda, host ve kimlik bilgisi gibi, küçük bir class yazmak karşılığını verir.

▸ **COLLAB DÜNYASINDA**
Bir `AxlClient`, CUCM host'unu ve girişi bir kere saklar. Sonra her script yalnızca `axl.get_phone(...)` ya da `axl.add_line(...)` çağırır.

Özetle

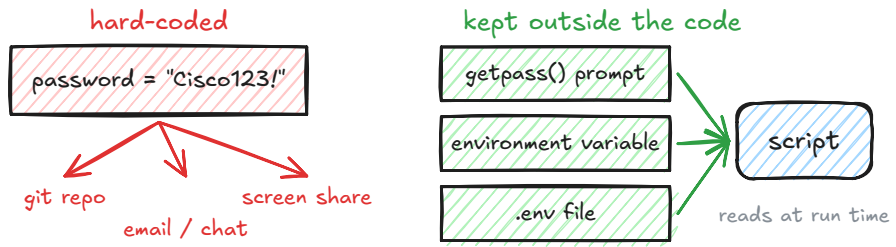
- Class, verisi (attribute) ve davranışı (method) olan yeni bir tiptir; ondan istediğiniz kadar object türetirsiniz.
- `__init__` object'i kurar; method'ların ilk parameter'ı `self`'tir ve Python onu otomatik doldurur.
- Çoğu zaman class'ları yazmazsınız, kütüphanelerden gelen object'leri kullanırsınız.

✎ **Kendin dene**
`Phone` class'ına `describe()` adında bir method ekleyin: `"SEP0A1B2C3D4E5F (Cisco 8851) - 1001"` biçiminde bir metin döndürsün. Sonra iki farklı `Phone` objesi yaratıp ikisinde de çağırın.

1.11 Credentials

Keep secrets out of your code

Kodun içine yazılan bir parola, script ile birlikte yolculuk eder: git'e, bir iş arkadaşına e-postayla gönderdiğiniz dosyaya, paylaşılan ekrana. Bir kere repo'ya giren parola, silseniz bile git geçmişinde kalır. Altın kural şudur: **gizli bilgileri kodun dışında tutun** ve script çalışırken okuyun.



Şekil 1.45. Parola koda değil, script'in dışındaki bir kaynağa yazılır ve çalışırken okunur.

```
1  password = "Cisco123!"          # asla böyle yapmayın
2
3  from getpass import getpass
4  password = getpass("AXL password: ")   # ekranda görünmez
5
6  import os
7  user = os.environ["AXL_USER"]   # yoksa KeyError
8  password = os.environ.get("AXL_PASSWORD")  # ya da None
```

```
export AXL_USER=axl_svc         # terminalde
```

KODU SATIR SATIR OKUYALIM

1 **Kötü örnek**: parola doğrudan kodda. Dosyayı gören herkes parolayı görür.

3-4 `getpass()` script her çalıştığında parolayı sorar ve yazdıklarınızı ekranda göstermez. Elle yapılan çalıştırmalar için idealdir.

6-7 `os.environ`, ortam değişkenlerine (environment variables) erişim sağlar. Bir `dict` gibi davranır: `[...]` olmayan bir isimde hata verir.

8 `.get()` ise olmayan bir isimde `None` döndürür.

10 Ortam değişkeni terminalde `export` ile tanımlanır.

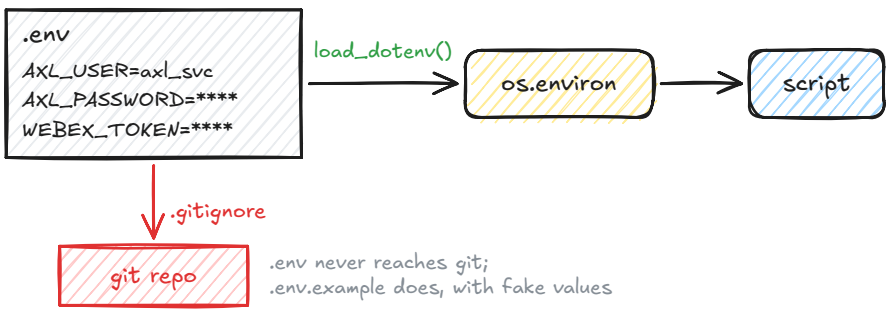
Kaynak	Ne için iyi
`getpass()`	elle çalıştırma: sorar, hiçbir şey göstermez
`os.environ`	kabukta ya da bir zamanlayıcıda tanımlanan değerler
`.env` dosyası	laptop'unuzdaki proje başına ayarlar

▸ **COLLAB DÜNYASINDA**

Kendi admin hesabınızı değil, yalnızca **Standard AXL API Access** ve **Standard CCM Admin Users** rollerine sahip, ayrılmış bir AXL application user (`axl_svc`) kullanın. Sızarsa zarar sınırlı kalır ve tek bir hesabı iptal edersiniz.

.env files and a safe setup

Ortam değişkenlerini her seferinde elle tanımlamak yorucudur. `python-dotenv` paketi bir `.env` dosyasını okur ve her satırı `os.environ`'a koyar. Script'in kodu laptop'unuzda, bir jump host'ta ve bir zamanlayıcıda aynı kalır; yalnızca `.env` dosyası değişir.



Şekil 1.46. `.env` dosyası repo'ya girmez; `load_dotenv()` onu ortama yükler.

```
# .env  (.gitignore'da)
CUCM_HOST=cucm-pub.clauto.lab
AXL_USER=axl_svc
AXL_PASSWORD=change-me
```

```
1 # pip install python-dotenv
2 import os
3 from dotenv import load_dotenv
4
5 load_dotenv()                    # .env → os.environ
6 missing = [k for k in ("AXL_USER", "AXL_PASSWORD")
7            if k not in os.environ]
8 if missing:
9     raise SystemExit(f"missing settings: {missing}")
```

KODU SATIR SATIR OKUYALIM

2 `python-dotenv` standart kütüphanede yoktur; önce `pip install python-dotenv` gerekir.

5 `load_dotenv()` `.env` dosyasını okur ve içindekileri `os.environ`'a yükler.

6–7 Bir list comprehension: gerekli ayarlardan **ortamda olmayanları** toplar.

8–9 Eksik varsa, script'i erkenden ve **anlaşılır bir mesajla** durdurur (`SystemExit`).

Gerekli bütün ayarları **en başta** kontrol etmek, 300 phone'luk bir işin ortasında anlaşılmaz bir hatayla düşmek yerine tek bir net mesaj verir.

Yap	Neden
`.env`'i `.gitignore`'a ekle	gizli bilgiler repo'ya hiç girmez
`.env.example` dosyasını commit et	başkaları hangi isimlerin gerektiğini görür
araç başına bir AXL kullanıcısı	denetlemesi ve iptali kolay
bir parolayı asla `print` etme	loglar ve ekranlar paylaşılır

Özetle

- Parolayı koda yazmayın. `getpass()`, `os.environ` ya da bir `.env` dosyası kullanın.
- `.env` her zaman `.gitignore`'da olmalı; repo'ya yalnızca isimleri gösteren `.env.example` girer.
- Gerekli ayarları script'in başında kontrol edin ve eksik varsa net bir mesajla durun.
- İşe özel, yetkisi sınırlı bir AXL kullanıcısı (`axl_svc`) kullanın.

✎ Kendin dene

Bir `.env` dosyası oluşturup `CUCM_HOST` ve `AXL_USER` tanımlayın, `.gitignore`'a ekleyin. Script'te `load_dotenv()` ile okuyup değerleri yazdırın. Sonra `AXL_USER` satırını silip script'in `SystemExit` ile nasıl durduğunu gözlemleyin.

2 APIs and Libraries

2.1 HTTP Basics
2.2 JSON and XML
2.3 requests
2.4 zeep and AXL
2.5 RisPort70
2.6 Netmiko
2.7 CMS REST API
2.8 Remote Phone Control
2.9 Wrap-up

Her Cisco collaboration ürünü bir API üzerinden yönetilebilir: CUCM AXL ve UDS ile, Unity Connection CUPI ile, Webex kendi REST API'siyle. Bu bölümde hepsinin arkasındaki web temellerini, konuştukları veri formatlarını ve işin ağır kısmını yapan iki Python kütüphanesini öğreniyoruz: REST için `requests`, SOAP için `zeep`.

BU BÖLÜMDE ÖĞRENECEKLERİNİZ

- Bir HTTP isteğinin ve cevabının nasıl okunacağını anlamak (SIP'e çok benzer)
- JSON ve XML verisini Python'da ayrıştırmak
- `requests` ile REST API çağırmak: kimlik doğrulama, hata yönetimi, paging
- `zeep` ile AXL üzerinden CUCM'i okumak ve yazmak
- RISPort70 ile cihaz register durumunu sorgulamak
- Netmiko ile CLI'dan komut çalıştırmak, CMS REST API'sini ve phone'ları uzaktan kontrol etmeyi görmek

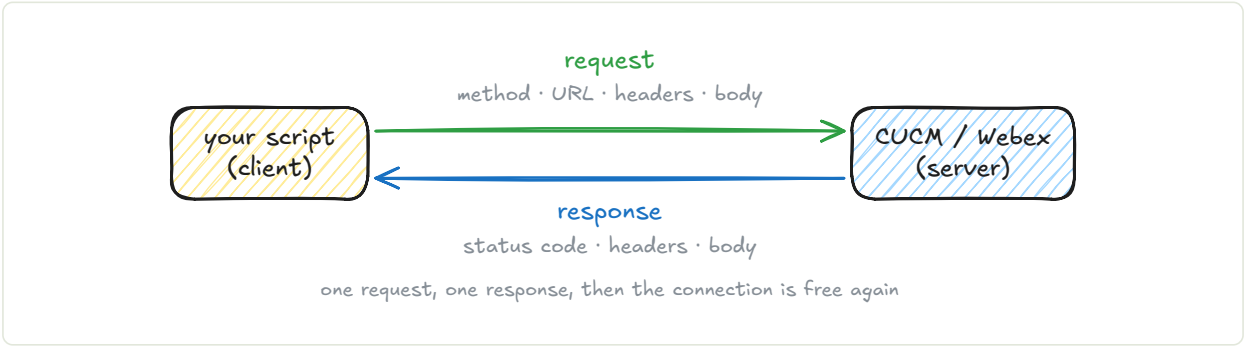
2.1 HTTP Basics

What is an API?

Bir CUCM yöneticisi bir phone eklemek için tarayıcıda **Cisco Unified CM Administration** arayüzünü açar, formu doldurur ve Save'e basar. Arkada tarayıcı, CUCM'e bir istek gönderir. Script'ler de aynı isteği, arayüze hiç uğramadan doğrudan gönderebilir. Programların kullanması için açılmış bu kapıya **API** (application programming interface) denir.

Bugünkü API'lerin neredeyse hepsi **HTTP(S)** üzerinden çalışır. Mantık çok basittir: istemci (client) bir **request** (istek) gönderir, sunucu bir **response** (cevap) ile yanıt verir.

> **≈ BENZETME**
> SIP'i biliyorsunuz: bir `INVITE` gönderirsiniz, bir `200 OK` ya da `404 Not Found` dönülür. İkisi de düz metin, ikisinde de bir ilk satır, bir dizi header ve isteğe bağlı bir body vardır. **HTTP aynı fikirdir**, sadece çağrı kurmak yerine veri okur ve yazar. SIP'te ezberlediğiniz şeylerin çoğu burada da işe yarar.



Şekil 2.1. Client bir request gönderir, server bir response ile cevap verir. İkisi de düz metindir.

```
GET /v1/people?email=alice.smith@clauto.lab HTTP/1.1
Host: webexapis.com
Authorization: Bearer <token>
Accept: application/json
```

```
HTTP/1.1 200 OK
Content-Type: application/json

{"items": [{"displayName": "Alice Smith", ...}]}
```

KODU SATIR SATIR OKUYALIM

İstek satırı — `GET` ne yapılacağını, `/v1/people?...` neyle yapılacağını, `HTTP/1.1` protokol sürümünü söyler. Bu satır, SIP'teki `INVITE sip:... SIP/2.0` satırının karşılığıdır.

Header'lar — `Host` hangi sunucuya gittiğini, `Authorization` kim olduğunuzu, `Accept` hangi formatta cevap istediğinizi söyler.

Durum satırı — `200 OK`: işlem başarılı. Sayı bir **status code**'dur.

Body — Boş satırdan sonra gelen asıl veri. Burada bir JSON.

Hem request hem response aynı yapıdadır: **ilk satır, header'lar ve isteğe bağlı bir body**. Bir API'yi öğrenmek, aslında hangi isteği gönderince hangi cevabın geldiğini öğrenmektir.

API	Stil
CUCM AXL, RISPort	SOAP: HTTPS üzerinden XML
CUCM UDS, Unity CUPI	REST: XML ya da JSON
Webex	REST: JSON

Methods and URLs

Her HTTP isteğinin ilk satırı iki şeyden oluşur: **method** (ne yapılacağı) ve **URL** (neyle yapılacağı). Method'lar fiildir, URL ise nesnedir.



Şekil 2.2. Bir URL: protokol, host, path ve `?`'den sonra query string.

```
GET    /v1/people?email=…    kullanıcı bul
POST   /v1/messages          mesaj gönder
PUT    /v1/people/{id}       kullanıcı kaydını değiştir
DELETE /v1/people/{id}       kullanıcıyı sil
```

Method	Anlamı
`GET`	veri oku, hiçbir şeyi değiştirmez
`POST`	yeni bir şey yarat ya da bir işlem çalıştır
`PUT`	var olan bir kaydı tamamen değiştir
`PATCH`	bir kaydın bir kısmını değiştir
`DELETE`	bir kaydı sil

Query string, `?`'den sonra gelen kısımdır ve sonucu filtreler ya da sayfalar: `?email=…`, `?max=100`. Birden fazla parametre `&` ile ayrılır.

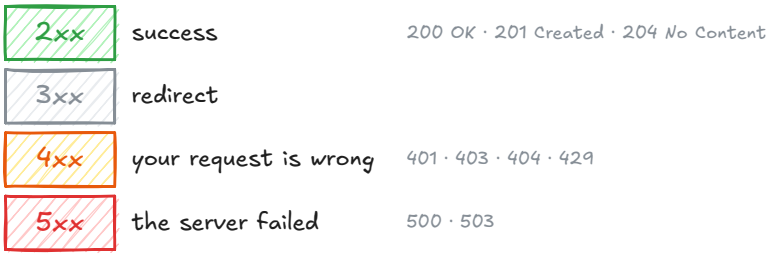
> **≈ BENZETME**
> `GET`'i CUCM'de bir phone'a **bakmak**, `POST`'u **eklemek**, `PUT`'u üzerine yazarak **güncellemek**, `DELETE`'i **silmek** olarak düşünün. AXL'deki `getPhone`, `addPhone`, `updatePhone`, `removePhone` operasyonları tam bu dört işin SOAP'taki karşılıklarıdır.

> **▸ COLLAB DÜNYASINDA**
> AXL gibi SOAP API'leri her şey için `POST` kullanır, okuma için bile. Operasyon (`getPhone`, `addLine`) XML body'nin içine yazılır. REST API'lerinde ise yapılacak işi method ve URL söyler.

Status codes, headers and auth

Cevabın ilk satırındaki **status code**, isteğin işe yarayıp yaramadığını hemen söyler. **Header**'lar iki taraf için ek bilgi taşır; en önemlileri format ve kimliktir.

> **≈ BENZETME**
> Status code'ları SIP'ten tanıyorsunuz: `2xx` başarı, `4xx` istemci hatası, `5xx` sunucu hatası. `401 Unauthorized` her iki dünyada da "kim olduğunu kanıtla" demektir.



Şekil 2.3. Status code sınıfları: 2xx başarı, 4xx istemci hatası, 5xx sunucu hatası.

```
Content-Type: application/json    # ben ne gönderiyorum
Accept: application/json          # ne geri istiyorum
Authorization: Basic YXhsX3N2YzpwYXNzd29yZA==
Authorization: Bearer <access token>
```

KODU SATIR SATIR OKUYALIM

`Content-Type` Body'de ne gönderdiğimi söyler.

`Accept` Cevabı hangi formatta istediğimi söyler.

`Basic` `user:password` metninin base64 ile kodlanmış hali. Her istekte gönderilir. CUCM AXL ve UDS bunu kullanır.

`Bearer` Bir **access token**. Webex gibi API'ler önce bir token verir, sonraki her isteğe bu token eklenir.

Kod	Sizin için genellikle anlamı
`401`	kullanıcı / parola yanlış ya da token süresi dolmuş
`403`	giriş doğru ama gerekli rol eksik
`404`	URL yanlış ya da kayıt yok
`429`	çok fazla istek: `Retry-After` saniye bekleyin
`500` `503`	sunucu hatası ya da servis kapalı

▸ COLLAB DÜNYASINDA

AXL **Basic** kimlik doğrulama (`user:password` 'ün base64'ü) kullanır. Webex ise **Bearer** token kullanır. Lab ortamındaki CUCM'lerin sertifikaları kendinden imzalıdır (self-signed), bu yüzden sertifika kontrolü ek dikkat ister; bunu `requests` bölümünde göreceğiz.

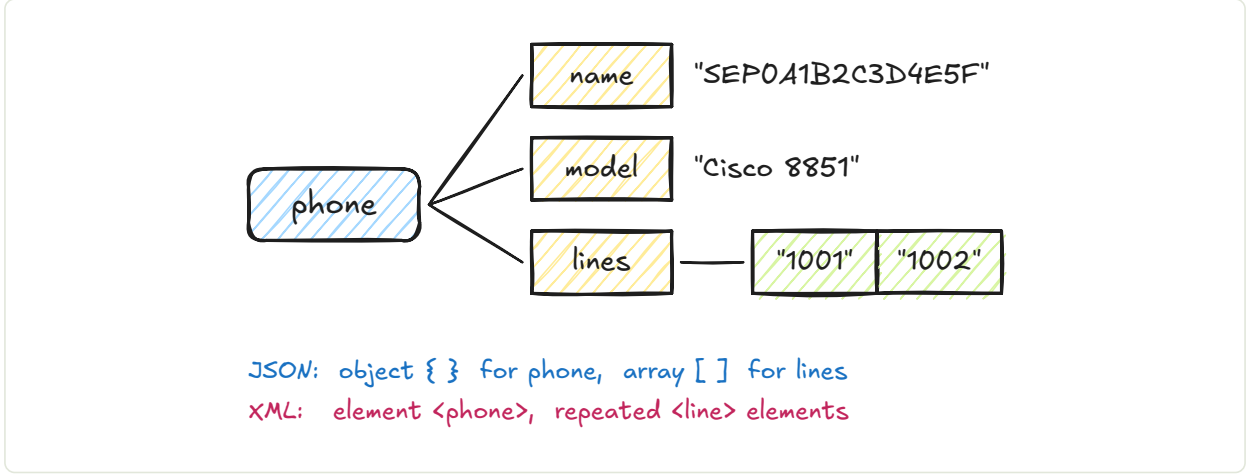
Özetle

- API, programlar için açılmış bir kapıdır. HTTP bir request / response döngüsüdür ve SIP'e çok benzer.
- Method ne yapılacağını, URL neyle yapılacağını söyler. Status code'un ilk rakamı sonucun sınıfını verir.
- `Authorization` header'ı kimliği taşır: **Basic** (kullanıcı / parola) ya da **Bearer** (token).

2.2 JSON and XML

Same data, two formats

API'ler yapılandırılmış veriyi **metin** olarak gönderir. En yaygın iki format JSON ve XML'dir. İkisi de aynı şeyi anlatır: bir **ağaç** (tree). Aşağıdaki phone'un bir adı, bir modeli ve bir line listesi var.



Şekil 2.4. Aynı veri, aynı ağaç yapısı: iki farklı yazım.

```
{"phone": {"name": "SEP0A1B2C3D4E5F",
           "model": "Cisco 8851",
           "lines": ["1001", "1002"]}}
```

```
<phone>
  <name>SEP0A1B2C3D4E5F</name>
  <model>Cisco 8851</model>
  <lines><line>1001</line><line>1002</line></lines>
</phone>
```

KODU SATIR SATIR OKUYALIM

1–3 JSON'da süslü parantez bir **nesneyi** (object) anlatır: `"key": value` çiftleri. Köşeli parantez bir **dizidir** (array). Bu, Python'daki `dict` ve `list` ile birebir örtüşür.

1–5 XML'de her bilgi bir **element**'tir: `<name>...</name>`. Açılış ve kapanış etiketleri (tag) arasında değer ya da başka element'ler bulunur.

4–5 XML'de bir dizi yoktur; aynı element tekrarlanır: iki `<line>` iki line demektir.

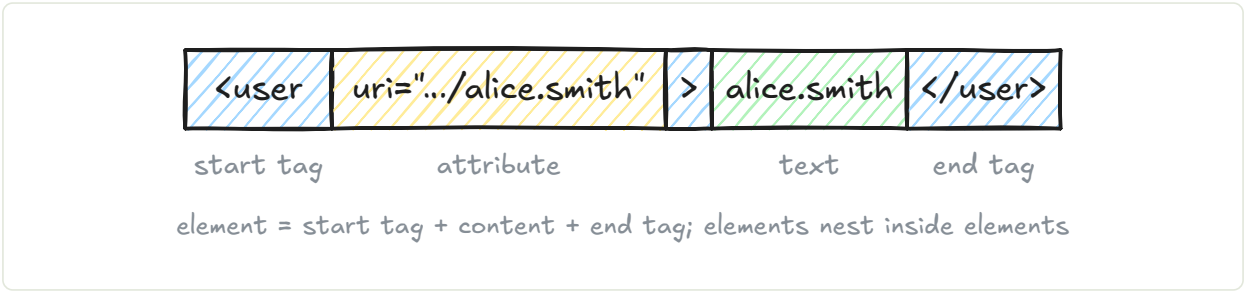
JSON	XML
nesne `{"k": v}`	element `<k>v</k>`
dizi `[a, b]`	aynı element tekrarlanır
`true` `null` ve sayılar	her şey metindir
kompakt, `dict` / `list`'e çevrilir	ayrıntılı, attribute ve namespace içerir

JSON, `json.loads()` ile doğrudan Python'a çevrilir (bkz. 1.8.2). XML ise ağacı gezen bir ayrıştırıcı (parser) ister.

> ▸ **COLLAB DÜNYASINDA**
> Webex ve çoğu yeni API JSON kullanır. AXL, RIS ve birçok CUCM ve Unity API'si XML ile cevap verir; ikisine de ihtiyacınız olacak.

Reading XML with ElementTree

Standart kütüphanedeki `ElementTree`, XML metnini bir **element ağacına** çevirir. Her element'in bir `tag`'ı, isteğe bağlı **attribute**'ları ve bir **text**'i vardır. Ağaçta `"user/userName"` gibi basit yollarla arama yaparsınız.



Şekil 2.5. Bir XML element'i: tag, attribute ve text.

```
1  import xml.etree.ElementTree as ET
2
3  root = ET.fromstring(xml_text)     # str → ağaç
4  root.get("totalCount")             # '2' (attribute)
5
6  for user in root.findall("user"):
7      print(user.findtext("userName"),
8            user.findtext("phoneNumber"))
9  # alice.smith 1001
10 # bob.jones 1002
11
12 [e.text for e in root.iter("phoneNumber")]
```

KODU SATIR SATIR OKUYALIM

1 `ElementTree`'yi `ET` takma adıyla getiriyoruz.

3 `fromstring()` XML metnini ayrıştırır ve **kök** (root) element'i döndürür.

4 `get()` bir **attribute** değerini okur. Attribute'lar `<users totalCount="2">` gibi etiketin içinde yazılır.

6 `findall("user")` kökün altındaki **bütün** `<user>` element'lerini bir list olarak verir.

7–8 `findtext()` bir alt element'in metnini döndürür.

12 `iter("phoneNumber")` ağacın **herhangi bir derinliğindeki** bütün `<phoneNumber>`'ları bulur. Burada bir comprehension ile metinlerini topluyoruz.

Metot	Ne yapar
`ET.fromstring(s)`	metni ayrıştırır, kök element'i döndürür
`el.find(path)`	ilk eşleşen alt element; yoksa `None`
`el.findall(path)`	bütün eşleşenlerin `list`'i
`el.findtext(path)`	ilk eşleşenin metni
`el.get(name)`	bir attribute değeri
`el.iter(tag)`	bu tag'e sahip her element, her derinlikte

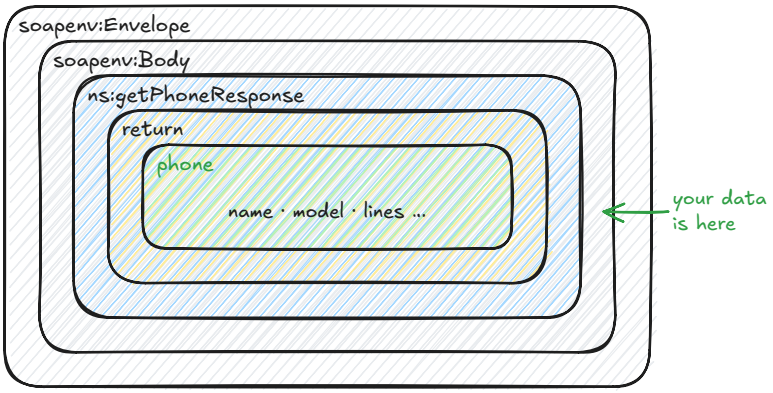
▸ **COLLAB DÜNYASINDA**

CUCM UDS kullanıcıları böyle bir XML olarak döndürür: her kullanıcı için bir `<user>` element'i ve içinde `<userName>` ile `<phoneNumber>`.

Namespaces and the SOAP envelope

Bir **namespace**, tag isimlerini benzersiz yapan bir URL'dir: `soapenv:Body` aslında `{http://…/envelope/}Body` anlamına gelir. Kısa önek (`soapenv:`, `ns:`) yalnızca `xmlns:` ile tanımlanan bir takma addır.

Bir SOAP cevabı, veriyi katmanlar halinde sarar: `Envelope`, `Body`, operasyonun cevabı, sonra `return`. Bu sarmalayıcıları atlayıp doğrudan veriye gitmek için `.//` yolu kullanılır: **herhangi bir derinlikte ara**.



Şekil 2.6. Bir SOAP cevabı: zarf (Envelope), gövde (Body) ve asıl veri.

```
1  SOAP = "http://schemas.xmlsoap.org/soap/envelope/"
2
3  root = ET.fromstring(axl_reply)
4  root.tag                          # '{' + SOAP + '}Envelope'
5  root.find(".//phone/name").text # 'SEP0A1B2C3D4E5F'
6
7  # önekli tag'ler namespace sözlüğü ister
8  root.find("soapenv:Body", {"soapenv": SOAP})
```

KODU SATIR SATIR OKUYALIM

1 Namespace URL'si bir variable'da tutulur.

3 AXL'den gelen cevabı ayrıştırıyoruz.

4 Kök element'in `tag`'ı kısa bir isim değil, namespace dahil uzun bir isimdir: `{http://schemas.xmlsoap.org/soap/envelope/}Envelope`.

5 `.//phone/name`: zarfın ve gövdenin hepsini atlayıp doğrudan `phone` altındaki `name`'e iner.

7–8 Önekli bir tag'i aramak için, öneki namespace URL'sine bağlayan bir **sözlük** verilir.

> ▸ **COLLAB DÜNYASINDA**
> AXL cevapları SOAP zarflarıdır ve AXL istekleri de aynı şekilde kurulmalıdır. Bunları elle yazmak yavaştır ve hataya açıktır. `zeep` onları sizin için kurar ve açar; bir sonraki bölümün konusu bu.

Özetle

- JSON ve XML aynı ağaç yapısını anlatır. JSON `dict` / `list`'e doğrudan çevrilir; XML için `ElementTree` kullanılır.
- `find`, `findall`, `findtext` ve `iter` ile XML ağacında gezinirsiniz.
- SOAP cevapları namespace'li zarflardır. `.//` yolu zarfı atlayıp doğrudan veriye gider.

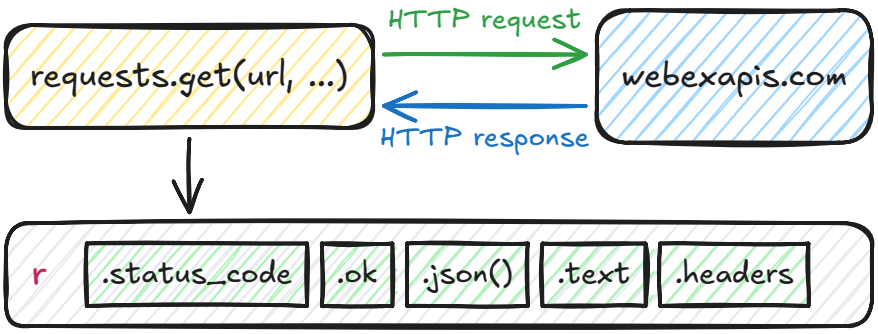
✎ Kendin dene

Küçük bir XML metni yazın: iki `<user>` element'i, her birinde `<userName>` ve `<phoneNumber>`. `ET.fromstring` ile ayrıştırıp kullanıcı adlarını ve numaralarını yazdırın.

2.3 requests

Your first request

`requests`, bir HTTP çağrısını tek bir function çağrısına çevirir. Her method için bir function vardır: `get`, `post`, `put`, `patch`, `delete`. Her biri bir `Response` object'i döndürür.



Şekil 2.7. `requests.get()` bir `Response` object'i döndürür: durum, header'lar ve body.

```python
import os
import requests          # pip install requests

token = os.environ["WEBEX_TOKEN"]
r = requests.get(
    "https://webexapis.com/v1/people/me",
    headers={"Authorization": f"Bearer {token}"},
    timeout=10,
)
r.status_code            # 200
r.json()["displayName"]  # 'Alice Smith'
r.raise_for_status()     # 4xx / 5xx ise hata
```

KODU SATIR SATIR OKUYALIM

1–2 `os` ve `requests` getirilir. `requests` standart kütüphanede değildir: `pip install requests` gerekir.

4 Token'ı koda yazmıyoruz; ortam değişkeninden okuyoruz (1.11 numaralı bölümdeki kural).

5–9 `requests.get(...)` bir GET isteği gönderir. İlk argument URL'dir. `headers=` bir dict ile header'ları verir. `timeout=10` en fazla 10 saniye beklemesini söyler.

10 `status_code` cevabın kodudur: `200`.

11 `r.json()` cevabın body'sini bir Python `dict`'ine çevirir.

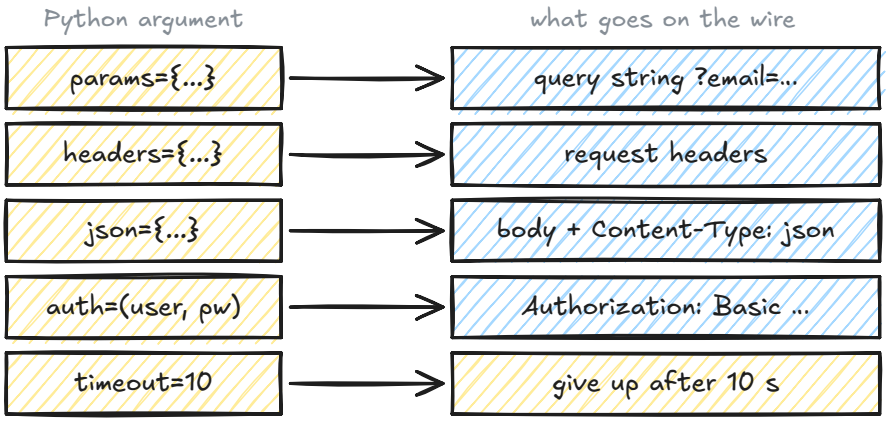
12 `raise_for_status()` cevap 4xx ya da 5xx ise bir `HTTPError` yükseltir.

Response	Verir
`r.status_code` `r.ok`	kod; 2xx ise `True`
`r.json()`	body'yi `dict` / `list`'e çevirir
`r.text` `r.content`	body `str` / `bytes` olarak
`r.headers`	cevap header'ları (bir dict)
`r.raise_for_status()`	4xx / 5xx ise `HTTPError` yükseltir

> ▸ **COLLAB DÜNYASINDA**
> `GET /v1/people/me`, Webex API'sinin "hello world"üdür: adınızı döndürüyorsa token çalışıyor demektir.

Parameters, headers and JSON bodies

`requests`'in keyword argument'ları HTTP isteğinin parçalarına birebir karşılık gelir. Kodlamayı `requests` yapar: query string'i kurar, `dict`'i JSON'a çevirir ve `Content-Type` header'ını ayarlar.



Şekil 2.8. Her keyword argument, isteğin bir parçasını oluşturur.

```
1  url = "https://webexapis.com/v1"
2  headers = {"Authorization": f"Bearer {token}"}
3
4  query = {"email": "alice.smith@clauto.lab"}
5  r = requests.get(f"{url}/people", headers=headers,
6                   params=query, timeout=10)
7  r.json()["items"][0]["displayName"]
8
9  msg = {"toPersonEmail": "bob.jones@clauto.lab",
10        "text": "SEP0A1B2C3D4E5F is down"}
11 r = requests.post(f"{url}/messages", headers=headers,
12                    json=msg, timeout=10)
```

KODU SATIR SATIR OKUYALIM

1-2 Ortak parçalar variable'lara konur: taban URL ve header'lar. Her çağrıda tekrar yazmazsınız.

4-6 `params=` bir dict alır ve URL'ye `?email=alice.smith%40clauto.lab` olarak ekler. Özel karakterleri de doğru kodlar.

7 Cevaptaki `items` bir list'tir; ilk elemanın `displayName` alanını okuruz. (Bu, 1.3.8 numaralı dict bölümündeki iç içe veri okumanın aynısı.)

9-10 Gönderilecek mesaj bir `dict` olarak hazırlanır.

11-12 `json=msg` dict'i JSON'a çevirir, body'ye koyar ve `Content-Type: application/json` ekler.

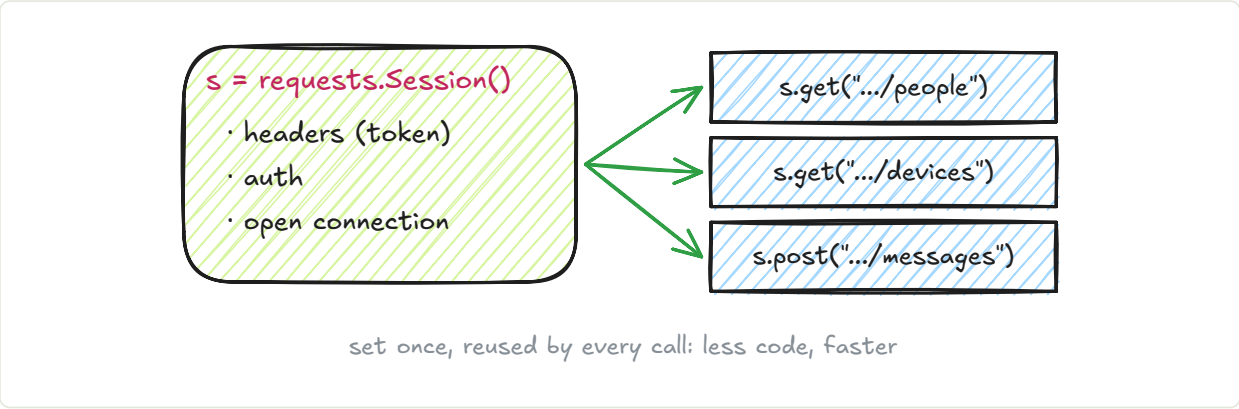
- `params=`: URL'deki filtreler için. Elle birleştirmeyin.
- `json=`: JSON body için (`POST`, `PUT`, `PATCH`).
- `headers=`: token ve formatlar için.

× SIK YAPILAN HATA

`timeout=` olmadan, sunucu cevap vermezse bir istek **sonsuza kadar** bekleyebilir ve script asılı kalır. Her zaman bir `timeout` verin, 10 ile 30 saniye arası.

Auth, sessions and certificates

Basic auth için `auth=(user, password)` yeterlidir: CUCM UDS ve Unity CUPI bunu kullanır. Aynı server'a çok sayıda istek göndereceğinizde bir `Session` kullanın: header'ları, kimlik bilgisini ve TCP bağlantısını çağrılar arasında saklar. Daha az kod, ve loop'larda çok daha hızlı.



Şekil 2.9. `Session` ayarları ve bağlantıyı çağrılar arasında yeniden kullanır.

```
1  r = requests.get(                        # Basic auth
2      "https://cuc-pub.clauto.lab/vmrest/users",
3      auth=("axl_svc", password),
4      headers={"Accept": "application/json"},
5      params={"query": "(alias is alice.smith)"},
6      verify="certs/clauto-ca.pem",        # lab CA'sine güven
7      timeout=10)
8
9  s = requests.Session()
10 s.headers.update({"Authorization": f"Bearer {token}"})
11 s.get(f"{url}/people/me", timeout=10)   # yeniden kullanılır
```

KODU SATIR SATIR OKUYALIM

1–2 Unity Connection'ın CUPI adresine bir GET.

3 `auth=(kullanıcı, parola)` Basic auth header'ını sizin yerinize kurar.

4 JSON cevap istiyoruz.

5 CUPI'ye özgü bir sorgu: alias'ı `alice.smith` olan kullanıcı.

6 `verify=` TLS sertifika kontrolünü belirler. Kendi CA sertifikanızın yolunu verirseniz kontrol çalışmaya devam eder.

9 Bir `Session` yaratılır.

10 Session'a bir kere header eklenir; bundan sonraki her çağrıda otomatik gider.

11 Çağrıda artık `headers=` yazmaya gerek yok.

> **! DIKKAT**
> `verify=False` sertifika kontrolünü kapatır ve bir `InsecureRequestWarning` yazdırır. İlk lab denemesi için kabul edilebilir, ama üretim script'lerinde **asla**.

Errors and timeouts

Bir istek üç farklı yerde başarısız olabilir ve `requests` her biri için farklı bir exception yükseltir. Hepsi `requests.exceptions` içindedir. (Exception mantığını 1.7'de gördünüz.)

```python
import requests

try:
    r = s.get(f"{url}/people/me", timeout=10)
    r.raise_for_status()
except requests.exceptions.Timeout:
    print("no answer in 10 s")
except requests.exceptions.ConnectionError:
    print("host unreachable: DNS, VPN, firewall?")
except requests.exceptions.HTTPError as e:
    print("API error:", e)
    # 401 Client Error: Unauthorized for url: …
else:
    me = r.json()
```

KODU SATIR SATIR OKUYALIM

4-5 İstek gönderilir ve hemen `raise_for_status()` ile cevap kodu kontrol edilir.

6-7 Sunucu `timeout` içinde cevap vermedi: `Timeout`.

8-9 Sunucuya hiç ulaşılamadı: DNS, VPN ya da firewall sorunu: `ConnectionError`.

10-12 Sunucu cevap verdi ama 4xx ya da 5xx: `HTTPError`. `e`, hatanın mesajını taşır.

13-14 `else`: hiç hata olmadıysa cevap kullanılır.

Durum	Yönetim
zamanında cevap yok	`timeout=` + `except Timeout`
host'a ulaşılamıyor	`except ConnectionError`
cevap 4xx / 5xx	`raise_for_status()` + `except HTTPError`

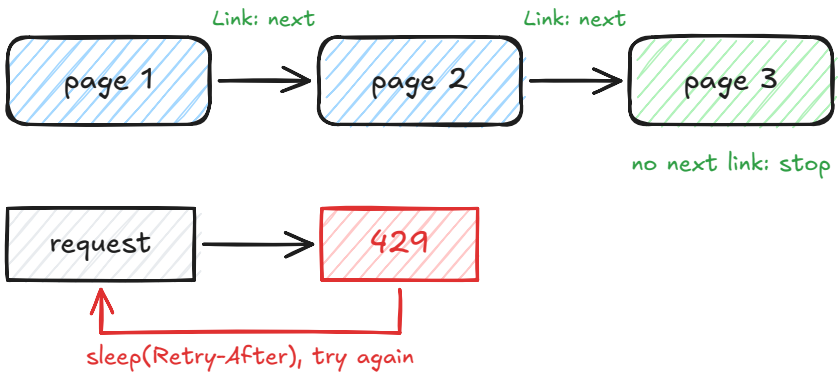
4xx ya da 5xx bir cevap kendi başına bir exception **değildir**: `r.status_code` sadece `401` olur. Onu bir exception'a çevirmek için `raise_for_status()` çağırın.

> ▸ **COLLAB DÜNYASINDA**
> Toplu işte bunları loop'un içinde yakalayın (bkz. 1.7) ve device'ı ile hatayı `failed.csv`'ye yazın.

Rate limits and paging

API'ler kendilerini korur. **Rate limit** göndereceğiniz istek sayısını sınırlar: sınırı aşarsanız `429 Too Many Requests` alırsınız ve `Retry-After` header'ı ne kadar bekleyeceğinizi söyler.

Paging uzun listeleri parçalara böler: her cevap tek bir sayfa taşır ve `rel="next"` içeren bir `Link` header'ı sonraki sayfayı gösterir. Sonraki link kalmayana kadar onu takip edersiniz.



Şekil 2.10. Her cevap bir sayfa taşır; `next` bağlantısı bitene kadar sayfalar gezilir.

```python
import time

people, next_url = [], f"{url}/people?max=100"
while next_url:
    r = s.get(next_url, timeout=10)
    if r.status_code == 429:           # yavaşla
        wait = int(r.headers.get("Retry-After", 5))
        time.sleep(wait)
        continue                       # aynı sayfa
    r.raise_for_status()
    people += r.json()["items"]
    next_url = r.links.get("next", {}).get("url")
```

KODU SATIR SATIR OKUYALIM

3 `people` boş bir list, `next_url` ilk sayfanın adresi. İkisi tek satırda tanımlanır (unpacking).

4 `while next_url:`: bir sonraki sayfa adresi var olduğu sürece devam. Adres `None` olunca `False` sayılır ve loop biter.

5 Mevcut sayfa istenir.

6–9 `429` geldiyse: `Retry-After` kadar (yoksa 5 sn) bekle ve `continue` ile **aynı sayfayı** yeniden dene.

10 Diğer hatalar `raise_for_status()` ile exception'a dönüşür.

11 Bu sayfanın `items` list'i `people`'a eklenir (`+=` ile).

12 `r.links` `Link` header'ını ayrıştırır. `next` yoksa `.get` boş dict verir ve `next_url` `None` olur: sonraki tur olmaz.

▸ **COLLAB DÜNYASINDA**

Bir Webex organizasyonunun bütün kullanıcılarını listelemek bir paging loop'udur. `r.links`, `Link` header'ını sizin için ayrıştırır: `r.links["next"]["url"]`.

Özetle

- `requests.get/post/...` bir `Response` döndürür. `r.json()` body'yi `dict`'e çevirir.
- `params=`, `json=`, `headers=` ve `auth=` isteğin parçalarını kurar. Her zaman `timeout=` verin.
- Çok çağrı için `Session` kullanın. Sertifika kontrolünü kapatmak yerine CA dosyasını `verify=` ile gösterin.
- Hataları üç kategoride yakalayın: `Timeout`, `ConnectionError`, `HTTPError`. `429` ve paging için bir loop yazın.

✎ **Kendin dene**

`https://webexapis.com/v1/people/me` isteğini bir `try / except` bloğuyla sarın. Önce yanlış bir token ile çalıştırıp `401 HTTPError`'ı yakalayın. Sonra `timeout=0.001` ile `Timeout` hatasını tetikleyin.

2.4 zeep and AXL

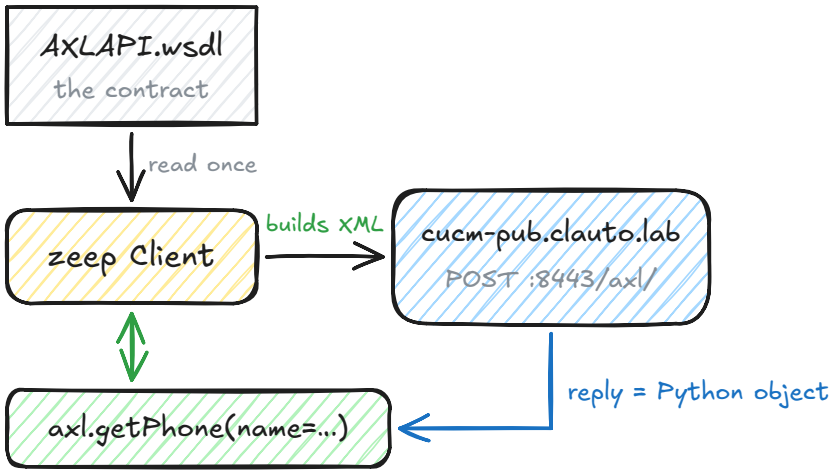
SOAP, WSDL and zeep

AXL (Administrative XML Web Service), CUCM'in konfigürasyon API'sidir. Yönetim arayüzünde yapabildiğiniz hemen her şeyi AXL ile de yapabilirsiniz: phone eklemek, line atamak, partition ve CSS oluşturmak, kullanıcıları listelemek. AXL bir **SOAP** API'sidir, yani istekler ve cevaplar XML zarfları olarak taşınır.

Bu zarfları elle yazmak yavaş ve hataya açıktır. İşte `zeep` kütüphanesi bunun için vardır. Her SOAP API'nin bir **WSDL** dosyası bulunur: bu dosya API'nin sözleşmesidir; her operasyonu, her alanı ve alanın tipini tarif eder.

`zeep` WSDL'yi bir kere okur ve her operasyonu bir Python method'una çevirir. Siz `axl.getPhone(name=...)` yazarsınız; `zeep` SOAP zarfını kurar, gönderir ve cevabı bir object'e çevirir.

> **≈ BENZETME**
> WSDL, bir cihazın **komut referansı** gibidir: hangi komutlar var, her birinin hangi parametreleri alır, hangi tiplerde. `zeep` bu referansı okuyup size her komut için hazır bir function verir.



Şekil 2.11. `zeep` WSDL'yi okur; siz bir Python method'u çağırırsınız, o SOAP zarfını kurup gönderir.

```
schema/
└── 15.0/                  her CUCM sürümü için bir klasör
    ├── AXLAPI.wsdl
    ├── AXLSoap.xsd
    └── AXLEnums.xsd
```

```
pip install zeep
```

KODU SATIR SATIR OKUYALIM

`schema/15.0/` WSDL dosyalarını CUCM **sürümüne göre** bir klasörde tutun.

`AXLAPI.wsdl` Ana sözleşme dosyası: operasyonların listesi.

`AXLSoap.xsd` Veri tiplerinin şeması: her alan ve tipi.

`AXLEnums.xsd` Sabit değer listeleri (enumeration'lar).

`pip install` `zeep`, standart kütüphanede yoktur; kurulması gerekir.

Başlamadan önce dört şeyin hazır olması gerekir:

Ön koşul	Nerede
Cisco AXL Web Service aktif	Serviceability, Publisher üzerinde
AXL application user	roller: Standard AXL API Access, Standard CCM Admin Users
Sizin sürümünüzün WSDL'si	CM Admin: Application → Plugins → AXL Toolkit
Tomcat CA sertifikası	OS Admin: Certificate Management

> **! DIKKAT**
> WSDL, cluster'ın sürümüyle eşleşmelidir. Eski bir şema "çoğunlukla çalışır" ama yeni alanları sessizce kaçırır. Her upgrade sonrası yeniden test edin.

Connecting to AXL

Bağlantı katmanlar halinde kurulur. `requests`'in bir `Session`'ı giriş bilgisini ve sertifika kontrolünü taşır. Bir `Transport` cache ve timeout ekler. `Client` WSDL'yi okur ve `create_service` onu sizin Publisher'ınıza yöneltir.

```
1  from requests import Session
2  from zeep import Client, Settings
3  from zeep.cache import SqliteCache
4  from zeep.transports import Transport
5
6  BINDING = "{http://www.cisco.com/AXLAPIService/}AXLAPIBinding"
7  URL = "https://cucm-pub.clauto.lab:8443/axl/"
8
9  session = Session()
10 session.auth = (user, password)       # os.environ'dan (1.11)
11 session.verify = "certs/clauto-ca.pem"
12 transport = Transport(session=session, timeout=30,
13                         cache=SqliteCache())
14 settings = Settings(strict=False, xml_huge_tree=True)
15 client = Client("schema/15.0/AXLAPI.wsdl",
16                 transport=transport, settings=settings)
17 axl = client.create_service(BINDING, URL)
18
19 axl.getCCMVersion()["return"]["componentVersion"]["version"]
20 # '15.0.1.11900-23'
```

KODU SATIR SATIR OKUYALIM

1–4 Gerekli sınıflar getirilir: `Session` (requests), `Client`, `Settings`, `SqliteCache` ve `Transport` (zeep).

6 `BINDING`: AXL servisinin WSDL içindeki adı. Her AXL bağlantısında aynıdır.

7 `URL`: Publisher'ın AXL adresi. İsim kullanıyoruz, IP değil.

9–11 `Session` kimlik bilgisini ve CA sertifika yolunu taşır. Parola koda yazılmaz: `user` ve `password`, 1.11'deki gibi ortam değişkenlerinden gelir.

12–13 `Transport` bu session'ı kullanır, 30 saniyelik timeout ve diske yazılan bir cache ekler.

14 `Settings` zeep'in davranışını ayarlar. `strict=False` Cisco şemasındaki küçük kusurları hoş görür.

15–16 `Client` WSDL dosyasını okur.

17 `create_service` client'ı belirli bir adrese bağlar. `axl` artık CUCM ile konuşabilen bir nesnedir.

19 Deneme çağrısı: CUCM sürümünü sorar. Cevap `["return"]` içindedir.

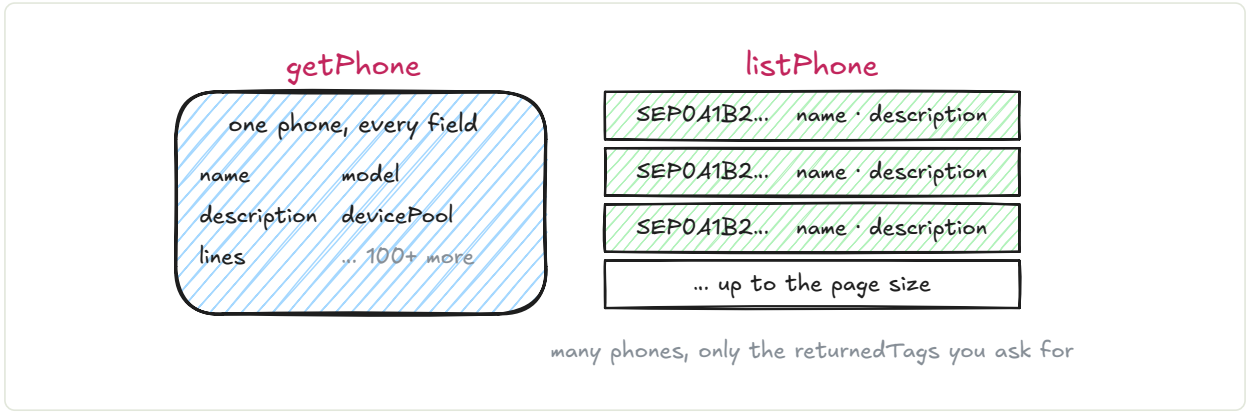
`SqliteCache`, ayrıştırılmış WSDL'yi diskte tutar: AXL şeması büyüktür ve her çalıştırmada yeniden ayrıştırmak yavaştır.

İlk iş olarak `getCCMVersion` çağırın. Bir sürüm dönerse dört şey aynı anda kanıtlanmış olur: **ağ, kimlik bilgisi, roller ve sertifika.**

> ▸ **COLLAB DÜNYASINDA**
> Bu satırları tek bir `get_axl()` function'ına ya da 1.10'daki `AxlClient` class'ına sarın. Böylece her script tek bir satırla başlar.

Reading one phone · getPhone

AXL operasyonları iki gruba ayrılır. `get` operasyonları, **tek bir** object'i **bütün alanlarıyla** döndürür. `list` operasyonları ise **birçok** object'i, yalnızca istediğiniz alanlarla döndürür. Önce `get`'e bakalım.



Şekil 2.12. `get` tek bir object'i bütün alanlarıyla, `list` birçok object'i seçilen alanlarla döndürür.

```
1  from zeep.helpers import serialize_object
2
3  resp = axl.getPhone(name="SEP0A1B2C3D4E5F")
4  phone = resp["return"]["phone"]
5
6  phone.name                # 'SEP0A1B2C3D4E5F'
7  phone.devicePoolName      # {'_value_1': 'DP_HQ',
8                            #  'uuid': '{…}'}
9  def axl_text(value):      # güvenle aç
10     return getattr(value, "_value_1", value)
11
12 axl_text(phone.devicePoolName)   # 'DP_HQ'
13 serialize_object(phone, dict)    # düz dict
```

KODU SATIR SATIR OKUYALIM

3 `getPhone(name=...)` bir phone'u adıyla ister. Operasyonun parameter'ları isimli argument'lardır (keyword argument).

4 Veri her zaman `resp["return"]` içinde, object'in adı altındadır: `phone`, `line`, `user`. `zeep` object'leri hem nokta hem köşeli parantezle çalışır.

6 Basit bir alan nokta ile okunur: `phone.name`.

7–8 Başka bir object'e işaret eden alanlar (device pool, CSS, location), düz bir metin değil, `{'_value_1': isim, 'uuid': ...}` biçiminde gelir.

9–10 Küçük bir yardımcı function: değer `_value_1` içeriyorsa onu, yoksa değerin kendisini verir.

12 `axl_text()` ile `'DP_HQ'` düz metin olarak çıkar.

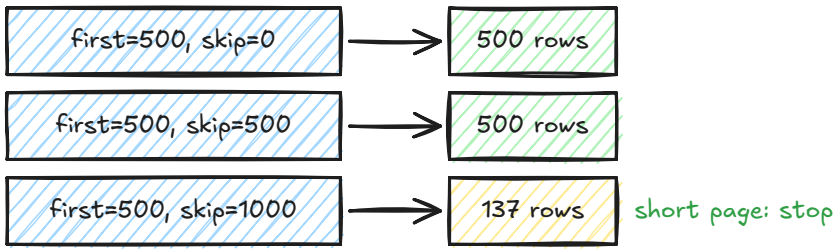
13 `serialize_object(phone, dict)` tüm zeep object'ini normal bir Python `dict`'ine çevirir; `json.dump` ile kaydetmek gibi işler için.

Bu `_value_1` şeklini kodun her yerine yaymayın: AXL sürümleri arasında şekil değişebilir. Tek bir yardımcı function üzerinden okuyun.

> **× SIK YAPILAN HATA**
> Var olmayan bir isim için `getPhone`, `None` döndürmez, bir `Fault` yükseltir. Bir sonraki konularda göreceğimiz gibi `try` / `except Fault` ile sarın.

Reading many phones · listPhone

`list` operasyonları **birçok** object döndürür, ama yalnızca `returnedTags`'te adını verdiğiniz alanlarla. Publisher'ın sağlıklı kalması için üç kural vardır.



Şekil 2.13. Sonuçlar sayfa sayfa alınır; bir sayfa kısa gelirse liste bitmiştir.

```
1  def list_all(method, criteria, tags, page=500):
2      skip = 0
3      while True:
4          resp = method(searchCriteria=criteria,
5                        returnedTags=tags,
6                        first=page, skip=skip)
7          rows = resp["return"]
8          batch = rows["phone"] if rows else []
9          yield from batch
10         if len(batch) < page:
11             return
12         skip += page
13
14 tags = {"name": "", "description": ""}
15 for p in list_all(axl.listPhone, {"name": "SEP%"},
16                   tags):
17     print(p.name, p.description)
```

KODU SATIR SATIR OKUYALIM

1 Genel bir function: hangi `list` operasyonu, hangi arama kriteri, hangi alanlar, sayfa boyutu kaç.

2 `skip`, kaç kayıt atlanacağını tutar; sayfalama için.

3 `while True`: bitene kadar dön (1.5'teki polling kalıbı).

4-6 `method(...)` seçilen `list` operasyonunu çağırır. `first` sayfa boyutu, `skip` atlanacak kayıt sayısıdır.

7-8 Sonuç boşsa `rows` `None` olabilir. Bu yüzden önce kontrol edilir; sonra bu sayfanın phone'ları `batch`'e alınır.

9 `yield from batch` bu sayfadaki phone'ları **tek tek dışarı verir**. Bu, function'ı bir **generator** yapar.

10-11 Sayfa dolu gelmediyse (`< page`) başka kayıt kalmamıştır: `return` ile bitirilir.

12 Sonraki sayfaya geç.

14 `returnedTags`: yalnızca bu alanlar istenir. Değer olarak boş string verilir.

15-17 `list_all` bir loop içinde kullanılır: her turda tek bir phone gelir.

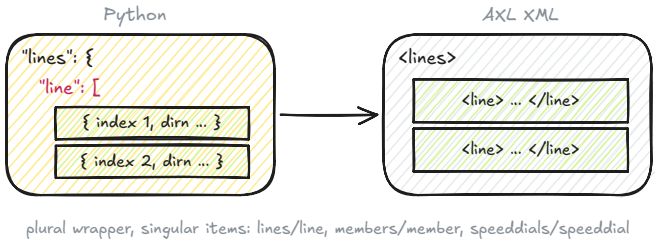
`yield from` ile `list_all` bir **generator** olur: phone'ları tek tek, sayfa sayfa dağıtır ve hepsini bellekte tutmaz. Bir comprehension'daki `(x for ...)` ile aynı fikirdir.

Kural	Neden
her zaman `returnedTags` verin	her alan × 5000 phone = aşırı yük
`searchCriteria`'yı daraltın	tek başına `%` bütün cluster'ı eşler
`first` / `skip` ile sayfalayın	sayfa kısa gelince durun

▸ **COLLAB DÜNYASINDA**
`list_all`, `listLine`, `listUser` ve diğer `list` operasyonlarıyla da çalışır: yalnızca sonuç anahtarı (`"phone"`) değişir.

Adding objects · nested data

`add` operasyonları, object'in adını taşıyan **tek bir** argument alır (`phone=`, `user=`, `css=`, `sipTrunk=`). Değeri, AXL XML'ini yansıtan iç içe bir `dict`'tir. `zeep` onu element element isteğe çevirir.



Şekil 2.14. `dict`'in yapısı AXL XML'inin yapısını yansıtır.

```
1  phone = {
2      "name": "CSFALICE",
3      "product": "Cisco Unified Client Services Framework",
4      "class": "Phone", "protocol": "SIP", "protocolSide": "User",
5      "commonPhoneConfigName": "Standard Common Phone Profile",
6      "locationName": "Hub_None", "devicePoolName": "DP_HQ",
7      "phoneTemplateName": "Standard Client Services Framework",
8      "ownerUserName": "alice.smith",
9      "lines": {"line": [
10         {"index": "1", "label": "1001",
11          "dirn": {"pattern": "1001",
12                   "routePartitionName": "PT_Internal"}},
13     ]},
14 }
15 axl.addPhone(phone=phone)      # yeni uuid'yi döndürür
```

KODU SATIR SATIR OKUYALIM

2–4 Phone'un kimliği: device adı, ürün tipi, sınıf ve protokol. Bunlar Jabber (CSF) için zorunlu alanlardır.

5–7 Phone'un bağlı olduğu diğer object'ler: common phone config, location, device pool ve phone template. Hepsi **isimle** verilir.

8 Phone'un sahibi olan kullanıcı.

9–13 Line'lar: `"lines"` altında `"line"` anahtarı bir **list** tutar. Her line bir dict'tir: `"index"` sıra numarası, `"dirn"` altında DN (`pattern`) ve partition.

15 `addPhone(phone=phone)` isteği gönderir. Başarılırsa yeni kaydın `uuid`'si döner.

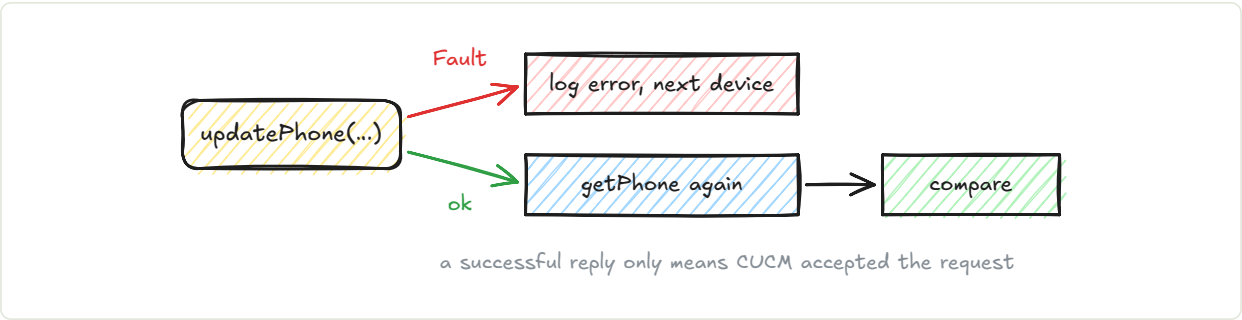
Her tekrar eden element aynı kalıbı izler: **çoğul** bir sarmalayıcı anahtar (`"lines"`), onun içinde **tekil** anahtar (`"line"`) altında bir **list**.

× **SIK YAPILAN HATA**
`updatePhone(name=…, lines={"line": […]})` bütün line listesini **değiştirir**. Yalnızca yeni line'ı değil, phone'un **tutmasını istediğiniz her line'ı** gönderin; yoksa eskileri silinir.

▸ **COLLAB DÜNYASINDA**
Aynı şekil, bir loop içinde partition eklemek (`addRoutePartition`), CSS üyeleri (`members/member`) ya da trunk destination'ları için de geçerlidir.

Writing data and handling faults

`add` , `update` ve `remove` operasyonları veritabanını değiştirir. Her yazmaya aynı şekilde davranın: `Fault` 'u yakalayın, sonucu **doğrulamak için tekrar okuyun** ve yazmalar arasında bekleyin.



Şekil 2.15. Yaz → hata yakala → tekrar oku ve doğrula → bekle.

```
1  import time
2  from zeep.exceptions import Fault
3
4  for name, desc in updates.items():
5      try:
6          axl.updatePhone(name=name, description=desc)
7      except Fault as e:
8          failed.append((name, e.message))
9          continue
10     now = axl.getPhone(name=name)["return"]["phone"]
11     if now.description != desc:
12         failed.append((name, "not applied"))
13     time.sleep(0.2)          # Publisher'ı zorlama
```

KODU SATIR SATIR OKUYALIM

2 `Fault` , AXL'in bir isteği reddettiğinde `zeep` 'in yükselttiği exception'dır.

4 `updates` bir dict: `{device adı: yeni description}` . `items()` ile çiftleri geziyoruz.

5–6 Güncelleme `try` içinde.

7–9 Hata olursa device ve hata mesajı kaydedilir, sonraki phone'a geçilir (1.7'deki toplu iş kalıbı).

10 Güncelleme sonrası phone tekrar okunur. `now` güncel veridir.

11–12 Description beklenen değer değilse, "uygulanmadı" olarak kaydedilir. İstek hatasız dönse de değişiklik uygulanmamış olabilir; bu yüzden doğrularız.

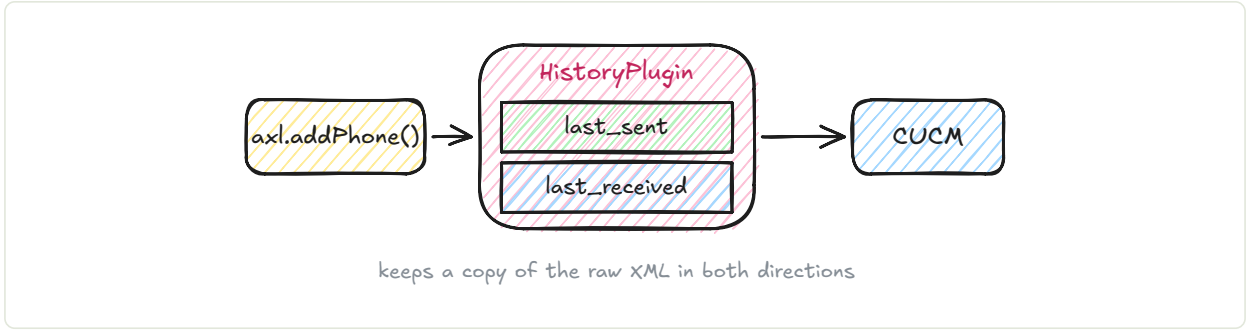
13 Yazmalar arasında kısa bir bekleme.

Fault metni (sürüme göre değişir)	Genellikle nedeni
`Item not valid … not found`	bir isim, pool ya da CSS'te yazım hatası
`duplicate value`	object zaten var
`User not authorized`	AXL rolü eksik
HTTP `503`	yazma kısıtlaması: bekleyin, sonra yeniden deneyin

> **! DIKKAT**
> Yazmaları yalnızca **Publisher**'a gönderin. AXL gerçek zamanlı değildir ve ani yük artışlarını kısıtlar. Ayrıca register durumunu bilmez: o bilgi RisPort70'te yaşar.

Seeing the raw XML · HistoryPlugin

AXL bir `Fault` ile cevap verdiğinde, tek başına mesaj çoğu zaman yeterli olmaz. `HistoryPlugin`, son isteği ve son cevabı XML olarak saklar; böylece `zeep`'in tam olarak ne gönderdiğini ve CUCM'in ne döndürdüğünü görebilirsiniz.



Şekil 2.16. `HistoryPlugin` giden isteği ve gelen cevabı ham XML olarak saklar.

```
1  from lxml import etree
2  from zeep.plugins import HistoryPlugin
3
4  history = HistoryPlugin()
5  client = Client(wsdl, transport=transport, plugins=[history])
6
7  def show_history():
8      for item in (history.last_sent, history.last_received):
9          print(etree.tostring(item["envelope"], encoding="unicode",
10                               pretty_print=True))
11
12 try:
13     axl.addPhone(phone=phone)
14 except Fault as f:
15     print(f.message)
16     show_history()          # tam olarak ne gönderildi?
```

KODU SATIR SATIR OKUYALIM

1–2 `lxml` XML'i güzel biçimde yazdırmak için; `HistoryPlugin` ise `zeep`'ten.

4 Bir `HistoryPlugin` object'i yaratılır.

5 `Client` yaratılırken `plugins=[history]` ile eklenir.

7–10 `show_history()` son gönderilen ve son alınan zarfı güzel biçimde yazdırır.

12–16 Hata olunca önce Fault mesajı, sonra ham XML yazdırılır.

Gönderilen XML'i AXL şema referansıyla ya da SoapUI'de çalışan bir istekle karşılaştırın: eksik alanlar, yanlış iç içe geçme ya da bir element adındaki yazım hatası hemen ortaya çıkar.

> **★ İPUCU**
> Script yeniyken her `except Fault` içinde `show_history()` çağırın. Script oturunca kaldırın ya da bir dosyaya loglayın.

SQL queries · executeSQLQuery

Thick AXL, CUCM veritabanında salt-okunur bir SQL sorgusu çalıştırır ve her sonuç satırı için bir `row` döndürür. Her satır bir XML element listesidir: `el.tag` sütun adı, `el.text` değerdir. Bir sorgu, onlarca `list` çağrısı gerektirecek tabloları birleştirebilir; bu da cluster geneli raporlar için idealdir.

```
sql = """
select d.name, n.dnorpattern
from device d
join devicenumplanmap m on m.fkdevice = d.pkid
join numplan n on n.pkid = m.fknumplan
where d.name like 'SEP%'
"""
resp = axl.executeSQLQuery(sql=sql)
rows = resp["return"]["row"] if resp["return"] else []

for row in rows:                # row = XML element listesi
    print({el.tag: el.text for el in row})
# {'name': 'SEP0A1B2C3D4E5F', 'dnorpattern': '1001'}
# {'name': 'SEP0A1B2C3D4E60', 'dnorpattern': '1002'}
```

KODU SATIR SATIR OKUYALIM

1–7 Üç tırnaklı bir metin, birden çok satıra yayılan SQL sorgusunu tutar. Sorgu `device`, `devicenumplanmap` ve `numplan` tablolarını birleştirir: her phone'un adını ve DN'ini getirir.

8 `executeSQLQuery(sql=sql)` sorguyu çalıştırır.

9 Sonuç boşsa `None` gelebilir: kontrol edilir.

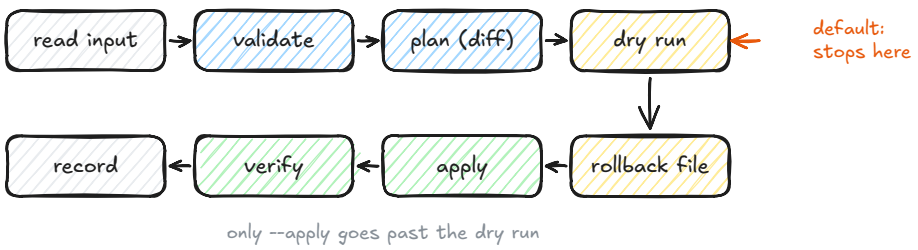
11–12 Her satır için bir dict comprehension: sütun adı → değer.

13–14 Her satır, bir phone ve onun DN'ini içeren küçük bir dict'tir.

> **! DIKKAT**
> Mümkünse normal AXL method'larını tercih edin: tablo ve sütun adları sürümler arasında değişir (kendi sürümünüzün **Data Dictionary**'sine bakın). `executeSQLUpdate`'i **asla** kullanmayın: CUCM'in iş mantığını atlar.

A safe AXL workflow

Yalnızca yazan bir script tehlikelidir. Önce kontrol eden, planı gösteren, yazan, tekrar kontrol eden ve kaydeden bir script ise bir change window'da güvenebileceğiniz bir araçtır.



Şekil 2.17. Güvenli iş akışı: oku → planla → göster → (onaylanırsa) yedekle → yaz → doğrula.

```python
import argparse

p = argparse.ArgumentParser()
p.add_argument("--input-file", required=True)
p.add_argument("--apply", action="store_true")
args = p.parse_args()

changes = plan(axl, read_input(args.input_file))
print_plan(changes)
if not args.apply:
    raise SystemExit("DRY RUN: use --apply to write")
write_rollback(changes)          # önce eski değerler
apply_and_verify(axl, changes)
```

KODU SATIR SATIR OKUYALIM

1 `argparse`, komut satırı seçeneklerini okuyan standart kütüphane modülüdür.

3–6 İki seçenek tanımlanır: zorunlu `--input-file` ve bayrak gibi çalışan `--apply`. `parse_args()` komut satırını okur.

8 Girdi dosyası okunur ve bir **değişiklik planı** üretilir. Henüz hiçbir şey yazılmadı.

9 Plan ekrana yazdırılır.

10–11 `--apply` verilmediyse script burada **durur**. Varsayılan davranış, hiçbir şeye dokunmamaktır.

12 Yazmadan önce **eski değerler** kaydedilir; bir şey ters giderse geri dönülebilir.

13 Değişiklikler uygulanır ve her biri tekrar okunarak doğrulanır.

Dry run varsayılandır. Yazmak açık bir `--apply` ister ve ilk yazmadan önce eski değerler kaydedilir.

Kullan	Ne zaman
BAT	tek tip değişiklik, BAT şablonuna uyuyor
Python + AXL	her object için karar, kontrol ya da başka bir sistemden veri gerekiyor

▸ **COLLAB DÜNYASINDA**

Çalışan örnekler: `github.com/ilkermansur/collab_automation` (AXL, RIS, CMS, Expressway, Netmiko). Daha fazlası için: “Automating Cisco Voice Administration (CUCM) with Python”, Cisco Community, 2026.

Özetle

- `zeep`, WSDL’den her AXL operasyonunu bir Python method’una çevirir. Bağlantı: `Session` → `Transport` → `Client` → `create_service`.
- `get` tek bir object’i okur (`resp["return"]["phone"]`), `list` birçok object’i seçilen alanlarla ve sayfa sayfa okur.
- `add` ve `update`, AXL XML’ini yansıtan iç içe `dict` ister. Her yazmayı `Fault` yakalayarak yapın ve sonucu tekrar okuyarak doğrulayın.
- `HistoryPlugin` ham XML’i gösterir. Güvenli iş akışı: dry run varsayılan, eski değerleri kaydet, yaz, doğrula.

✎ Kendin dene

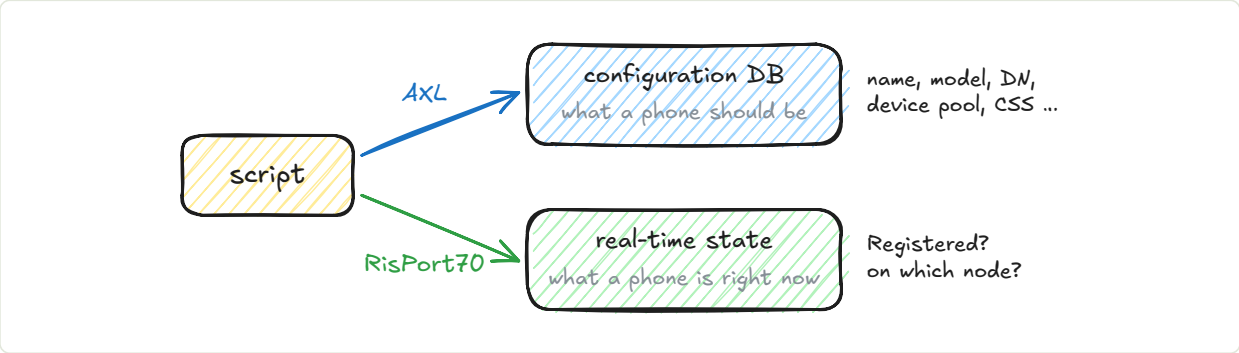
`getCCMVersion` ile bağlantıyı doğrulayın. Sonra `list_all` ile `SEP%` ile başlayan ilk 10 phone’u listeleyin ve `name` ile `description` yazdırın. Son olarak var olmayan bir phone için `getPhone` çağırıp `Fault`’u `try` / `except` ile yakalayın.

2.5 RisPort70

Real-time status with RisPort70

AXL yalnızca **konfigürasyonu** bilir: bir phone vardır, DN'i `1001`'dir, `DP_HQ` device pool'undadır. O phone'un **şu an register olup olmadığı** ise başka bir servisin işidir: **Real-time Information Service** (RIS). RisPort70, RIS'in SOAP API'sidir.

> **≈ BENZETME**
> AXL, bir **dial plan dokümanı** gibidir: ne olması gerektiğini söyler. RIS ise bir **canlı monitör** gibidir: şu an ne olduğunu söyler. Bir mühendis ikisini de kullanır: dokümanda var olan ama monitörde görünmeyen bir cihaz, bir sorundur.



Şekil 2.18. AXL konfigürasyonu (olması gerekeni), RIS durumu (şu an olanı) bilir.

`RisPort70`, AXL ile aynı araçlarla çalışır: bir `requests` `Session`, bir `zeep` `Transport` ve `create_service`. WSDL'yi CUCM'in kendisi sunar, yani indirmeniz gereken bir toolkit yoktur.

```
1 HOST = "https://cucm-pub.clauto.lab:8443"
2 RIS = HOST + "/realtimeservice2/services/RISService70"
3 NS = "{http://schemas.cisco.com/ast/soap}"
4
5 client = Client(RIS + "?wsdl", transport=transport)
6 ris = client.create_service(NS + "RisBinding", RIS)
```

KODU SATIR SATIR OKUYALIM

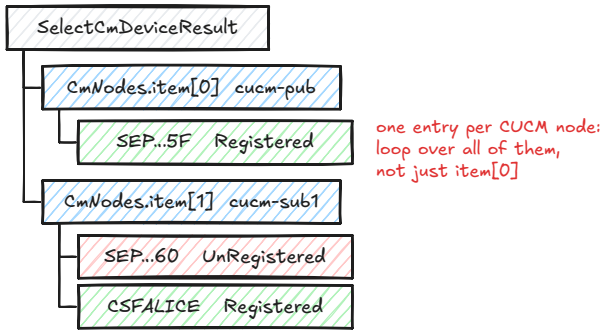
1. CUCM Publisher'ın temel adresi. Yine isim, IP değil.
2. RIS servisinin tam adresi.
3. WSDL'deki namespace.
5. `RIS + "?wsdl"`: WSDL dosyasını CUCM'in kendisinden alıyoruz. `transport` AXL bölümünde kurduğumuzla aynıdır.
6. `create_service` client'ı RIS adresine bağlar. `ris` artık RIS çağrıları yapabilir.

AXL	RisPort70
`/axl/`	`/realtimeservice2/services/RISService70`
WSDL, AXL Toolkit'ten	WSDL, `…/RISService70?wsdl` adresinden
konfigürasyon: ne **olmalı**	durum: şu an ne **var**

> **▸ COLLAB DÜNYASINDA**
> Application user'ın **Standard CCM Admin Users** ve **Standard RealtimeAndTraceCollection** rollerine sahip olması gerekir.

Asking for device status · selectCmDevice

`selectCmDevice` bir **seçim kriteri** alır: hangi device sınıfı, hangi durum ve bir isim listesi (`SEP*` gibi joker karakterler de çalışır). Cevap **CUCM node'una göre gruplanmış** gelir: bir phone, register olduğu node'un altında görünür. Bu yüzden her node üzerinde dönmek gerekir.



Şekil 2.19. RIS cevabı: her CUCM node'unun altında o node'a register olan device'lar.

```
1  def ris_status(names):
2      criteria = {
3          "MaxReturnedDevices": 1000, "DeviceClass": "Phone",
4          "Model": 255, "Status": "Any",   # 255 = her model
5          "SelectBy": "Name",
6          "SelectItems": {"item": [{"Item": n} for n in names]},
7      }
8      resp = ris.selectCmDevice(StateInfo="",
9                                CmSelectionCriteria=criteria)
10     status = {}
11     for node in resp["SelectCmDeviceResult"]["CmNodes"]["item"]:
12         found = node["CmDevices"]    # node'da yoksa None
13         for dev in (found["item"] if found else []):
14             status[dev["Name"]] = (dev["Status"], node["Name"])
15     return status
```

KODU SATIR SATIR OKUYALIM

1 Device adlarının listesini alıp, her biri için `(durum, node)` veren bir function.

2–7 Seçim kriteri bir `dict`: en fazla 1000 device, sınıf `Phone`, her model (`255`), her durum, **isme göre** seç. `SelectItems` içinde bir comprehension, her isim için bir `{"Item": isim}` kurar.

8–9 `selectCmDevice` kriterle çağrılır.

10 Sonuçlar bir dict'te toplanacak: `{device adı: (durum, node adı)}`.

11 Dış loop, cevaptaki **her CUCM node'unu** gezer.

12 Bir node'da hiç device yoksa `CmDevices` `None` olur; bu yüzden önce kontrol edilir.

13–14 İç loop, node'daki her device için `(durum, node)` kaydeder.

15 Toplanan sözlük geri verilir.

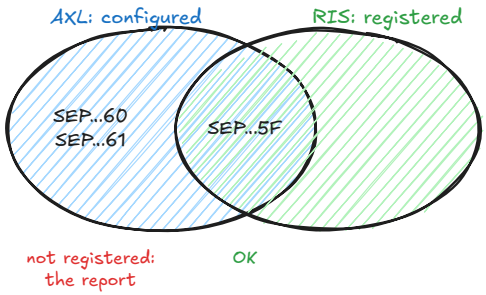
Durum	Anlamı
`Registered`	o node'da online
`UnRegistered`	register olmuştu, şimdi yok
`Rejected`	CUCM register'ı reddetti

× SIK YAPILAN HATA
Yalnızca `CmNodes["item"][0]` okumak sadece Publisher'ı kontrol eder. Subscriber'a register olan phone'lar **sessizce** kaçırılır.

Limits and an unregistered-phone report

RIS, cluster'ı sert limitlerle korur: istek başına en fazla **1000 device** ve varsayılan olarak dakikada yaklaşık **15 istek**. Daha büyük envanterler parçalar halinde, aralarda bekleyerek gönderilir.

Son CUCM restart'ından beri register olmayı hiç denememiş bir phone, RIS'te hiç yer almaz. Onu yalnızca AXL envanteriyle birleştirerek bulabilirsiniz: konfigüre edilmiş ama "görülmemiş".



Şekil 2.20. AXL envanteri ile RIS durumu birleştirilir; RIS'te hiç görünmeyenler de yakalanır.

```
1  def chunks(items, size=1000):        # istek başına maks.
2      for i in range(0, len(items), size):
3          yield items[i:i + size]
4
5  phones = list_all(axl.listPhone, {"name": "SEP%"},
6                    {"name": ""})              # AXL
7  names = [p.name for p in phones]
8  status = {}
9  for part in chunks(names):
10     status.update(ris_status(part))          # RIS
11     time.sleep(5)                            # limit
12
13 for name in names:
14     state, node = status.get(name, ("not seen", "-"))
15     if state != "Registered":
16         print(name, state, node)
```

KODU SATIR SATIR OKUYALIM

1–3 `chunks()` bir list'i 1000'erli parçalara böler. `range(0, len(items), size)` adım adım ilerler; `yield` her parçayı tek tek verir (generator).

5–6 AXL ile envanter alınır: `SEP%` ile başlayan bütün phone'lar.

7 Bir comprehension yalnızca isimleri toplar.

9–11 Her parça için RIS sorulur; sonuçlar `status` sözlüğüne eklenir. Her istekten sonra 5 saniye beklenir: RIS'in dakikada 15 istek sınırına uymak için.

13 **Her** konfigüre edilmiş phone için döngü.

14 `status.get(name, ...)`: phone RIS cevabında yoksa varsayılan `("not seen", "-")` kullanılır. Tuple **unpacking** ile `state` ve `node` ayrılır.

15–16 Durum `Registered` değilse (kayıtsız, reddedilmiş ya da hiç görülmemiş), phone raporlanır.

▸ **COLLAB DÜNYASINDA**
Bu raporu her sabah çalıştırın: AXL listesi **ne online olması gerektiğini**, RIS **ne olduğunu** söyler. Aradaki fark, saha ekibine gidecek listedir.

Özetle

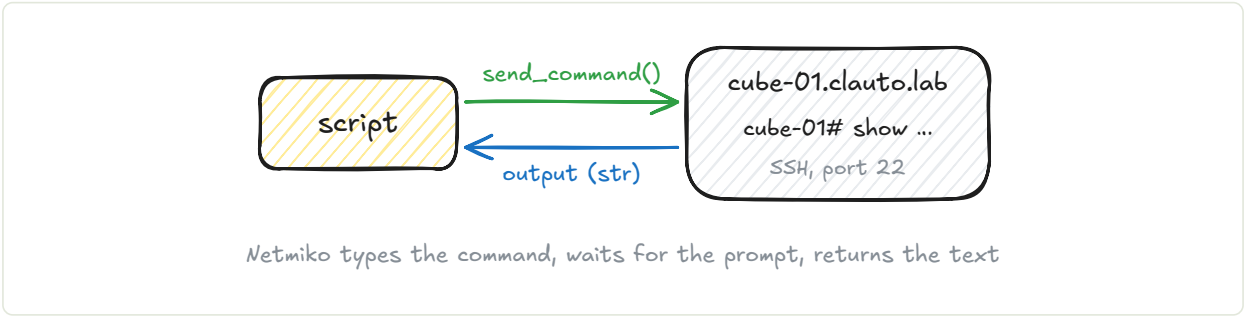
- AXL konfigürasyonu, RIS canlı durumu bilir. İkisini birleştirmek "olması gereken ve olan" farkını verir.
- `selectCmDevice` sonuçları node'a göre gruplar: **bütün** node'ları gezin.
- RIS'in limitleri: istek başına 1000 device, dakikada 15 istek. Parçalayın ve bekleyin.

✎ **Kendin dene**
`ris_status` function'ını iki phone adıyla (`["SEP0A1B2C3D4E5F", "SEP0A1B2C3D4E60"]`) çağırın. Dönen sözlüğü bir `for` loop'u ile `ad: durum (node)` biçiminde yazdırın.

2.6 Netmiko

CLI automation over SSH

Her cihazın bir API'si yoktur. CUBE ve IOS voice gateway'ler **CLI** üzerinden yapılandırılır. **Netmiko**, bir SSH session'ı açar, bir komut gönderir, prompt'u bekler ve çıktıyı bir `str` olarak döndürür. Elle SSH'a girip komut yazıp çıktıyı kopyalamanın Python karşılığıdır.



Şekil 2.21. Netmiko SSH ile bağlanır, komutu gönderir ve çıktıyı metin olarak döndürür.

```
1  import os
2  from netmiko import ConnectHandler   # pip install netmiko
3
4  cube = {
5      "device_type": "cisco_xe",
6      "host": "cube-01.clauto.lab",
7      "username": os.environ["CUBE_USER"],
8      "password": os.environ["CUBE_PASSWORD"],
9  }
10
11 with ConnectHandler(**cube) as conn:     # SSH girişi
12     print(conn.send_command("show version | include uptime"))
13 # cube-01 uptime is 12 weeks, 3 days, 4 hours
```

KODU SATIR SATIR OKUYALIM

2 `ConnectHandler`, bağlantıyı kuran sınıftır. `pip install netmiko` ile kurulur.

4–9 Bağlantı bilgileri bir `dict`'te tutulur: cihaz tipi, host adı, kullanıcı ve parola. Kullanıcı ve parola, koda değil ortam değişkenlerinden okunur (1.11).

11 `ConnectHandler(**cube)` : `**` dict'i isimli argument'lara açar (1.6.1). `with` ile bağlanılır; blok bitince oturum kapanır.

12 `send_command()` tek bir `show` komutu çalıştırır ve çıktıyı metin olarak döndürür.

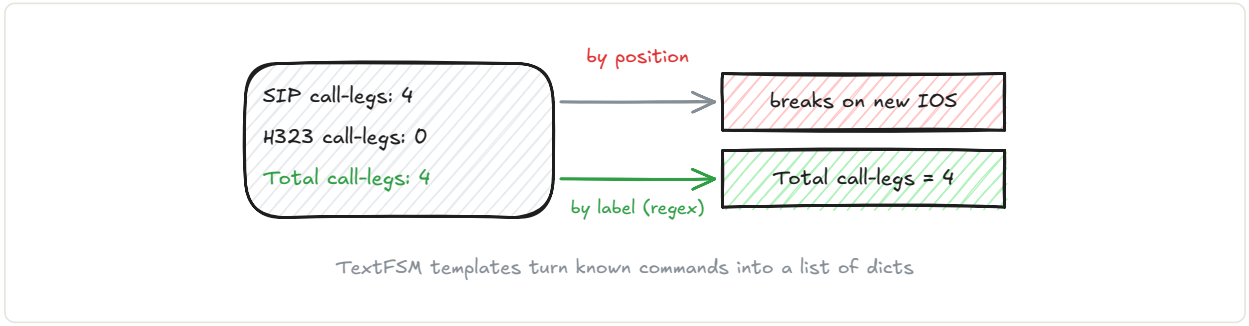
Parça	Anlamı
`device_type`	CLI türü: IOS-XE üzerindeki CUBE için `cisco_xe`
`ConnectHandler(**d)`	giriş yapar; `**` dict'i açar (1.6.1)
`with … as conn`	bitince otomatik çıkış yapar
`send_command(cmd)`	bir `show` komutu çalıştırır, metni verir

× SIK YAPILAN HATA
`with` kullanmıyorsanız `conn.disconnect()` çağırmayı kendiniz hatırlayın. Unutulan oturumlar, router'ın VTY hatları tükenene kadar açık kalır.

Turning show output into data

CLI çıktısı, insanlar için üretilmiş bir metindir. Bir değeri satır ve sütun numarasıyla seçmek (`splitlines()[13].split()[4]`), bir yazılım güncellemesi bir satır eklediği anda bozulur.

- **Regex** (`re`), bir değeri **etiketiyle** bulur: `Total call-legs:` satırı nereye kayarsa kaysın çalışır.
- **TextFSM** (`use_textfsm=True`), bilinen komutları ntc-templates koleksiyonuyla bir dict list'ine ayrıştırır.



Şekil 2.22. Ham CLI metninden regex ya da TextFSM ile yapılandırılmış veri çıkarılır.

```
1  import re
2
3  out = conn.send_command("show call active voice brief")
4  m = re.search(r"Total call-legs:\s*(\d+)", out)
5  legs = int(m.group(1)) if m else 0
6  calls = legs // 2          # CUBE'da bir call = iki leg
7
8  rows = conn.send_command("show ip interface brief",
9                           use_textfsm=True)
10 rows[0]   # {'interface': 'GigabitEthernet1', 'status': 'up', …}
```

KODU SATIR SATIR OKUYALIM

1 `re` standart kütüphanenin düzenli ifade (regular expression) modülüdür.

3 Aktif call'ları gösteren komutun çıktısı alınır.

4 `re.search()` metinde bir **desen** arar. `Total call-legs:` sabit etikettir; `\s*` "sıfır ya da daha fazla boşluk"; `(\d+)` "bir ya da daha fazla rakam, parantezle yakala" demektir. Başındaki `r` metni **ham string** yapar: `\` karakterleri olduğu gibi kalır.

5 Bulunduysa (`m` `None` değilse), yakalanan rakamlar `m.group(1)` ile alınır ve `int`'e çevrilir; bulunamadıysa `0`.

6 CUBE'da her call iki leg'den oluşur: yarısı call sayısıdır.

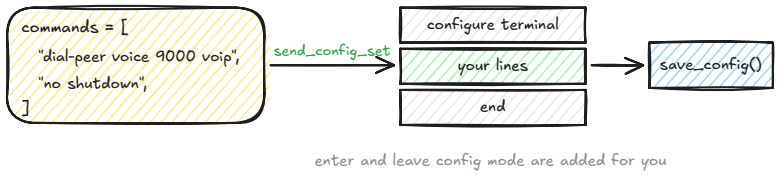
8–9 `use_textfsm=True` çıktıyı bilinen bir şablonla otomatik olarak dict'lerin list'ine çevirir.

10 Her satır bir dict: sütun adı → değer.

> **! DIKKAT**
> ntc-templates birçok routing komutunu kapsar ama sesle ilgili komutları neredeyse hiç kapsamaz (bugün için yalnızca `show isdn status`). CUBE için regex'i planlayın.

Changing configuration

`send_config_set`, bir **list** olarak config satırları alır: configuration moduna girer, her satırı gönderir ve çıkar. `save_config()` ise running config'i startup'a yazar (`write memory`).



Şekil 2.23. `send_config_set` config moduna girer, satırları gönderir ve çıkar.

```
1  def alternate_announcement(conn, enable):
2      commands = [
3          "dial-peer voice 9000 voip",
4          "no shutdown" if enable else "shutdown",
5      ]
6      out = conn.send_config_set(commands)
7      if "% Invalid" in out:
8          raise RuntimeError(out)
9      conn.save_config()              # write memory
10
11 with ConnectHandler(**cube) as conn:
12     alternate_announcement(conn, enable=True)
13     print(conn.send_command("show dial-peer voice summary"))
```

KODU SATIR SATIR OKUYALIM

1 Alternatif anons dial-peer'ini açan ya da kapatan bir function. `enable` bir `bool`.

2–5 Gönderilecek komutlar bir list'te: önce dial-peer `9000`'e gir, sonra `enable`'a göre `no shutdown` ya da `shutdown` yaz. Tek satırlık `x if koşul else y` kalıbı (1.4).

6 Komutlar gönderilir; Netmiko config moduna girip çıkar. Router'ın cevabı `out`'ta toplanır.

7–8 Router yanlış bir komutta bir **exception yükseltmez**; `% Invalid input` ile cevap verir. Bu yüzden çıktıyı kendimiz kontrol eder ve hata varsa `raise` ile bir exception yükseltiriz.

9 Her şey yolundaysa konfigürasyon kaydedilir.

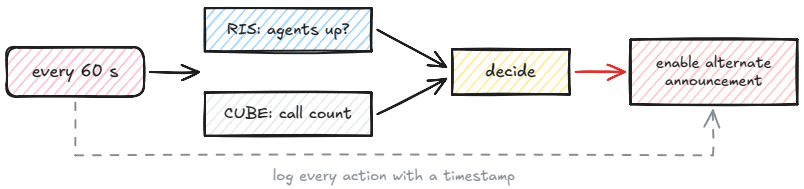
11–13 Bağlan, değişikliği uygula ve sonucu bir `show` komutuyla **doğrula**.

Router yanlış bir komuta exception ile değil metinle cevap verdiği için, dönen metni kontrol edin ve sonucu bir `show` komutuyla doğrulayın.

> **× SIK YAPILAN HATA**
> Listeye `"conf t"` koymayın: Netmiko zaten config moduna girdi; fazladan satır yalnızca gürültü ya da hata ekler.

Scheduled checks · RIS + CUBE

Script'ler, kaynakları birleştirdiğinde en yararlı hale gelir: RIS contact-center agent'larının register olup olmadığını bilir, CUBE ise çağrıların nereye gideceğine karar verir. `time.sleep()` ile bir loop (ya da `schedule` paketi) bu kontrolü her dakika çalıştırır.



Şekil 2.24. RIS'ten agent durumu okunur; sonuca göre CUBE'daki dial-peer açılır ya da kapatılır.

```
1  import time
2  from netmiko import (NetmikoTimeoutException,
3                       NetmikoAuthenticationException)
4
5  agents = ["CSFAGENT01", "CSFAGENT02"]
6
7  while True:
8      status = ris_status(agents)                  # 2.5
9      up = any(s == "Registered" for s, _ in status.values())
10     try:
11         with ConnectHandler(**cube) as conn:
12             alternate_announcement(conn, enable=not up)
13     except (NetmikoTimeoutException,
14             NetmikoAuthenticationException) as e:
15         print(time.asctime(), "CUBE not reachable:", e)
16     time.sleep(60)
```

KODU SATIR SATIR OKUYALIM

2–3 Netmiko'nun kendi iki exception'ı: cihaz cevap vermezse `NetmikoTimeoutException`, giriş yanlışsa `NetmikoAuthenticationException`.

5 İzlenecek iki agent Jabber client'ı.

7 `while True`: sonsuza kadar kontrol et.

8 RIS'ten (2.5) agent'ların durumu sorulur: `{ad: (durum, node)}`.

9 `any(...)`: agent'lardan **en az biri** `Registered` ise `up` `True` olur. Generator içinde `for s, _ in ...` tuple'ı açar; `_` "bu değer işime yaramıyor" demenin gelenekselidir.

10–12 CUBE'a bağlanıp dial-peer'i ayarlarız: hiçbir agent online değilse (`not up`) alternatif anons **açılır**.

13–15 CUBE'a ulaşılamazsa ya da giriş başarısız olursa loop'u öldürmek yerine hata zaman damgasıyla yazılır.

16 60 saniye bekle, sonra yeniden kontrol et.

Netmiko, cevap vermeyen bir host ve yanlış bir giriş için kendi exception'larını yükseltir. Onları yakalayın, loglayın ve loop'u hayatta tutun.

> ▸ **COLLAB DÜNYASINDA**
> Hiçbir agent'ın Jabber'ı register değilse, arayanları alternatif bir anonsa gönderen dial-peer'i açın. Bir agent geri dönünce yeniden kapatın.

Özetle

- Netmiko, API'si olmayan cihazlarda CLI'ı otomatikleştirir: `ConnectHandler` ile bağlan, `send_command` ile oku, `send_config_set` ile yaz.
- CLI çıktısını satır numarasıyla değil, etiketiyle ayrıştırın: regex ya da TextFSM.
- Router yanlış komuta hata fırlatmaz: çıktıyı kontrol edin ve sonucu `show` ile doğrulayın.

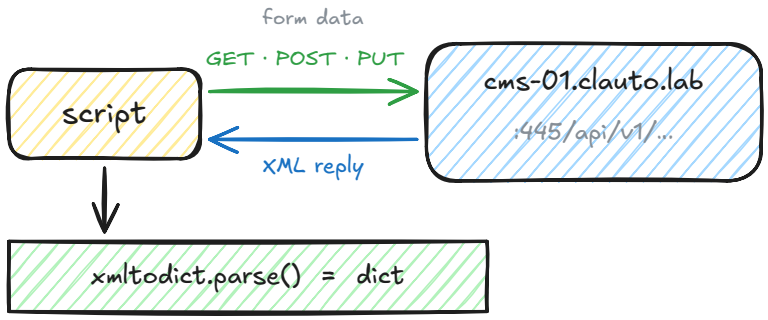
✎ Kendin dene

`send_command("show version | include uptime")` çıktısından regex ile "weeks" önündeki sayıyı yakalayıp yazdırın. (Lab ortamınız yoksa örnek çıktıyı bir string olarak yazıp `re.search`'ü onun üzerinde deneyin.)

2.7 CMS REST API

Cisco Meeting Server API

CMS'in, **Web Admin** portunda (genellikle 445) bir REST API'si vardır. Basic auth kullanır, yazma işlemlerinde **form data** gönderir ve JSON değil **XML** ile cevap verir. `xmltodict` kütüphanesi XML'i iç içe `dict`'lere çevirir; böylece geri kalanı Webex'ten gelen JSON ile çalışmak gibi hissettirir.



Şekil 2.25. Script CMS'e Basic auth ile bağlanır; cevap XML olarak gelir ve `xmltodict` ile dict'e çevrilir.

```
1  import os
2  import requests
3  import xmltodict                  # pip install xmltodict
4
5  BASE = "https://cms-01.clauto.lab:445/api/v1"
6  s = requests.Session()
7  s.auth = (os.environ["CMS_USER"], os.environ["CMS_PASSWORD"])
8  s.verify = "certs/clauto-ca.pem"
9
10 r = s.get(f"{BASE}/system/status", timeout=10)
11 r.raise_for_status()
12 status = xmltodict.parse(r.content)["status"]
13 status["softwareVersion"], status["callLegsActive"]
14 # ('3.10', '2')
```

KODU SATIR SATIR OKUYALIM

3 `xmltodict` bir üçüncü parti pakettir: `pip install xmltodict`.

5 CMS API'sinin taban adresi (API portu `445`).

6–8 Bir `Session` yaratılır; kullanıcı, parola ve CA sertifikası bir kere ayarlanır. Yine kimlik bilgisi ortam değişkeninden gelir.

10–11 Sistem durumu istenir; hata varsa exception yükseltilir.

12 `r.content` ham XML (bytes'tır). `xmltodict.parse()` onu iç içe bir dict'e çevirir. `["status"]` kök element'in içine iner.

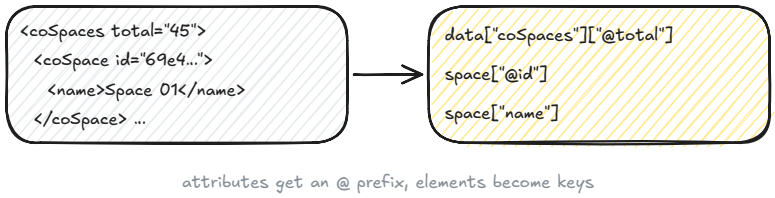
13 Yazılım sürümü ve aktif call leg sayısı. XML'deki değerler her zaman **metin** olarak gelir.

Object	Endpoint
space'ler	`/coSpaces` , `/coSpaces/<id>`
aktif toplantılar	`/calls`
katılımcılar	`/calls/<id>/callLegs`
sunucu sağlığı	`/system/status`

> ▸ **COLLAB DÜNYASINDA**
> Script'ler için, admin girişinden ayrı, CMS'te bir **API user** oluşturun (`api_svc`) ve onu `.env` içinde tutun (1.11).

Example 1 · list all spaces

CMS listeleri küçük sayfalar halinde döndürür. Kök element, toplam sayıyı (`total`) taşır. `total` kadar kayıt toplayana kadar `offset` 'i sayfa sayfa ilerletirsiniz. `filter` aramayı sunucuda isme göre yapar: bir space'i bulmak için hepsini indirmezsiniz.



Şekil 2.26. XML'deki `@` ile başlayan anahtarlar attribute'lardır; diğerleri alt element'lerdir.

```python
def list_spaces(name_filter=""):
    spaces, offset = [], 0
    while True:
        params = {"offset": offset, "limit": 20,
                  "filter": name_filter}
        r = s.get(f"{BASE}/coSpaces", params=params, timeout=10)
        r.raise_for_status()
        xml = xmltodict.parse(r.content, force_list=("coSpace",))
        data = xml["coSpaces"]
        page = data.get("coSpace", [])
        spaces += page
        offset += len(page)
        if not page or offset >= int(data["@total"]):
            return spaces

for sp in list_spaces():
    print(sp["@id"], sp["name"])
```

KODU SATIR SATIR OKUYALIM

1 `name_filter` varsayılan olarak boş: filtre yok.

2 Toplanan space'ler ve sayfalama konumu (`offset`).

4–5 Her sayfada 20 kayıt iste, isteğe bağlı bir isim filtresiyle.

6–7 İstek gönderilir ve hata kontrol edilir.

8 `force_list=("coSpace",)`: tek bir sonuç gelse bile `coSpace` **her zaman bir list** olsun. Bu konunun altındaki uyarıya bakın.

9 Kök element: toplam sayıyı ve sayfayı taşır.

10 Bu sayfadaki space'ler. Sonuç yoksa boş list.

11–12 Sayfa `spaces`'e eklenir ve `offset` sayfa boyutu kadar ilerler.

13–14 Sayfa boşsa ya da toplam sayıya ulaşıldıysa, bitir.

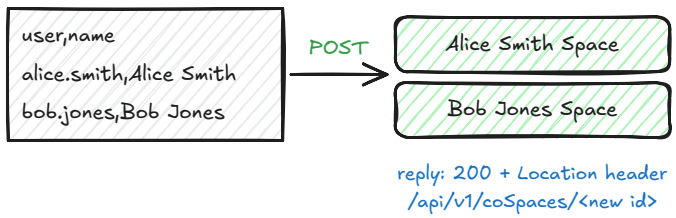
16–17 Sonuçlar bir loop ile yazdırılır: `@id` bir attribute, `name` bir alt element.

× SIK YAPILAN HATA

Tek bir sonuç olduğunda `xmltodict` bir `list` yerine **tek bir** `dict` döndürür ve `for` loop'u onun key'lerini gezer. Her zaman bir list almak için `force_list=("coSpace",)` verin.

Example 2 · create spaces from a CSV

CMS'e yazma işlemleri **form data** gönderir: `requests`'te `json=` değil `data=` kullanılır. Başarılı bir `POST` boş bir body döndürür; yeni objenin kimliği `Location` header'ındadır.



Şekil 2.27. CSV'deki her satır için bir `POST`; yeni space'in kimliği `Location` header'ından okunur.

```
1  import csv
2
3  with open("users.csv", newline="", encoding="utf-8") as f:
4      for row in csv.DictReader(f):
5          form = {"name": f"{row['name']} Space",
6                  "uri": f"{row['user']}.space",
7                  "passcode": "4711"}
8          r = s.post(f"{BASE}/coSpaces", data=form, timeout=10)
9          if r.status_code != 200:
10             print("failed:", row["user"], r.text)
11             continue
12         space_id = r.headers["Location"].split("/")[-1]
13         print("created", form["uri"], space_id)
14
15 s.put(f"{BASE}/coSpaces/{space_id}", data={"passcode": "2026"})
16 s.delete(f"{BASE}/coSpaces/{space_id}")
```

KODU SATIR SATIR OKUYALIM

3–4 CSV 1.8.1'deki gibi `DictReader` ile satır satır okunur.

5–7 Her satırdan bir **form** sözlüğü kurulur: space adı, URI ve passcode. f-string içinde dict'e erişirken **tek tırnak** kullanılır (`row['name']`), çünkü dış tırnak çifttir.

8 `data=form` ile form olarak gönderilir.

9–11 Başarısızsa satırı ve hata metnini yazdır, sonraki satıra geç (`continue`).

12 `Location` header'ı yeni objenin adresidir: `.../coSpaces/<id>`. `split("/")[-1]` adresin son parçasını, yani kimliği alır.

13 Başarı mesajı.

15 `PUT`: yalnızca gönderdiğiniz alanları değiştirir; burada passcode.

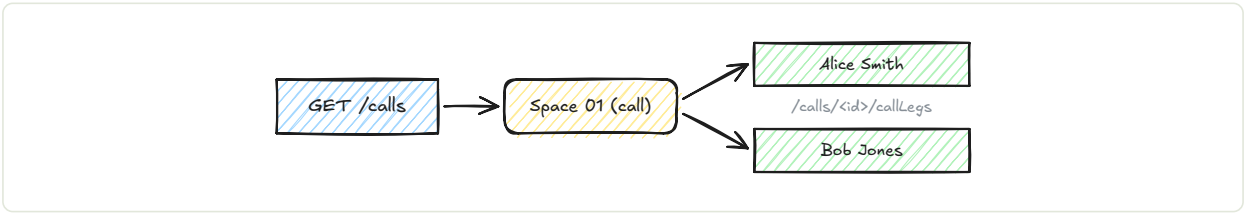
16 `DELETE`: space'i siler.

Çağrı	Etkisi
`POST /coSpaces`	bir space yaratır
`PUT /coSpaces/<id>`	yalnızca gönderdiğiniz alanları değiştirir
`DELETE /coSpaces/<id>`	space'i siler

Hatalar `400` kodu ve bir `<failureDetails>` element'iyle gelir; örneğin `duplicateCoSpaceUri`. Loglayın ve devam edin (1.7).

Example 3 · who is in which meeting

Aktif bir toplantı bir **call**'dur; içindeki her katılımcı bir **call leg**'dir. İç içe iki istek, canlı bir toplantı raporu verir. `get_xml()` gibi küçük bir yardımcı, ayrıştırmayı tek bir yerde tutar; böylece her yeni rapor birkaç satır olur.



Şekil 2.28. Her call'ın altında katılımcıları (call leg'leri) tutan ikinci bir istek yapılır.

```
1  def get_xml(path, key, item):
2      r = s.get(f"{BASE}{path}", timeout=10)
3      r.raise_for_status()
4      return xmltodict.parse(r.content, force_list=(item,))[key]
5
6  calls = get_xml("/calls", "calls", "call")
7  for call in calls.get("call", []):
8      legs = get_xml(f"/calls/{call['@id']}/callLegs",
9                     "callLegs", "callLeg")
10     print(f"{call['name']}: {legs['@total']} participants")
11     for leg in legs.get("callLeg", []):
12         print("   ", leg["name"], leg["remoteParty"])
13 # Space 01: 2 participants
14 #     Alice Smith alice.smith@clauto.lab
15 #     Bob Jones bob.jones@clauto.lab
```

KODU SATIR SATIR OKUYALIM

1–4 Yardımcı function: bir adrese GET atar, hatayı kontrol eder, XML'i ayrıştırır ve verilen `key`'in altını döndürür. `force_list` ile liste olması garanti edilir.

6 Aktif bütün call'lar (toplantılar) alınır.

7 Her call için döngü. Aktif toplantı yoksa `.get("call", [])` boş list verir ve loop hiç dönmez.

8–9 Her call'ın katılımcıları (`callLegs`) ayrı bir istekle alınır. `call['@id']` call'ın kimliğidir.

10 Toplantı adını ve katılımcı sayısını yazdır.

11–12 İç loop: her katılımcının adını ve uzak tarafı (kim olduğu) yazdır.

> ▸ **COLLAB DÜNYASINDA**
> Bakımdan önce çalıştırın: aktif call yoksa, CMS bir toplantıyı düşürmeden yeniden başlatılabilir. Tek bir sabah özeti için bunu günlük RIS raporuna (2.5) ekleyin.

Özetle

- CMS REST API'si XML ile konuşur. `xmltodict` onu dict'lere çevirir: `@` attribute, diğerleri alt element'tir.
- Listeler sayfalıdır: `offset` ve `total` ile gezin. Tek sonuç sorununu `force_list` ile çözün.
- Yazmalarda `data=` (form data) kullanın. Yeni kimlik `Location` header'ındadır.

✎ Kendin dene

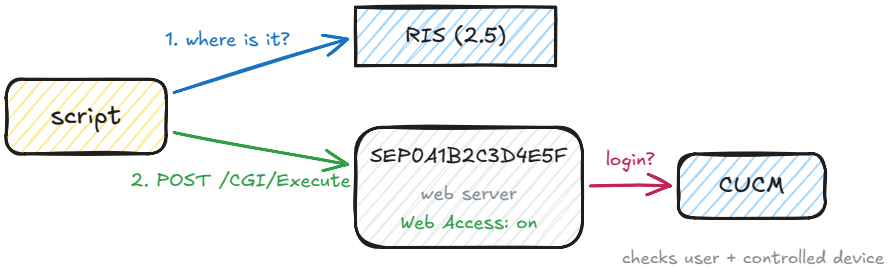
`list_spaces()` function'ını `name_filter="Sales"` ile çağırın. Sonuç boş olduğunda ve tek bir space döndüğünde `for sp in list_spaces(...)` loop'unun nasıl davrandığını, `force_list` olmadan ve onunla gözlemleyin.

2.8 Remote Phone Control

The phone's own web server

Bazı işlerin hiç API'si yoktur: ITL dosyasını silmek, bir menü ayarını sıfırlamak, ekranın fotoğrafını çekmek. Ama her Cisco IP phone küçük bir web server çalıştırır ve `/CGI/Execute`, bir script'in sanki phone'un başında biri duruyormuş gibi **tuşlara basmasına** ya da **numara çevirmesine** izin verir.

> **≈ BENZETME**
> Bir phone'un kendi web server'ı, cihazın üzerindeki **tuş takımının** ağdan erişilebilir bir kopyası gibidir. Script, parmaklarınızın yerine geçer: Settings'e basar, menüde yürür, tuşa basar.



Şekil 2.29. Script, phone'un `/CGI/Execute` adresine bir XML gönderir; phone onu tuş basması gibi uygular.

```
POST http://<phone>/CGI/Execute
Content-Type: application/x-www-form-urlencoded

XML=<CiscoIPPhoneExecute>
      <ExecuteItem URL="Key:Settings"/>
    </CiscoIPPhoneExecute>
```

KODU SATIR SATIR OKUYALIM

`POST` Phone'un kendi adresine, `/CGI/Execute` yoluna bir istek.

`Content-Type` Form data olarak gönderilir.

`XML=...` Body'de `XML` adlı tek bir alan vardır. İçindeki `ExecuteItem`, tek bir eylemdir: burada `Settings` tuşuna bas.

Phone'un adresini elle yazmak yerine RIS'ten alırsınız. `ris_status` function'ının (2.5) bir benzeri, durum yerine IP adresini de saklar:

```python
def phone_hosts(names):      # ris_status (2.5), ama
    ...                      # adresi de saklar
    for dev in devs:
        ip = dev["IPAddress"]["item"][0]["IP"]
        hosts[dev["Name"]] = ip
    return hosts
```

KODU SATIR SATIR OKUYALIM

1–2 RIS sorgusu aynıdır; `...` yer tutucudur.

3 RIS cevabındaki her device için…

4 …device'ın IP adresini alır: `IPAddress` bir list'in içinde bir `item` listesi, ilk elemanın `IP` alanı.

5 `{device adı: IP adresi}` sözlüğüne ekler.

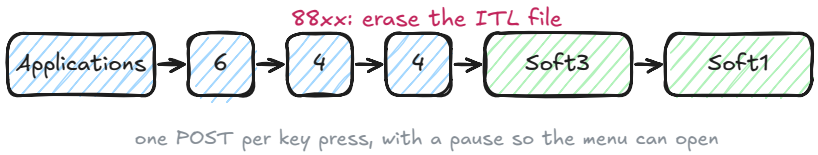
6 Sözlük geri verilir.

Ön koşul	Nerede
Web Access etkin	CUCM'de phone konfigürasyonu
phone'a sahip bir end user	kullanıcının **Controlled Devices** listesi
phone adresi	RIS'ten (`IPAddress`), asla elle yazılmaz

Phone, girişi kontrol etmesi için CUCM'e sorar; bu yüzden phone'a sahip normal bir end user yeterlidir. Cevap küçük bir XML belgesidir: `<ResponseItem … Status="0"/>` başarı demektir.

Pressing keys remotely

Her `ExecuteItem` tek bir eylemdir. Bir tuş dizisi bir list'tir; sırayla, kısa bekleme araları vererek, her biri ayrı bir `POST` ile gönderilir: tam olarak bir insanın menüde yürümesi gibi.



Şekil 2.30. Bir tuş dizisi: her eleman ayrı bir istek; aralarda kısa bir bekleme.

```python
import time
import xml.etree.ElementTree as ET
import requests

def press(host, item, wait=1):
    xml = f'<CiscoIPPhoneExecute><ExecuteItem URL="{item}"/>' \
          '</CiscoIPPhoneExecute>'
    r = requests.post(f"http://{host}/CGI/Execute",
                      data={"XML": xml}, auth=auth, timeout=10)
    r.raise_for_status()
    time.sleep(wait)                      # menü açılsın
    return ET.fromstring(r.content).find("ResponseItem") \
             .get("Status") == "0"

ITL_88XX = ["Key:Applications", "Key:KeyPad6", "Key:KeyPad4",
            "Key:KeyPad4", "Key:Soft3", "Key:Soft1"]
for name, host in phone_hosts(phones).items():   # RIS'ten
    ok = all(press(host, key) for key in ITL_88XX)
    print(name, "done" if ok else "check manually")
```

KODU SATIR SATIR OKUYALIM

5 `press()` tek bir tuşa basar. `wait` basıştan sonra beklenecek saniye; menünün açılması için.

6–7 İstek XML'i bir f-string ile kurulur. Sondaki `\` satırın devam ettiğini söyler.

8–9 XML, `XML` form alanı olarak gönderilir. `auth` phone'a sahip end user'ın kimlik bilgisidir.

10 HTTP hatası varsa exception.

11 Menünün açılması için bekleme.

12–13 Cevap XML'ini ayrıştırır, `ResponseItem`'ı bulur ve `Status` değeri `"0"` ise (başarı) `True` döndürür.

15–16 Bir 88xx phone'da belirli bir menü yolunu (ITL silme) tarif eden tuş sırası.

17 RIS'ten alınan her phone için…

18 `all(...)`: dizideki **her** tuş başarıyla basılırsa `True`. Biri başarısız olursa daha fazla denemez.

19 Sonuç: `done` ya da elle kontrol edilmesi gereken phone.

Eleman	Eylem
`Key:Settings` `Key:Applications`	sabit tuşlar
`Key:KeyPad0` … `Key:KeyPad9`	tuş takımı
`Key:Soft1` … `Key:Soft4`	ekranın altındaki soft key'ler
`Dial:1002`	bir call başlat

> **! DIKKAT**
> Menü yolları model ve firmware'e göre değişir: her model için ayrı bir tuş listesi tutun ve önce tek bir phone'da deneyin. Script ekranı göremediği için kaçırılan bir adım fark edilmez (sonraki konu).

Screenshots · CGI/Screenshot

`/CGI/Screenshot` adresine bir `GET`, o anki ekranı bir resim olarak döndürür. `r.content` ham `bytes`'ı tutar ve doğrudan bir `.png` dosyasına yazılır. Tuşlara basmanın ne yaptığını **doğrulamak** için ya da bir sorunu masaya gitmeden belgelemek için kullanın: "line register mı?", "ekranda hangi hata var?"

```
from pathlib import Path

def screenshot(name, host):
    r = requests.get(f"http://{host}/CGI/Screenshot",
                     auth=auth, timeout=10)
    r.raise_for_status()
    out = Path("shots") / f"{name}.png"
    out.parent.mkdir(exist_ok=True)
    out.write_bytes(r.content)     # bytes, 1.3.10
    return out

for name, host in phone_hosts(phones).items():
    try:
        print(screenshot(name, host))
    except requests.exceptions.RequestException as e:
        print(name, "failed:", e)
# shots/SEP0A1B2C3D4E5F.png
```

KODU SATIR SATIR OKUYALIM

1 `pathlib.Path` yol işlemleri için (1.8).

3–6 Ekran görüntüsü istenir ve hata kontrol edilir.

7 Dosya yolu `shots/<phone adı>.png` olarak kurulur. `/` operatörü yol parçalarını birleştirir.

8 `mkdir(exist_ok=True)`: `shots` klasörü yoksa yaratır, varsa hata vermez.

9 `write_bytes()` ham byte'ları dosyaya yazar. Resim **metin değil, bytes**'tır (1.3.10).

12–16 Her phone için bir `try`: bir phone başarısız olursa diğerleri devam eder. `RequestException`, `requests`'in bütün hatalarının ortak atasıdır.

▸ **COLLAB DÜNYASINDA**
Toplu bir ITL silme işleminden sonra her phone'un ekran görüntüsünü alın ve klasörü gözden geçirin: tek bakışta hangi phone'ların beklenen menüye ulaşamadığını görürsünüz.

⋆ **İPUCU**
Çalışan bir referans: kurs repository'sindeki `erase_itl.py`, 69xx, 78xx, 88xx ve 99xx modelleri için test edilmiş tuş dizilerini içerir.

Özetle

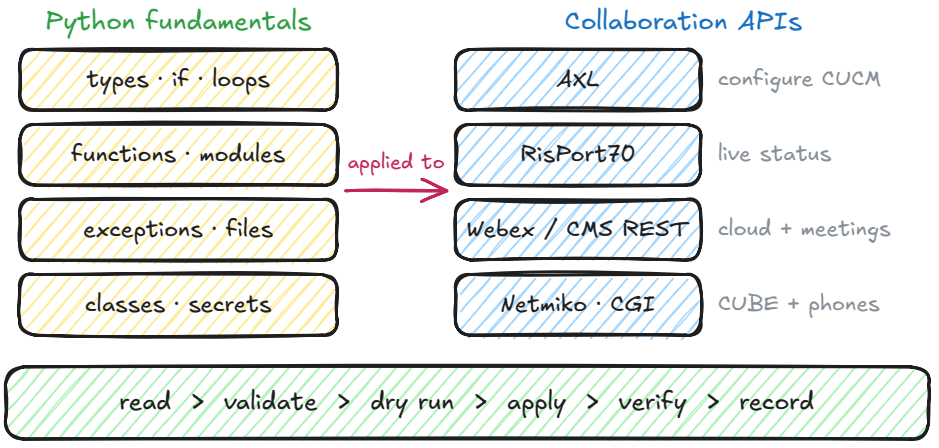
- Her Cisco IP phone, `/CGI/Execute` ile tuşlara basmaya ve `/CGI/Screenshot` ile ekran görüntüsü almaya izin veren bir web server çalıştırır.
- Phone adreslerini RIS'ten alın. Her tuş bir `POST`; aralarda bekleyin.
- Script ekranı göremez: sonucu bir screenshot ile doğrulayın.

✎ **Kendin dene**
`press(host, "Key:Settings")` çağrısını tek bir lab phone'unda deneyin. Ardından bir screenshot alıp phone ekranının gerçekten değiştiğini kontrol edin.

2.9 Wrap-up

What you can do now

İlk `print`'ten başlayıp bir collaboration cluster'ını baştan sona okuyan, değiştiren ve kontrol eden script'lere geldiniz. Şimdi yapabildikleriniz üç başlıkta toplanıyor:



Şekil 2.31. Kitabın haritası: Python dili, kapılar (API'ler) ve güvenli çalışma biçimi.

- **Dil olarak Python:** veri, kararlar, loop'lar, function'lar, dosyalar ve hatalar.
- **Kapılar olarak API'ler:** CUCM'de AXL ve RIS, Webex ve CMS'te REST, CUBE'ta SSH, her phone'daki web server.

- **Güvenli bir çalışma biçimi:** gizli bilgiler kodun dışında, önce dry run, her değişiklikten sonra doğrulama, bir kayıt tutmak.

> ▸ **COLLAB DÜNYASINDA**
> Küçük başlayın: her sabah çalışan salt-okunur bir rapor. Sonraki her script aynı bağlantıyı, aynı loop'u ve aynı hata yönetimini yeniden kullanır.

Kitabın özeti

- Python'da her şey bir object'tir; variable bir etikettir. Tipi bilmek, davranışı bilmektir.
- Loop ve function, tekrar eden işi bir kere yazmanızı sağlar. Hataları yakalayın, toplu işte durmayın.
- Her API aynı üç adımı ister: bağlan, iste, cevabı ayrıştır. `requests` REST için, `zeep` SOAP için, Netmiko CLI için.
- Yazmadan önce oku, yazarken kaydet, yazdıktan sonra doğrula.

Teşekkürler

Sorular, fikirler, bir sonraki yazmak istediğiniz script'ler?

Pratik yapmaya devam edin

- Haftalık elle yaptığınız bir işi bir script'e çevirin.
- Ona `--apply`, bir dry run ve bir sonuç CSV'si ekleyin.
- Ekibinizle bir git repository'sinde paylaşın.

Kaynaklar

- Kurs script'leri: `github.com/ilkermansur/collab_automation`
- AXL ve RisPort70: `developer.cisco.com/docs/axl`
- Webex API'leri: `developer.webex.com`
- Python dokümantasyonu: `docs.python.org/3`